

EN3150 Pattern Recognition
Deep Neural Networks

Most of the contents are based on

1. C. M. Bishop, Pattern Recognition and Machine Learning.
2. Kevin P. Murphy, Machine Learning: A Probabilistic Perspective.
3. Prof. Svetlana Lazebnik, University of Illinois at Urbana-Champaign.
4. CS231n: Deep Learning for Computer Vision-Stanford.
5. Prof. Justin Johnson, University of Michigan.

Presented by M. T. U. Sampath K. Perera,
Department of Electronic and Telecommunication Engineering,
University of Moratuwa.
(sampathk@uom.lk).
Semester 5 – Batch 21.

Recap

Linear regression

$y(\mathbf{x}) = w_0 + w_1 x_{1,i} + \cdots, + w_D x_{D,i} + \epsilon = \mathbf{w}^T \boldsymbol{x_i} + \epsilon$

Polynomial regression

$y(\mathbf{x}) = \mathbf{w}^T \phi(\boldsymbol{x_i}) + \epsilon$. For two feature $D = 2$, $\phi(\boldsymbol{x_i}) = \left[1, x_{1,i}, x_{2,i}, x_{1,i}^2, x_{1,i}x_{2,i}, x_{2,i}^2\right]$

$$\boldsymbol{x_i} = \begin{bmatrix} 1 \\ x_{1,i} \\ \cdot \\ \cdot \\ \cdot \\ x_{D,i} \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \\ \cdot \\ \cdot \\ \cdot \\ w_D \end{bmatrix}$$

Logistic regression

$p(y = C_1|\boldsymbol{x_i}, \mathbf{w}) = \text{sigm}\ (\mathbf{w}^T \boldsymbol{x_i}))$

$p(y = C_1|\boldsymbol{x_i}, \mathbf{w}) = \text{sigm}\ (\mathbf{w}^T \phi(\boldsymbol{x_i}))$

Multiclass logistic regression

$p(y_i = C_k|\boldsymbol{x_i}, \mathbf{W}) = \frac{\exp(w_{0k}+w_{1k}x_{1,i}+\cdots+w_{Dk}x_{D,i})}{\sum_j \exp(w_{0k}+w_{1k}x_{1,i}+\cdots+w_{Dk}x_{D,i})} = \frac{\exp(\boldsymbol{w}_k^T \boldsymbol{x}_i)}{\sum_j \exp(\boldsymbol{w}_j^T \boldsymbol{x}_i)} = \text{Cat}\ (y_i|\text{softmax}\ (\widetilde{\boldsymbol{W}}\boldsymbol{x}_i + \boldsymbol{b}))$.

$p(y_i = C_k|\boldsymbol{x_i}, \mathbf{W}) = \frac{\exp(\boldsymbol{w}_k^T \phi(\boldsymbol{x_i}))}{\sum_j \exp(\boldsymbol{w}_j^T \phi(\boldsymbol{x_i}))} = \text{Cat}\ (y_i|\text{softmax}\ (\widetilde{\boldsymbol{W}}\phi(\boldsymbol{x_i}) + \boldsymbol{b}))$.

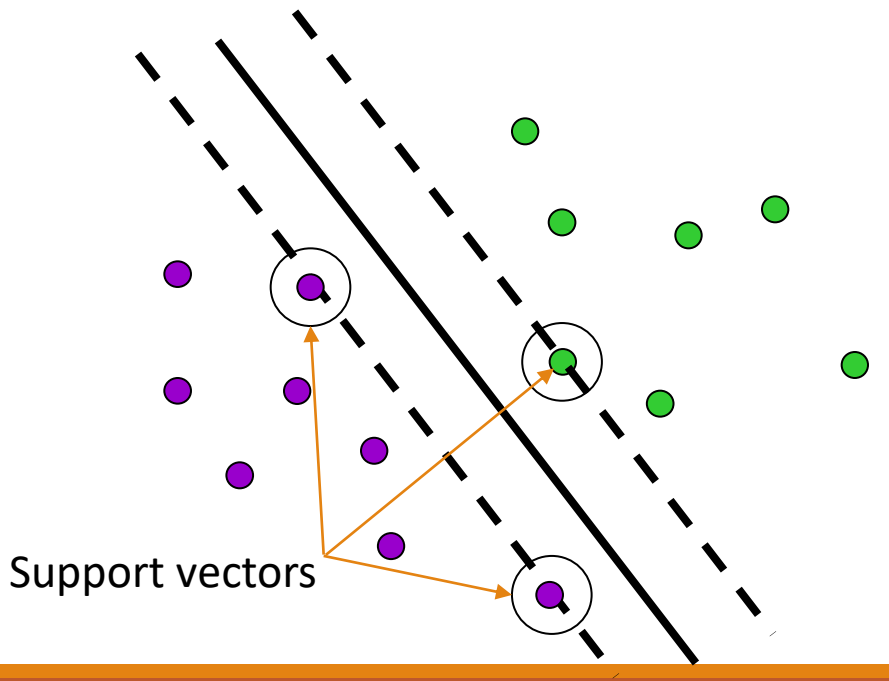
$$\boldsymbol{y}(\boldsymbol{x}, \boldsymbol{w}) = f\left(\sum_{j=0}^{D} w_j \Phi_j(x_j)\right)$$

Recap

- Linear combinations of **predefined basis** functions.
- For large-scale problems ➔ it needs to customize the basis functions to suit the data.
- SVM address this issue to some extend
  - SVMs define basis functions centered on training data points.
  - During training, they select a **subset of these basis functions**.
  - Number of basis functions is typically **much smaller** than the number of training points
  - A key advantage is that SVMs use convex optimization for training.
  - This makes the optimization problem relatively straightforward.

Decision boundary

$$\sum_i \alpha_i y_i \varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}) + b = \sum_i \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$



Feed-forward Network

Alternative approach

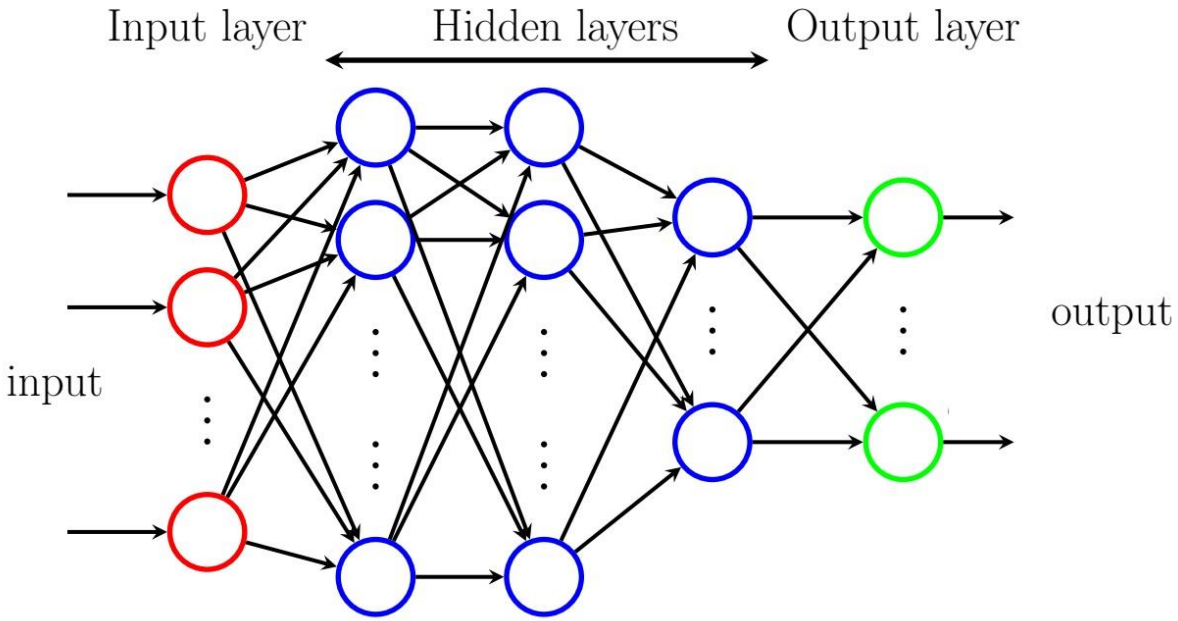
- Fix the number of basis functions.
- Parameters are not fixed and adjusted during training.
- Most successful pattern recognition model of this type is the feed-forward neural network, commonly known as the multilayer perceptron (MLP).

The term "neural network" originated from efforts to mathematically describe information processing in biological systems, with contributions from researchers like McCulloch and Pitts (1943), Widrow and Hoff (1960), Rosenblatt (1962), and Rumelhart et al. (1986).

What is Neural networks (NNs)

- "Neural Network or Artificial Neural Network" is a system that inspired by the structure and function of human brain and mimicking the way that biological neurons signal to one another [1]
- ANN can be shallow or deep (more than one hidden layer)

- Feedforward ANNs
  - Data flows from input to output layers without feedback loops

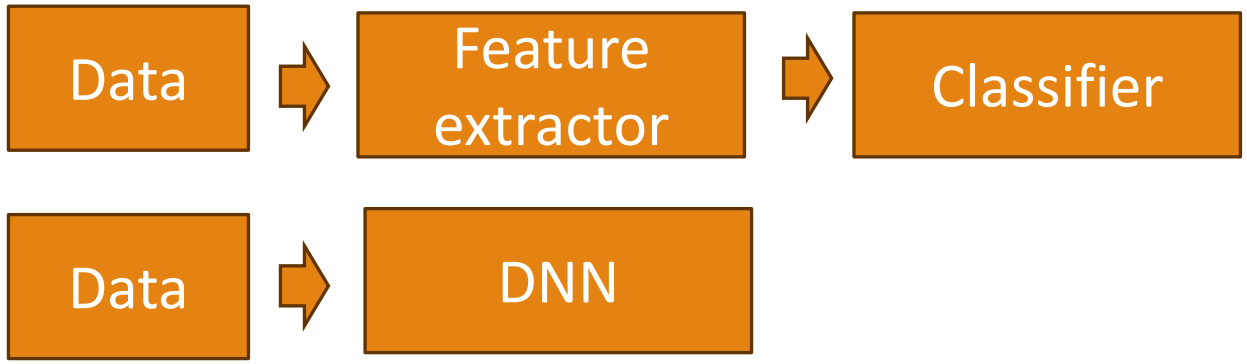


[1] What are Neural Networks? | IBM

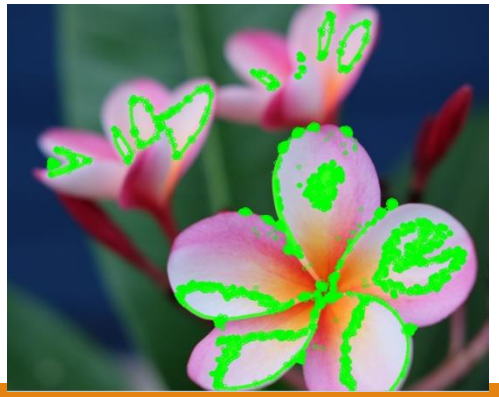
What is Neural networks (NNs)

- ANN can be shallow or deep (more than one hidden layer)
- Deep Neural Network (DNN)
  - Multiple hidden layers, typically more than one or two
  - **Automatically discover** (learn) features (representations)





Feature extractor
(**corner detection**)



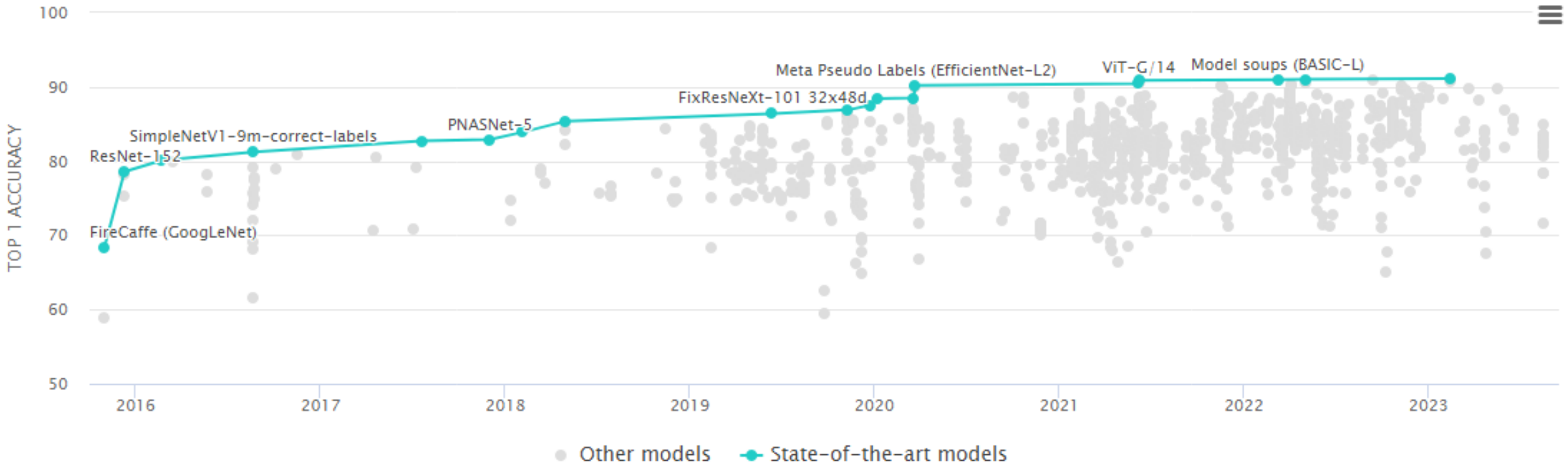
[1] What are Neural Networks? | IBM

Applications of DNN

- **Image Recognition and Classification**
- **Natural Language Processing (NLP)**
- **Speech Recognition**
- **Autonomous driving**
- **Human Activity Recognition**
- **Anomaly Detection**
- **Quality Control in Manufacturing**

Image classification on ImageNet

ImageNet (image-net.org)



ImageNet Benchmark (Image Classification) | Papers With Code

Image classification on ImageNet

Rank	Model	Top 1 Accuracy	Number of params	GFLOPs	Top 5 Accuracy	Continual Weighted Accuracy	Extra Training Data	Paper
1	**BASIC-L** (Lion, fine-tuned)	91.1%	2440M				×	Symbolic Discovery of Optimization Algorithms
2	**CoCa** (finetuned)	91.0%	2100M				×	CoCa: Contrastive Captioners are Image-Text Foundation Models
3	**Model soups** (BASIC-L)	90.98%	2440M				×	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time
4	**Model soups** (ViT-G/14)	90.94%	1843M				×	Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time
5	**ViT-e**	90.9%	3900M				×	PaLI: A Jointly-Scaled Multilingual Language-Image Model
6	**CoAtNet-7**	90.88%	2440M	2586			✓	CoAtNet: Marrying Convolution and Attention for All Data Sizes
7	**ViT-G/14** (Lion)	90.71%	1843M				×	Symbolic Discovery of Optimization Algorithms
8	**CoCa** (frozen)	90.60%	2100M				×	CoCa: Contrastive Captioners are Image-Text Foundation Models

ImageNet Benchmark (Image Classification) | Papers With Code

History of Neural Networks

1943: McCulloch and Pitts: Proposed the first mathematical model of a neural network, the McCulloch-Pitts neuron, which laid the foundation for artificial neural networks.

1950s : Frank Rosenblatt: Developed the perceptron, a single-layer neural network capable of linear classification, and introduced the concept of supervised learning.

1960s : Marvin Minsky and Seymour Papert: Published a book demonstrating the limitations of perceptrons, leading to a decrease in interest in neural networks.

1980s: The rediscovery of the backpropagation algorithm for training multi-layer neural networks rekindled interest in artificial neural networks.

1990s Yann LeCun : introduced convolutional NNs for image and pattern recognition tasks. Achieves state-of-the-art performance.

2000s: Hinton, Bottou, Bengio, LeCun, Ng, and others keep trying stuff with deep networks but without much success in vision.

2010-2011: Substantial progress.

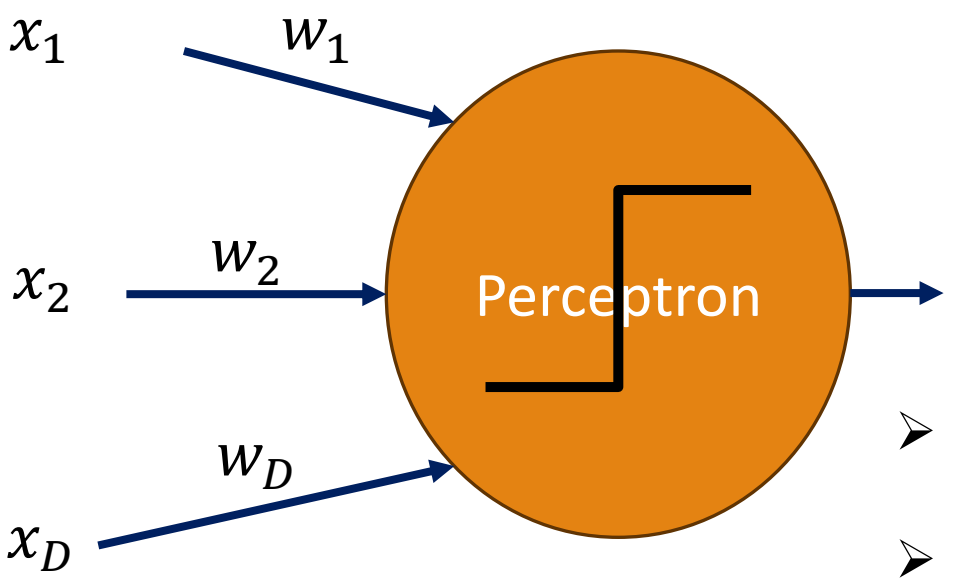
2012 - ImageNet Challenge: Geoffrey Hinton's team achieved a significant breakthrough in the ImageNet Large Scale Visual Recognition Challenge using deep convolutional neural networks, sparking a deep learning revolution.

In 2015: RNNs with architectures like Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRUs) gained prominence for sequence modeling.

Neural Network types

- **Feedforward neural networks (the basic ANNs, also known as MLPs)**
- **Convolutional neural networks (CNNs)**
- Recurrent neural networks (RNNs)
- Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU)
- Autoencoder
- Variational Autoencoder (VAE)
- Generative Adversarial Network (GAN)
- Radial Basis Function Network (RBFN)

The Perceptron



$$f(a) = \begin{cases} +1, & \text{if } a \geq 0 \\ -1, & \text{if } a < 0 \end{cases}$$

For binary classes

$$y = \text{sgn}(b + w_1 x_1 + \cdots, + w_D x_D) = \text{sgn}(\boldsymbol{w} \cdot \boldsymbol{x} + b)$$

For K classes

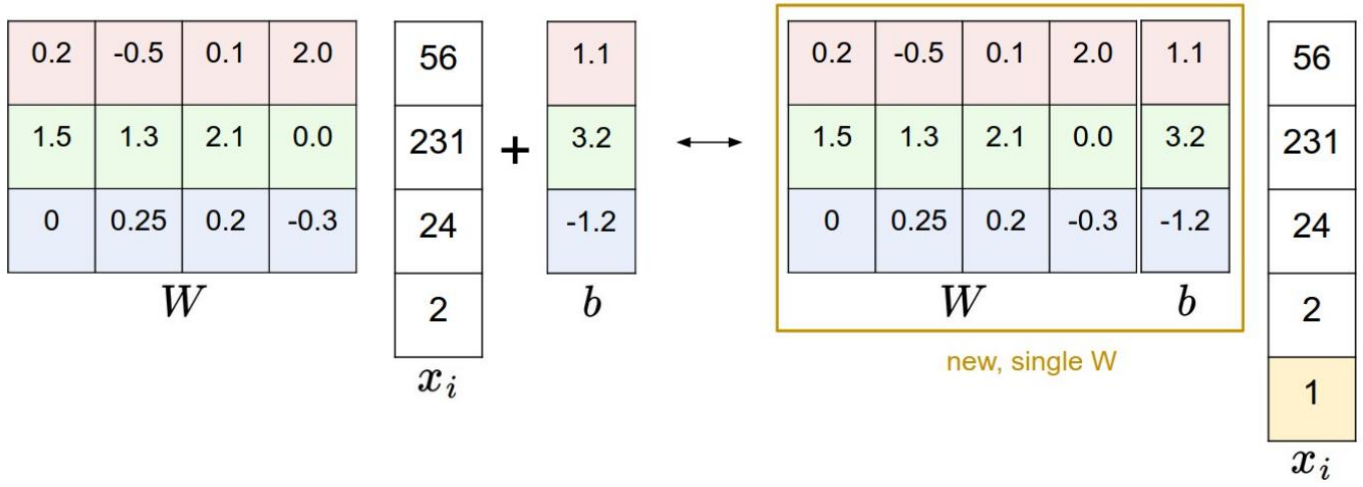
$$\boldsymbol{x} \in \mathbb{R}^D, \boldsymbol{W} \in \mathbb{R}^{K \times D}, y = \text{sgn}(\boldsymbol{W} \cdot \boldsymbol{x} + \boldsymbol{b})$$

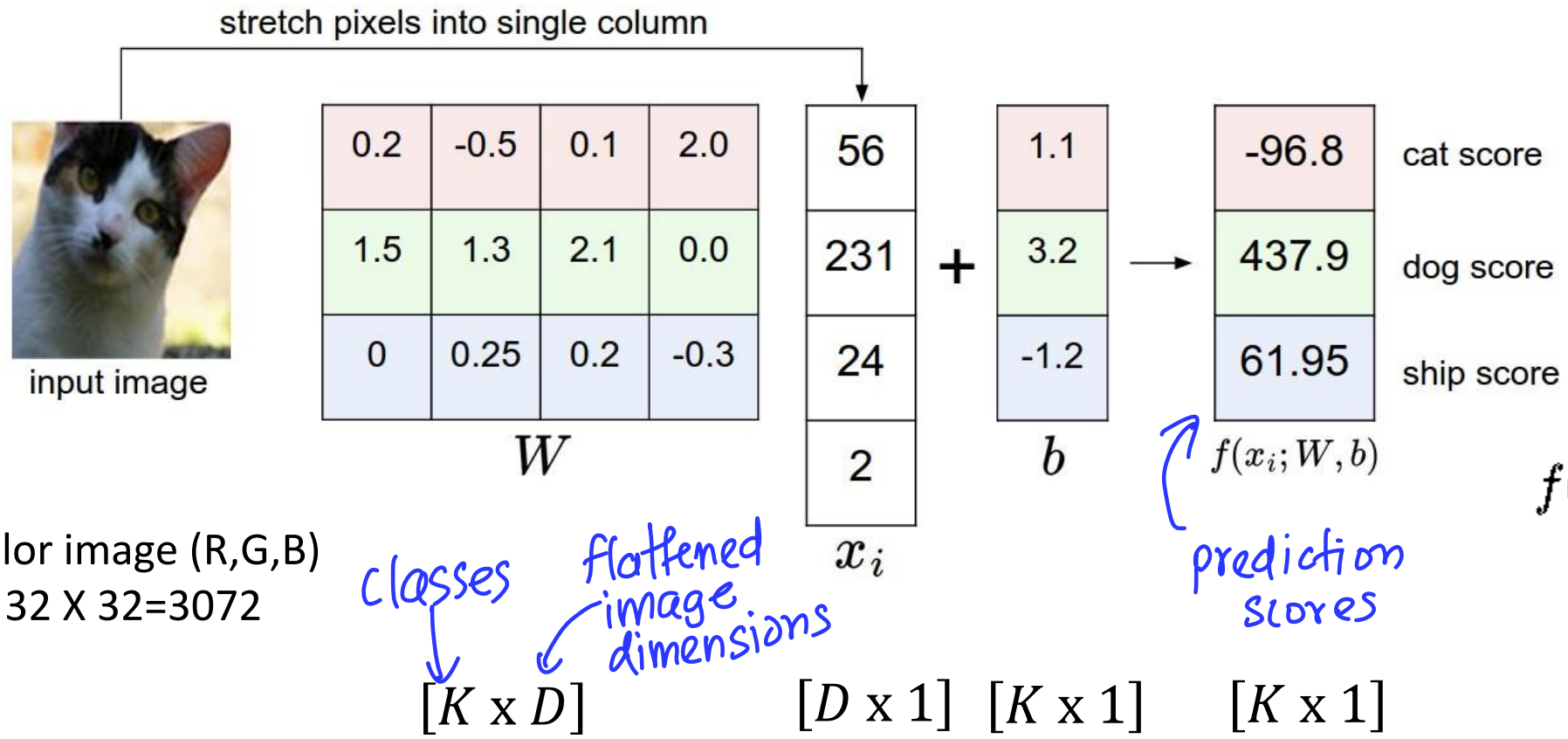
- Developed by Frank Rosenblatt in the late 1950s and was one of the earliest models for machine learning
- Perceptron has one input layer and one neuron
- Primary use is for binary classification tasks.
- Can only solve linearly separable problems (not powerful).

Rosenblatt, Frank. "The perceptron: a probabilistic model for information storage and organization in the brain." *Psychological review* 65.6 (1958): 386.

Linear Classifier

https://cs231n.github.io/linear-classify/





- This set of weights, W, isn't effective.
- It gives our cat image a very low cat score.
- It seems to believe the image is of a dog.

32 Color image (R,G,B)
D=3X 32 X 32=3072

$$f(x_i, W, b) = Wx_i + b$$

Linear Classifier

Interpretation of linear classifiers as template matching

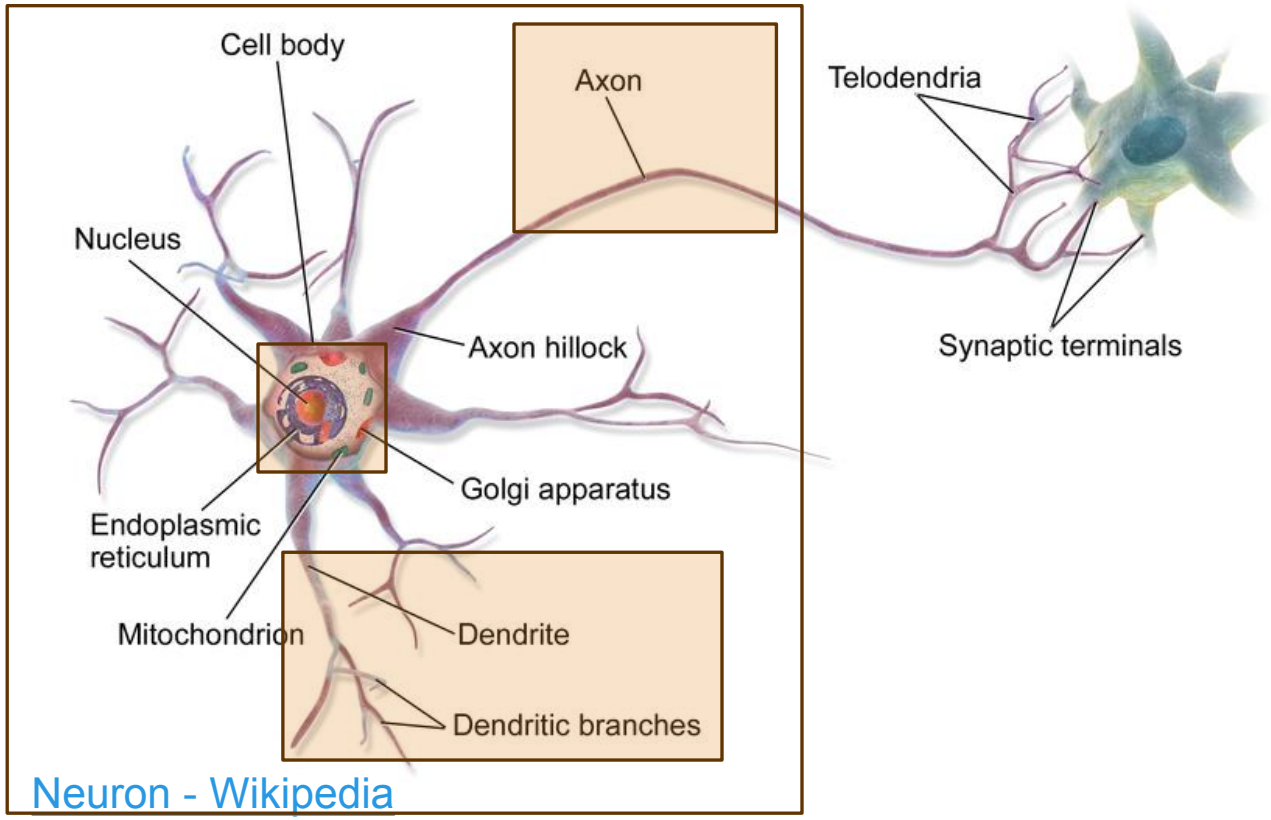
- Each row of weights W acts as a template (or prototype) for a class.
- The score for each class is determined by comparing the image to each template using the inner product (dot product).
- The linear classifier performs template matching with learned templates



- The horse template appears to feature a two-headed horse because it combines left- and right-facing horses from the dataset.
- The car classifier merges several variations into one template that identifies cars from all angles and colors
- The resulting template is predominantly red, indicating that red cars are more common in the CIFAR-10 dataset.
- The linear classifier is too weak to properly account for different-colored cars
- Neural networks can develop intermediate neurons in hidden layers to detect specific car types (e.g., green car facing left, blue car facing front).

https://cs231n.github.io/linear-classify/

Neuron (biological)



Neuron - Wikipedia

Fundamental building block of the nervous system in biological organisms.

Play a central role in transmitting and processing information in the form of electrical and chemical signals

Responsible for various functions, including sensing, processing, and transmitting information within the nervous system

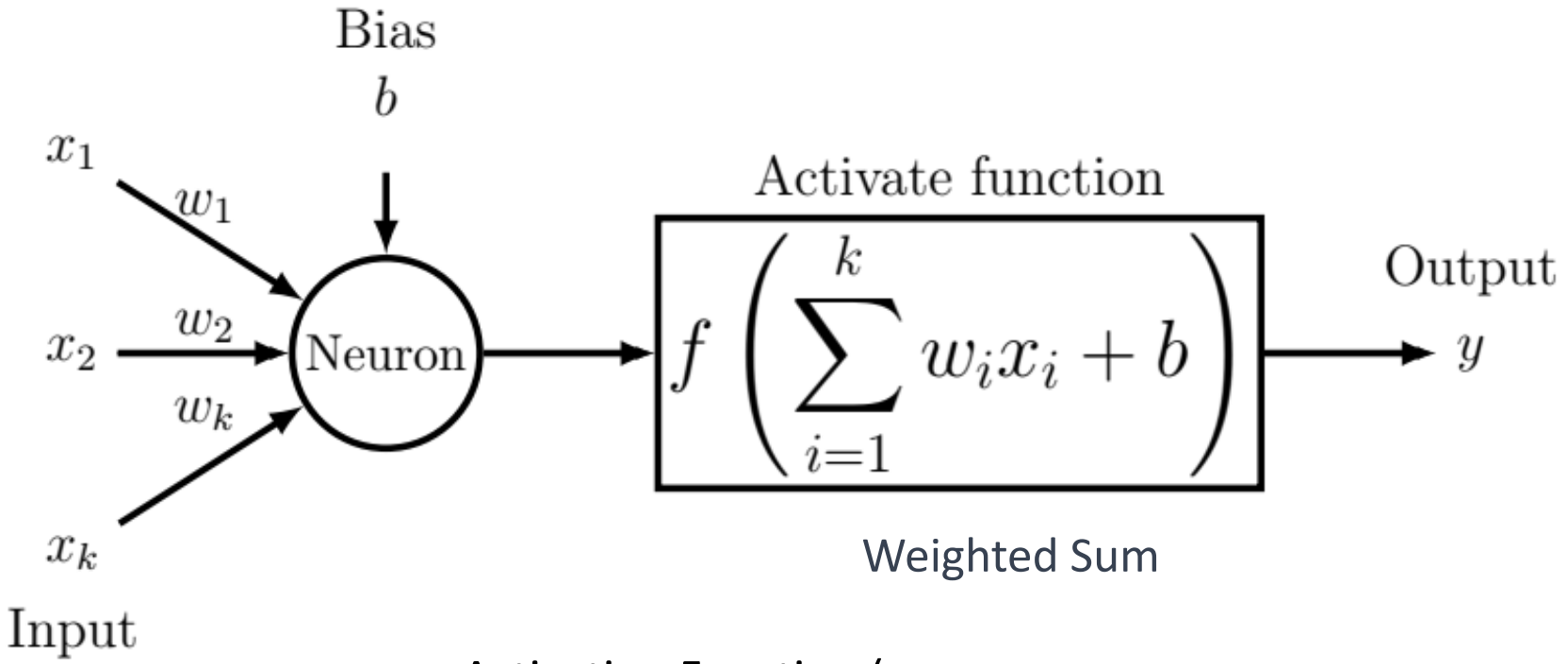
Dendrites: information is received to the neuron

Nucleus: processing the information

Axons: output of a neuron is passed to other neurons via the axon

Artificial Neuron

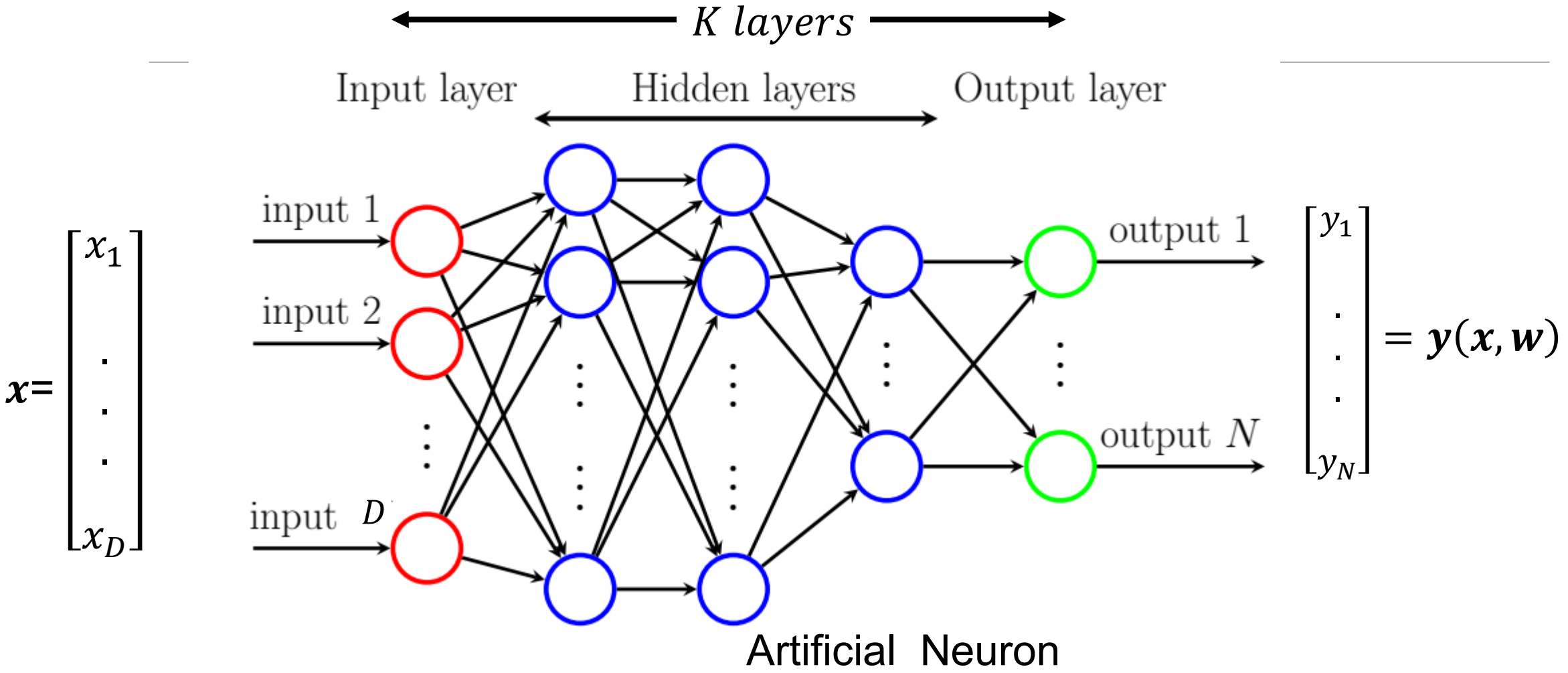
Artificial neurons, also known as nodes or artificial neurons, serve as the basic processing units in an ANN



Activation Function (introduces non-linearity)

Each input is associated with a weight

Feed-forward Network



Activation Functions

Sigmoid Function

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

ReLU (Rectified Linear Unit) Function

$$\mathrm{ReLU}(x) = \max(0, x)$$

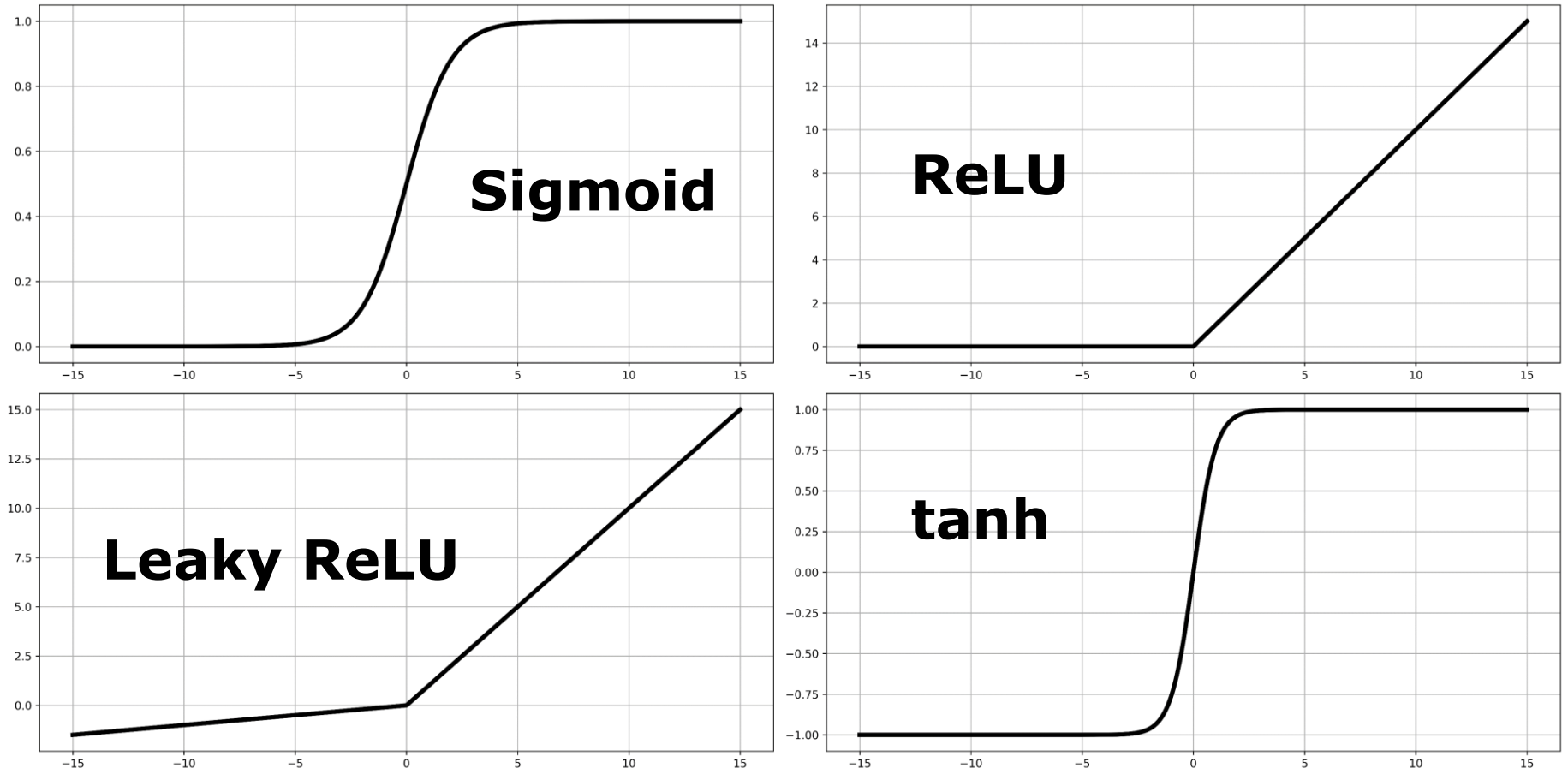
Hyperbolic Tangent (tanh) Function

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$

Leaky ReLU (LReLU)

$$\mathrm{LReLU}(x, \alpha) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

Activation Functions



Recap: Discriminant functions

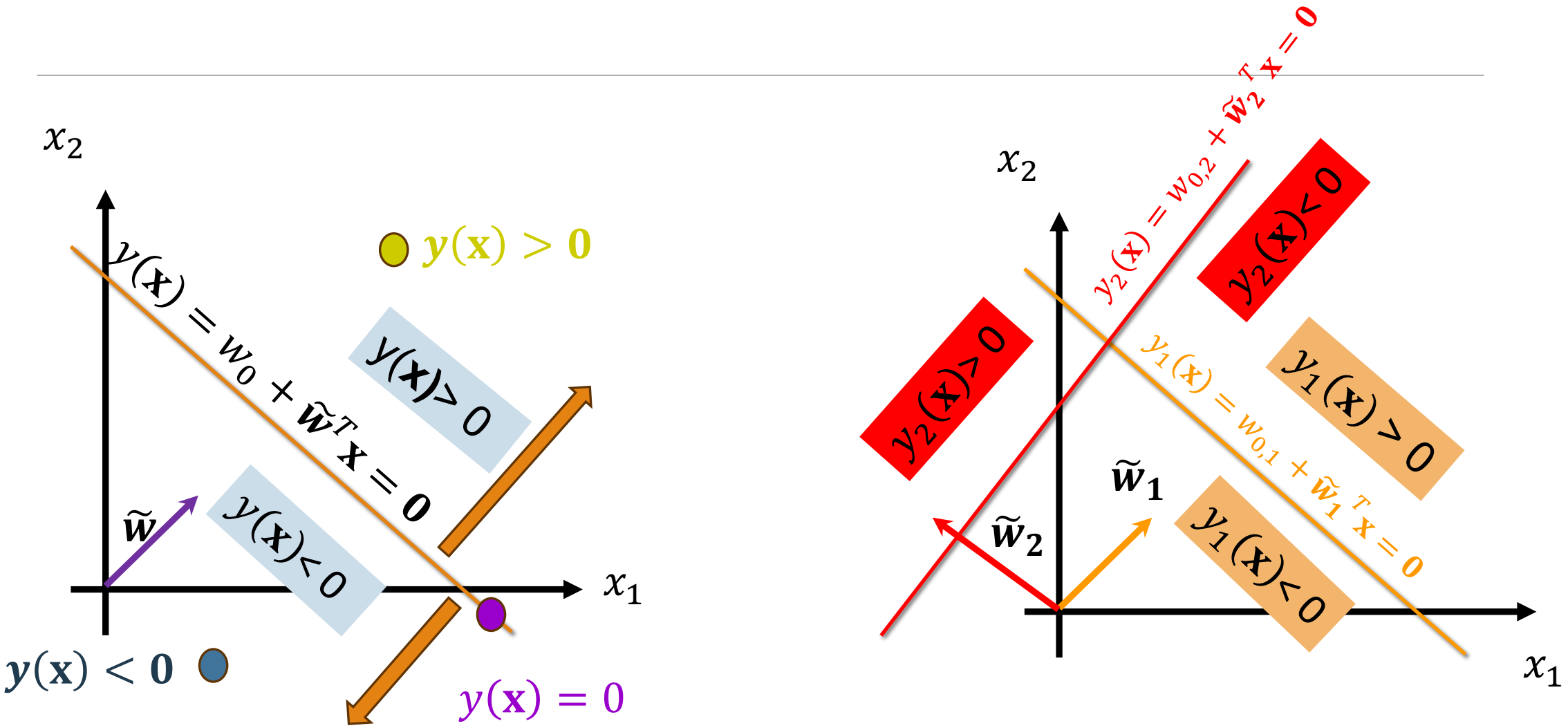
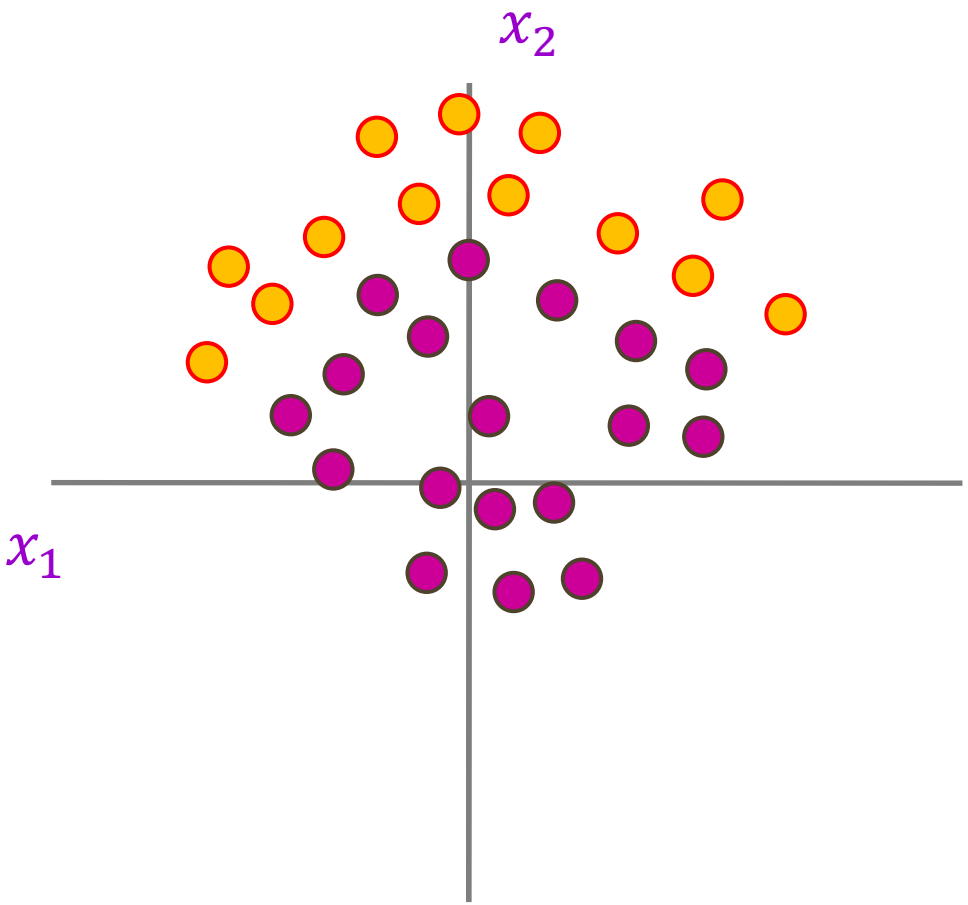


Image is adapted from C. M. Bishop, Pattern Recognition and Machine Learning, Springer NY, 2006

The power of nonlinearities

Points not linearly
separable in original space

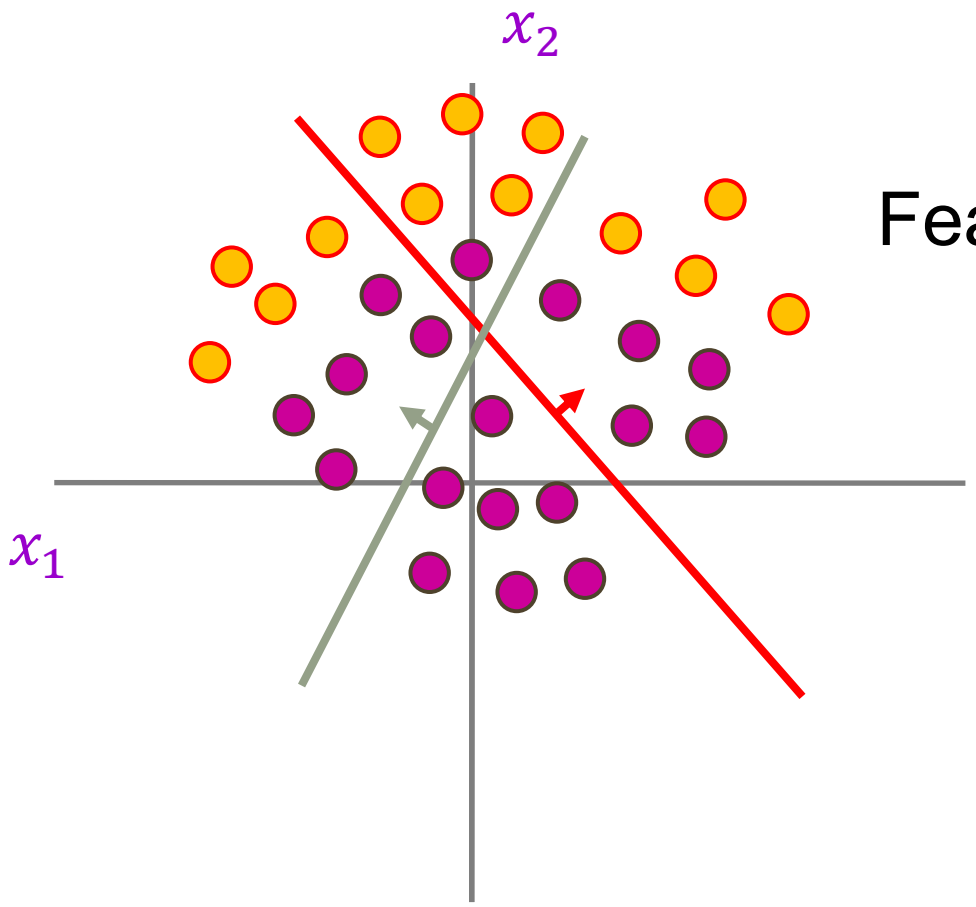


Source: J. Johnson

The power of nonlinearities

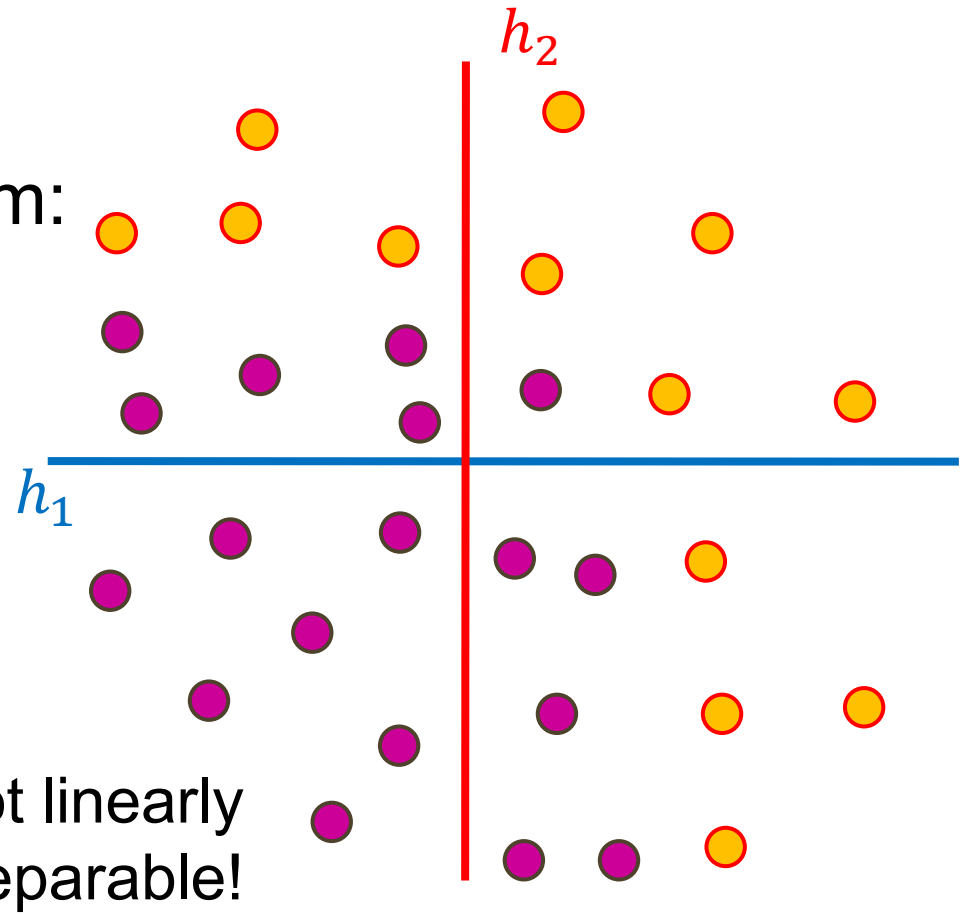
Points not linearly separable in original space

Consider a linear transform: $h = Wx + b$
Where x, h, b are 2-dimensional



Feature transform:

$h = Wx + b$



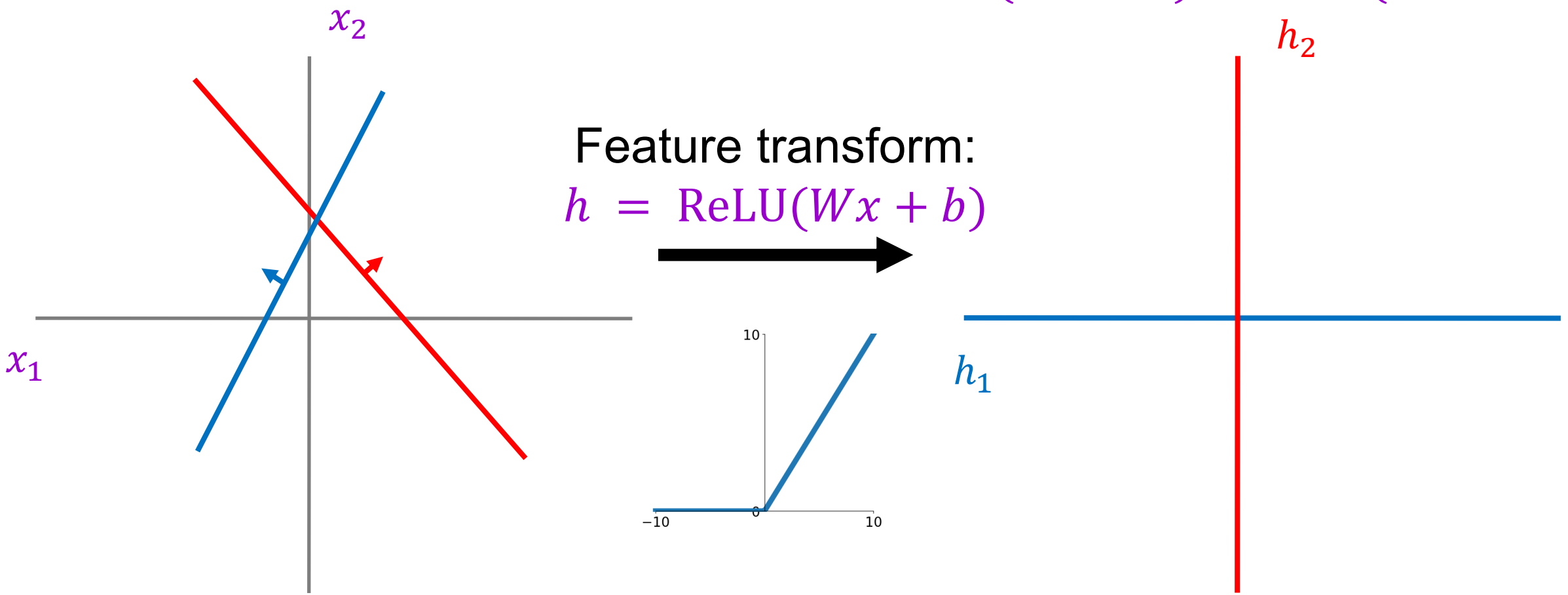
Still not linearly separable!

Source: J. Johnson

The power of nonlinearities

Let's add a nonlinearity:

$$h = \mathrm{ReLU}(Wx + b) = \max(0, Wx + b)$$

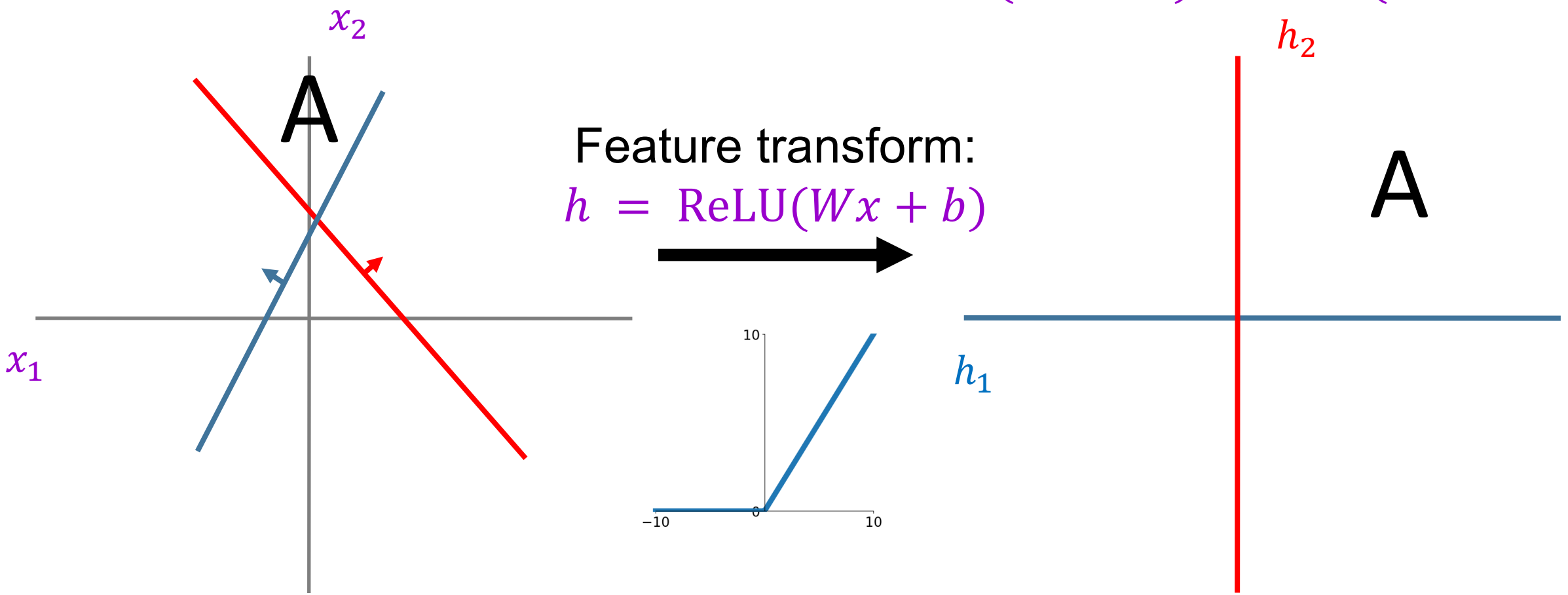


Source: J. Johnson

The power of nonlinearities

Let's add a nonlinearity:

$$h = \mathrm{ReLU}(Wx + b) = \max(0, Wx + b)$$

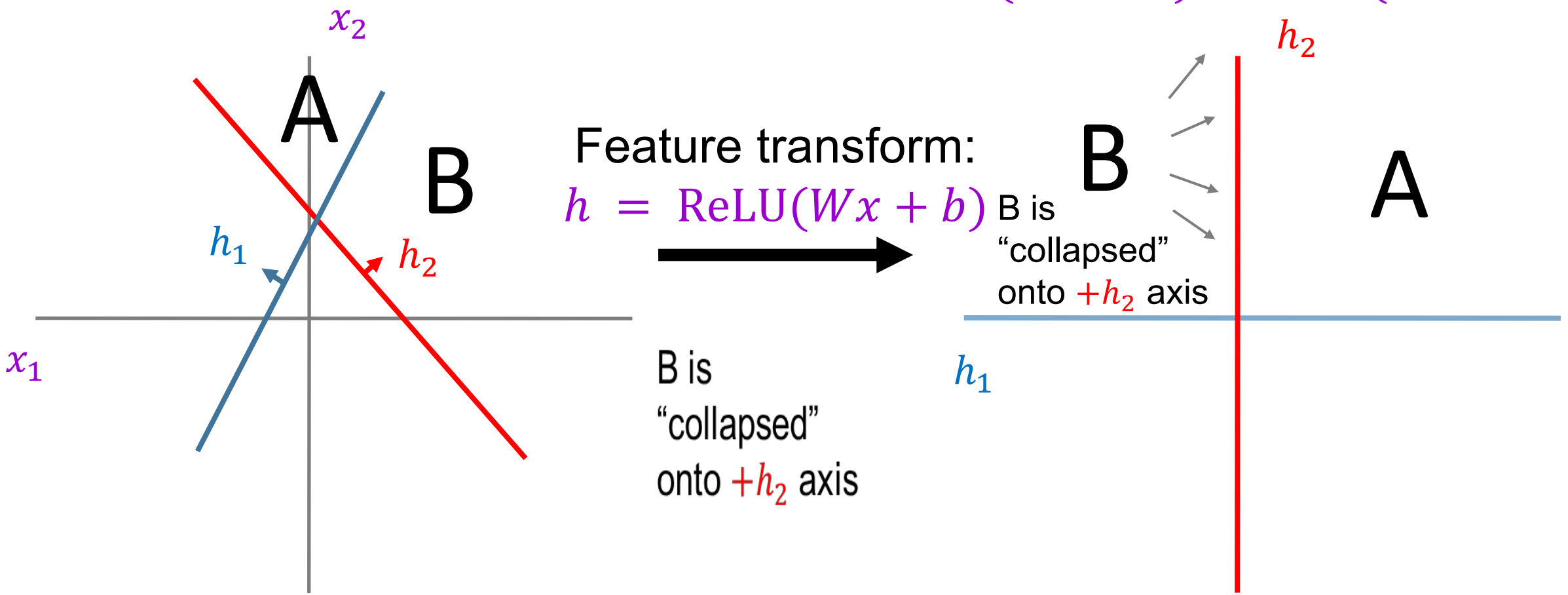


Source: J. Johnson

The power of nonlinearities

Let's add a nonlinearity:

$$h = \mathrm{ReLU}(Wx + b) = \max(0, Wx + b)$$

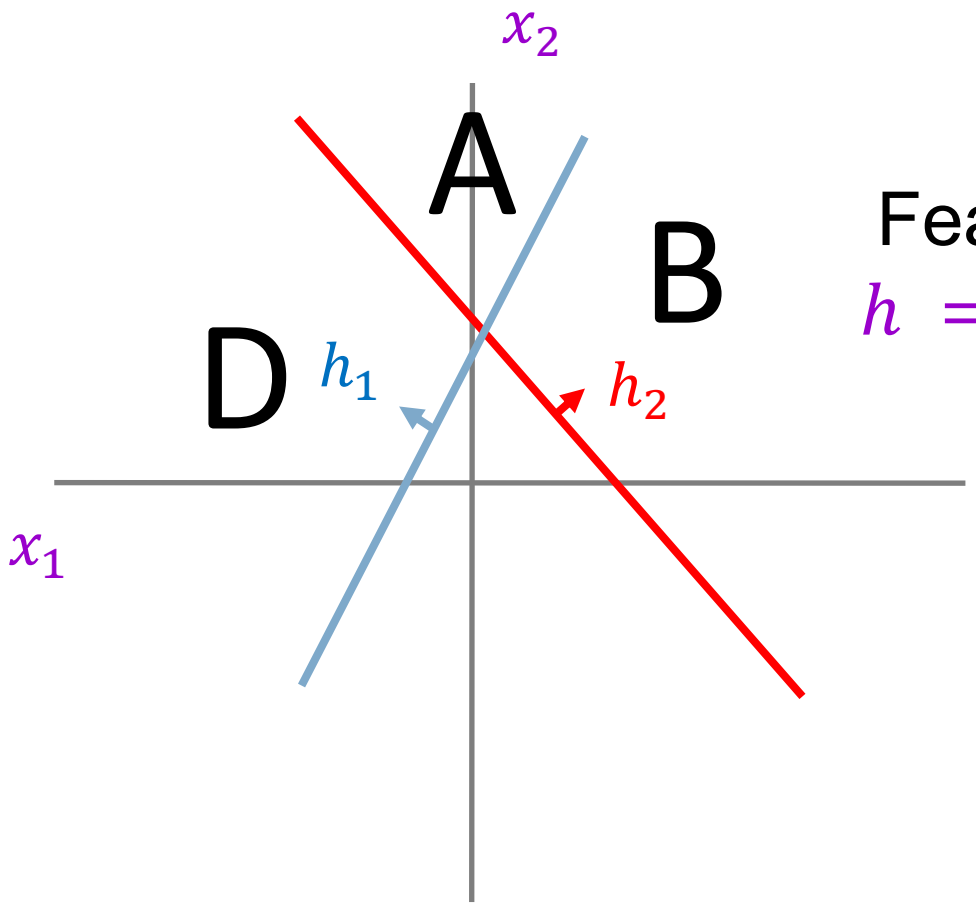


Source: J. Johnson

The power of nonlinearities

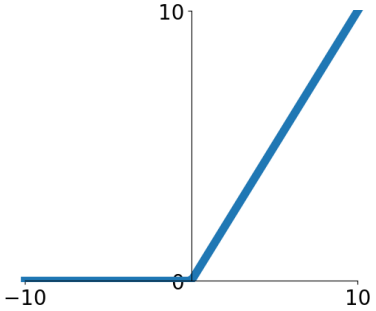
Let's add a nonlinearity:

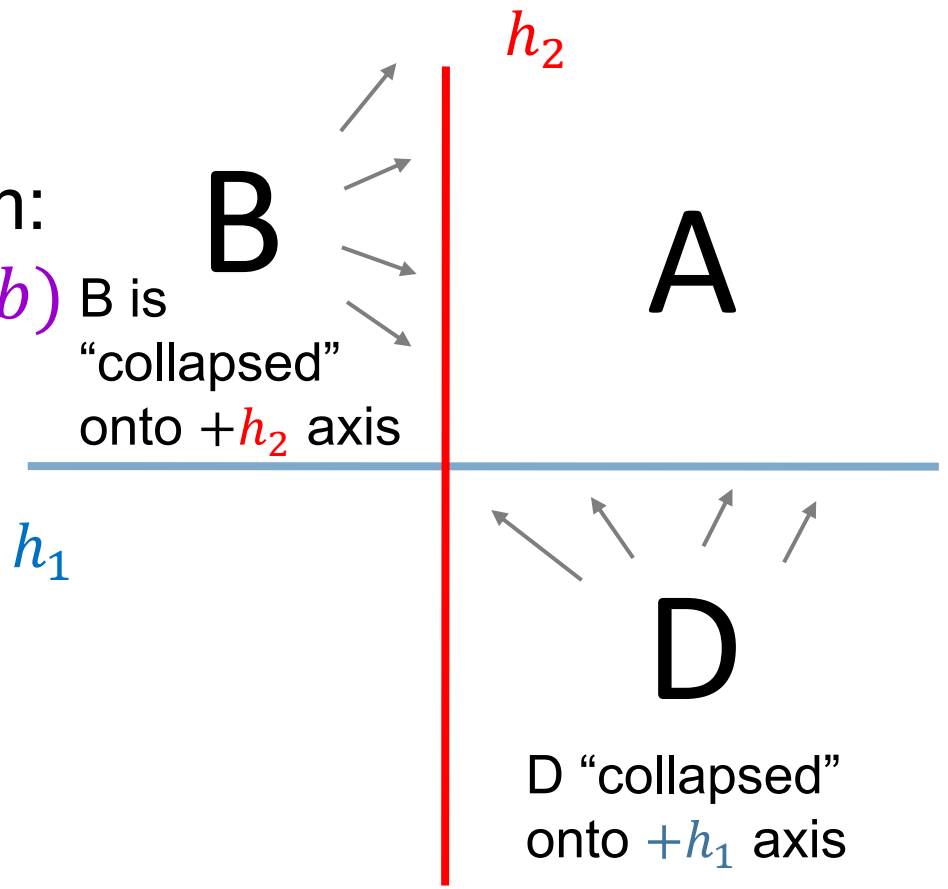
$$h = \mathrm{ReLU}(Wx + b) = \max(0, Wx + b)$$



Feature transform:

$$h = \mathrm{ReLU}(Wx + b)$$





B is "collapsed" onto $+h_2$ axis

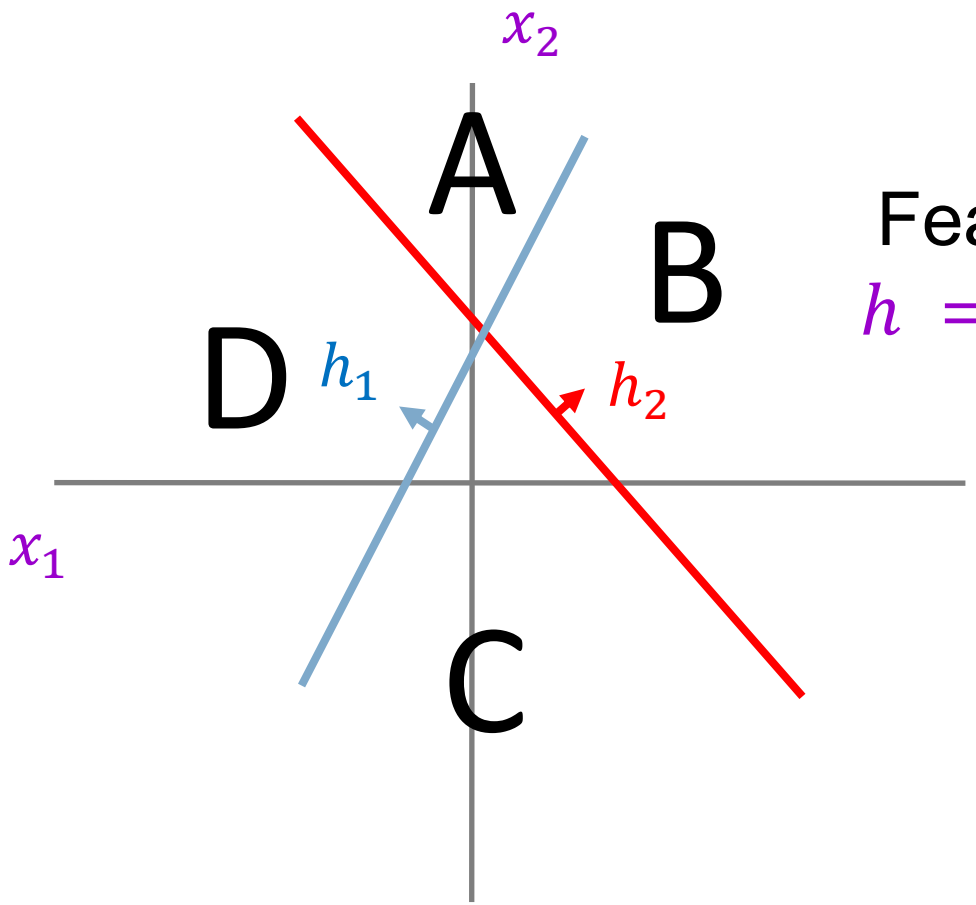
D "collapsed" onto $+h_1$ axis

Source: J. Johnson

The power of nonlinearities

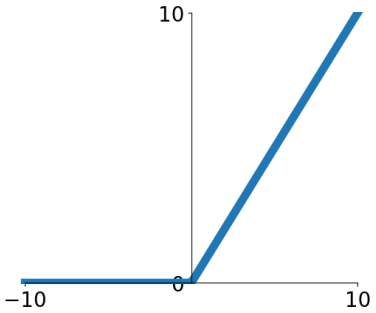
Let's add a nonlinearity:

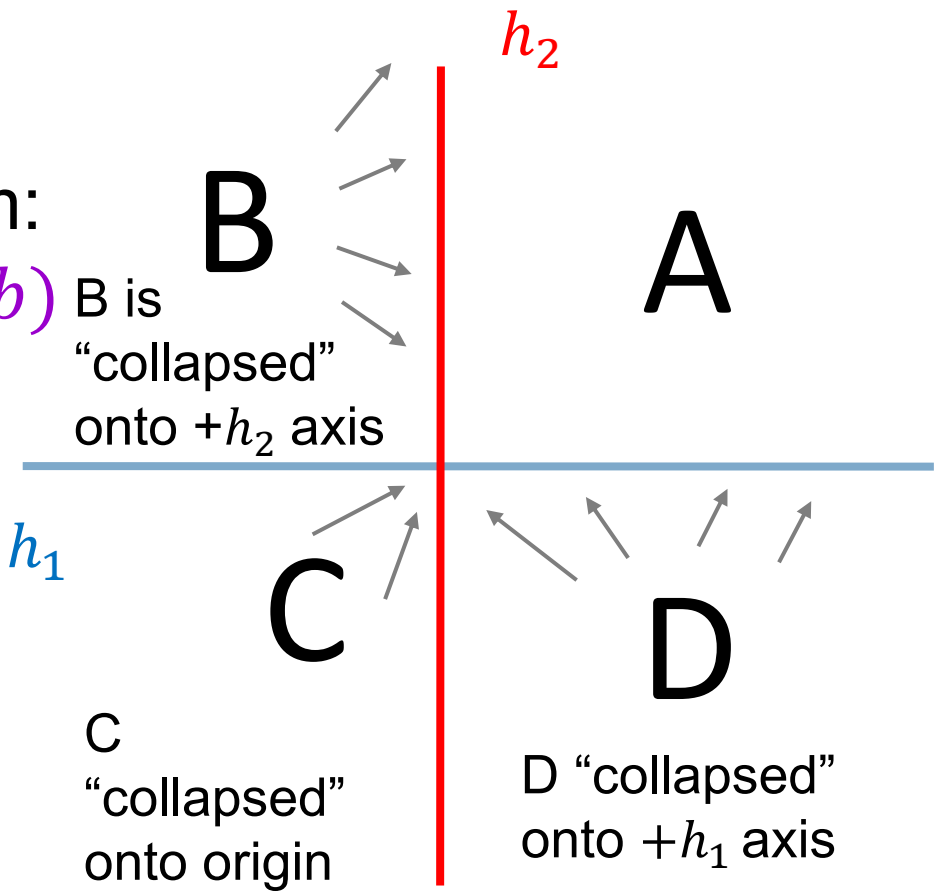
$$h = \mathrm{ReLU}(Wx + b) = \max(0, Wx + b)$$



Feature transform:

$$h = \mathrm{ReLU}(Wx + b)$$





B is "collapsed" onto $+h_2$ axis

C "collapsed" onto origin

D "collapsed" onto $+h_1$ axis

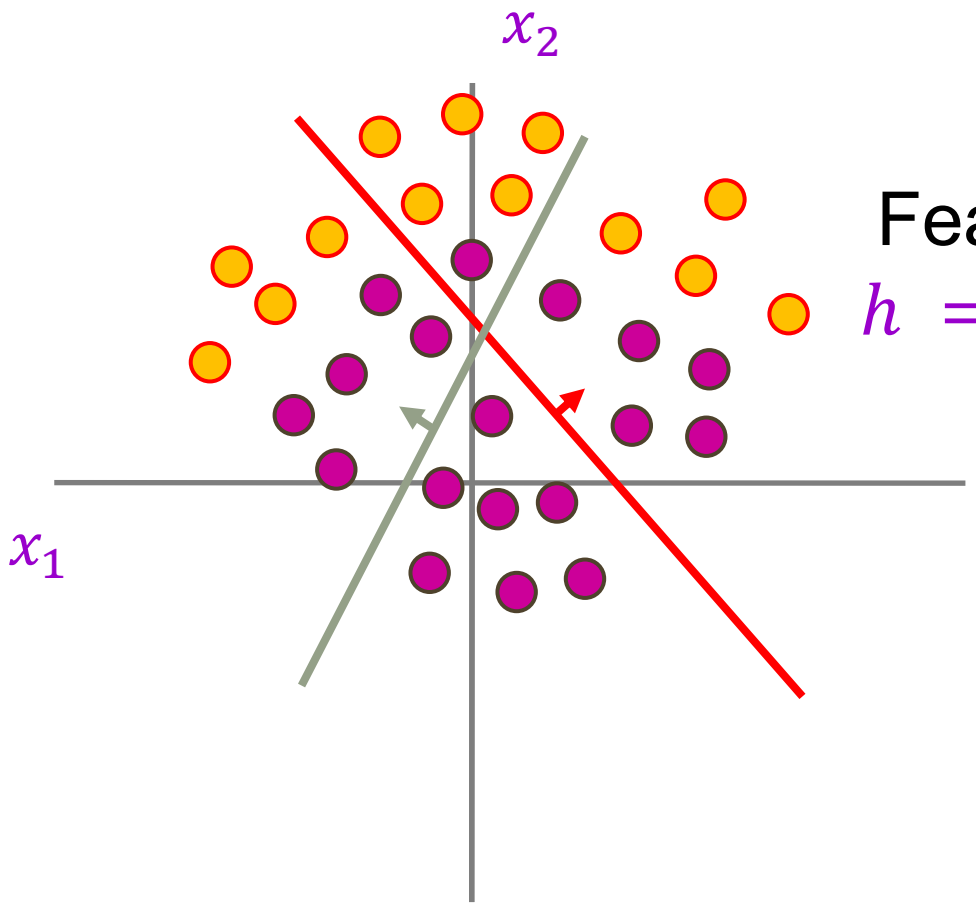
Source: J. Johnson

The power of nonlinearities

Points not linearly separable in original space

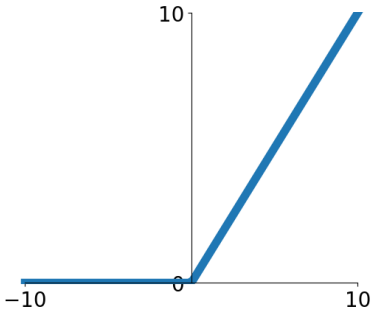
Let's add a nonlinearity:

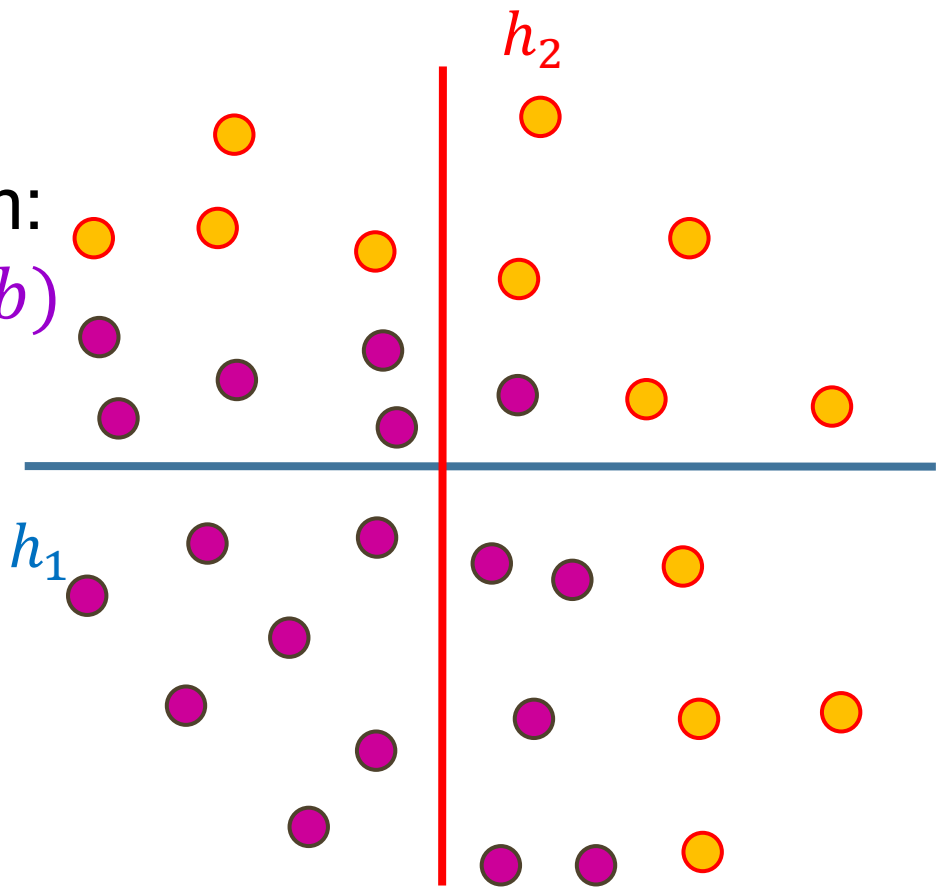
$$h = \text{ReLU}(Wx + b) = \max(0, Wx + b)$$



Feature transform:

$$h = \text{ReLU}(Wx + b)$$





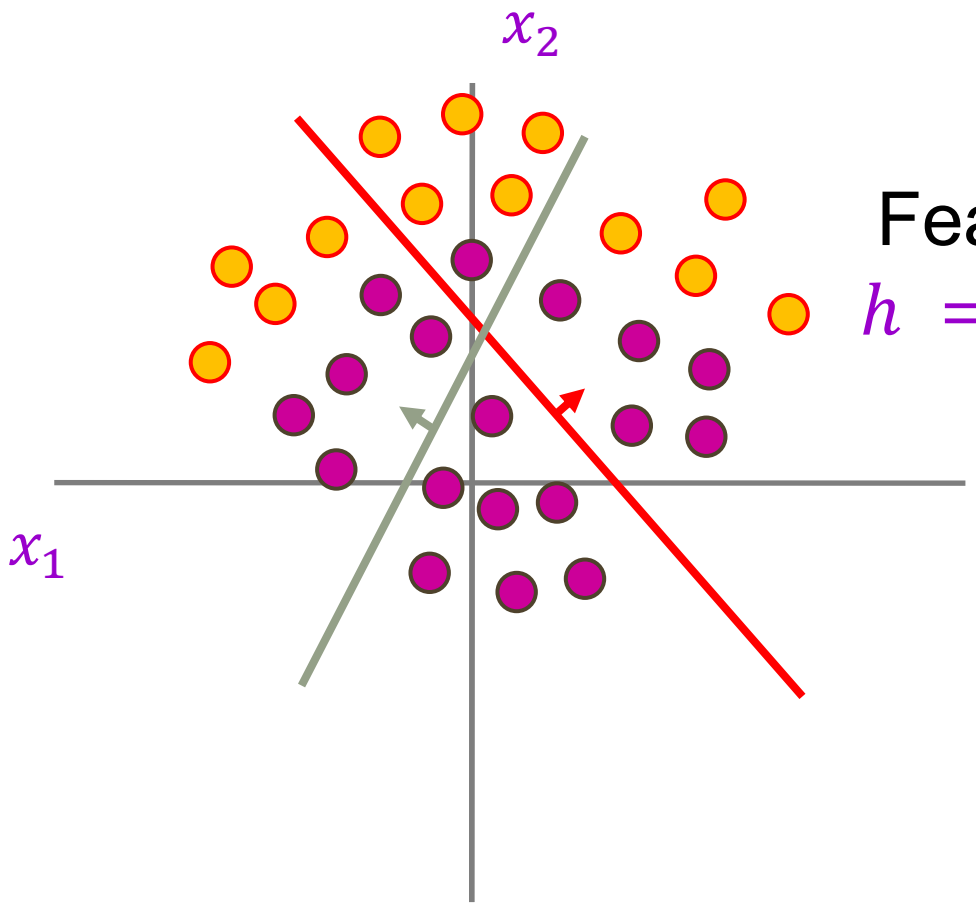
Source: J. Johnson

The power of nonlinearities

Points not linearly separable in original space

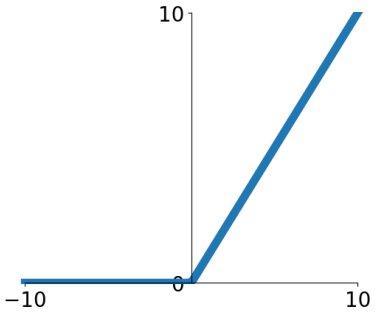
Let's add a nonlinearity:

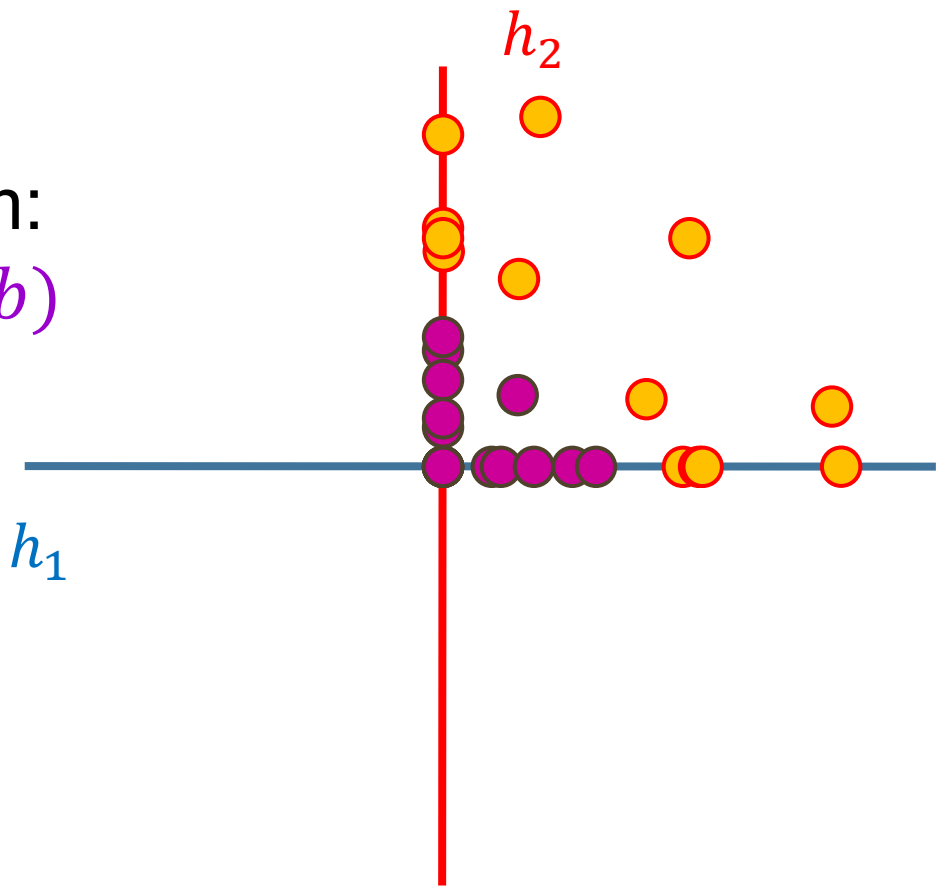
$$h = \text{ReLU}(Wx + b) = \max(0, Wx + b)$$



Feature transform:

$$h = \text{ReLU}(Wx + b)$$





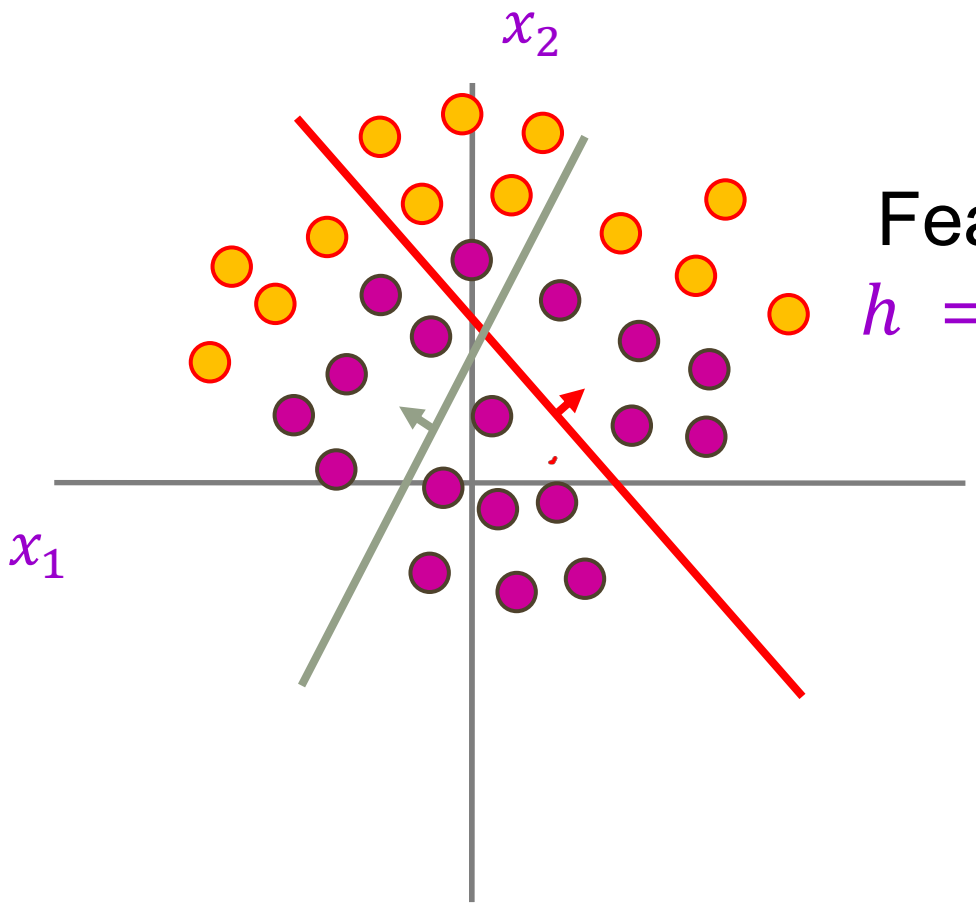
Source: J. Johnson

The power of nonlinearities

Points not linearly separable in original space

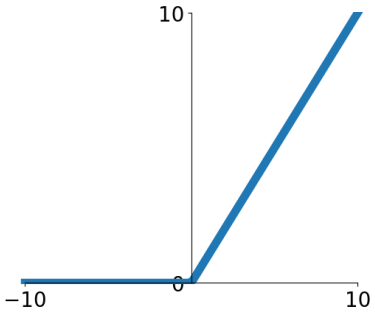
Let's add a nonlinearity:

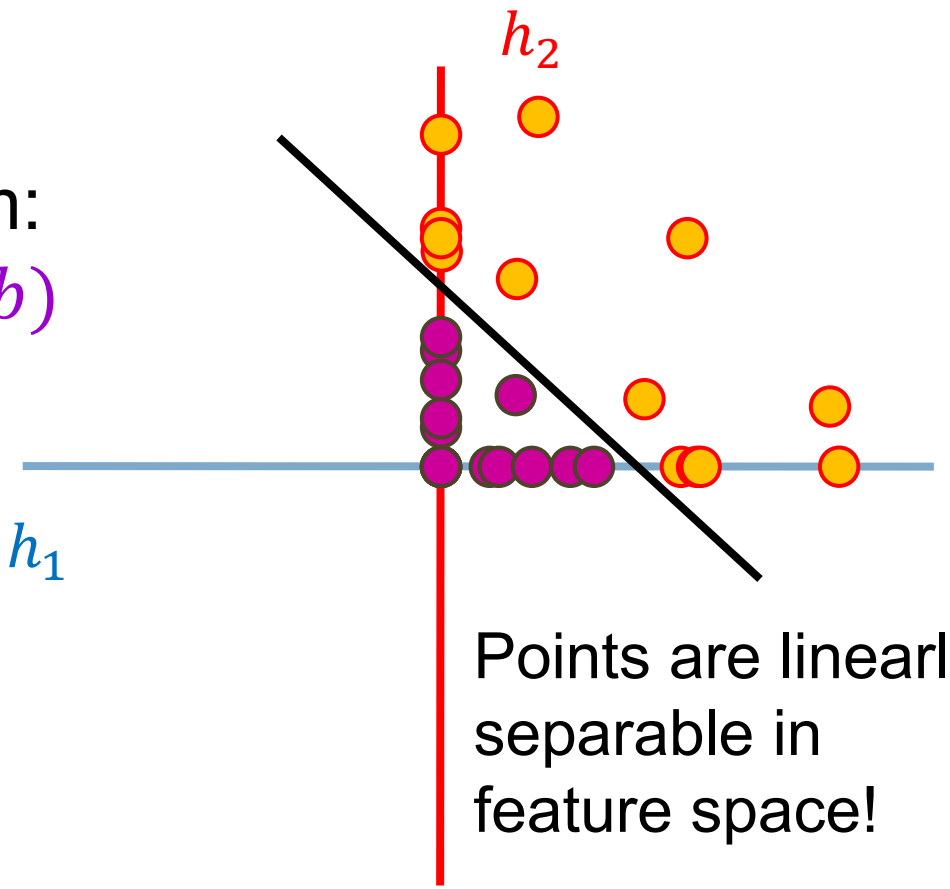
$$h = \text{ReLU}(Wx + b) = \max(0, Wx + b)$$



Feature transform:

$$h = \text{ReLU}(Wx + b)$$





Points are linearly separable in feature space!

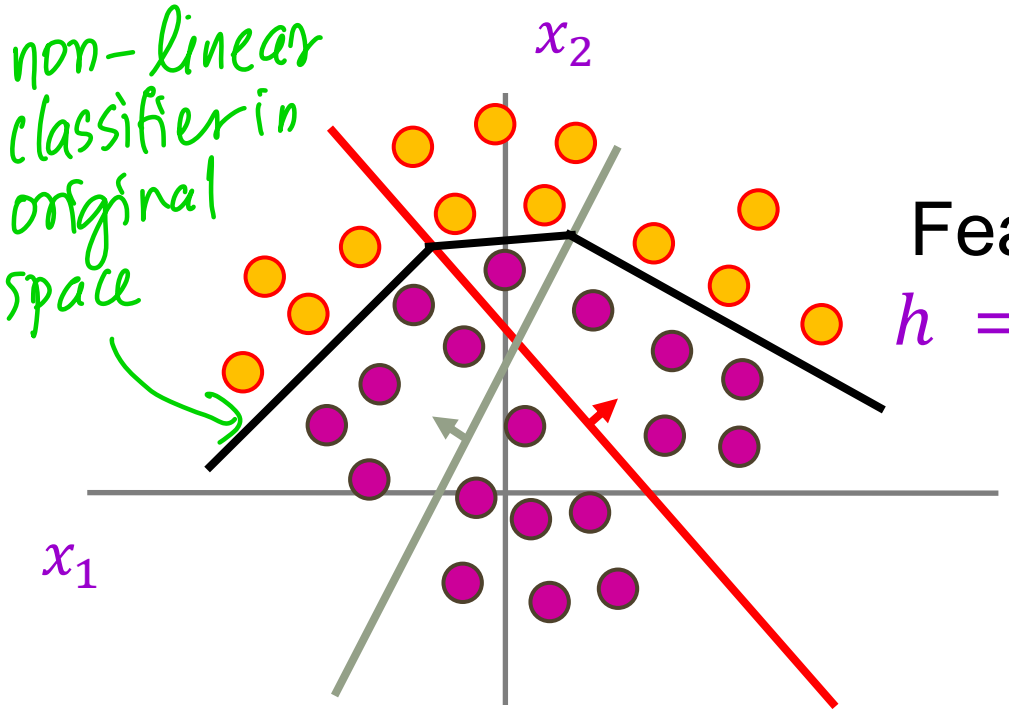
Source: J. Johnson

The power of nonlinearities

Points not linearly separable in original space

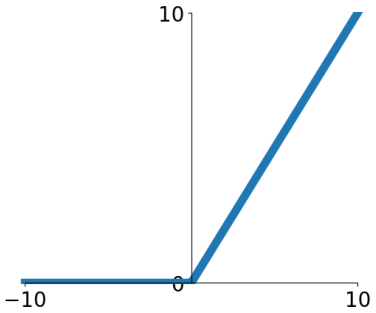
Let's add a nonlinearity:

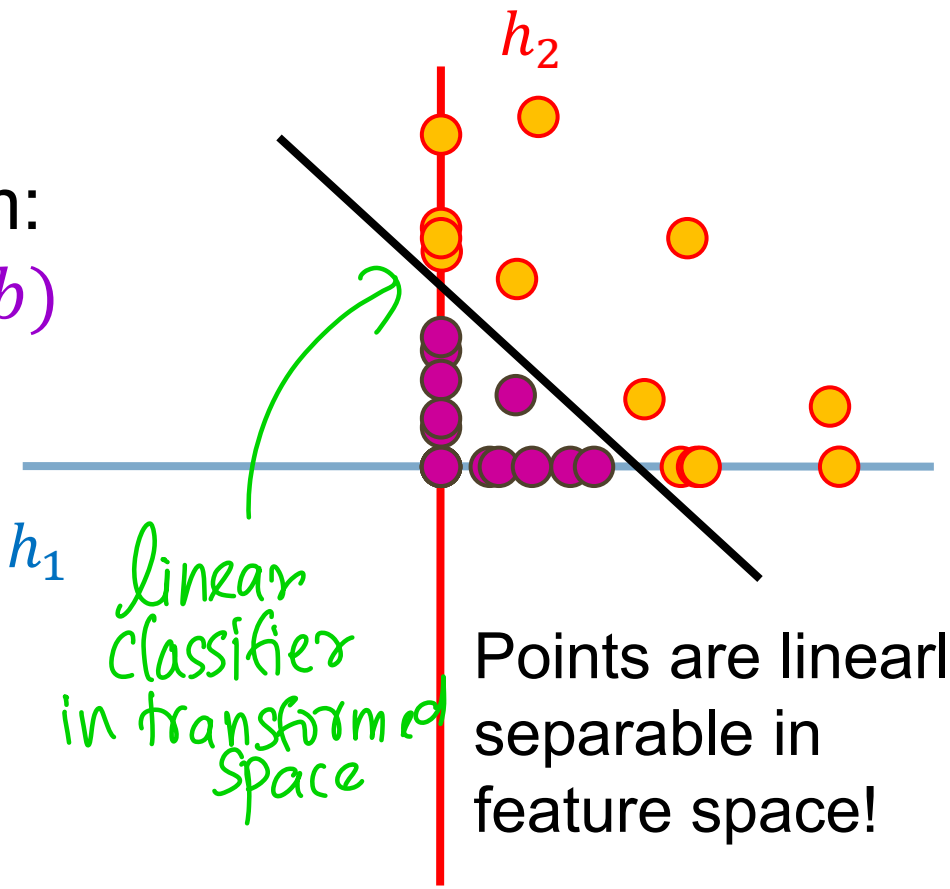
$$h = \text{ReLU}(Wx + b) = \max(0, Wx + b)$$



Feature transform:

$$h = \text{ReLU}(Wx + b)$$



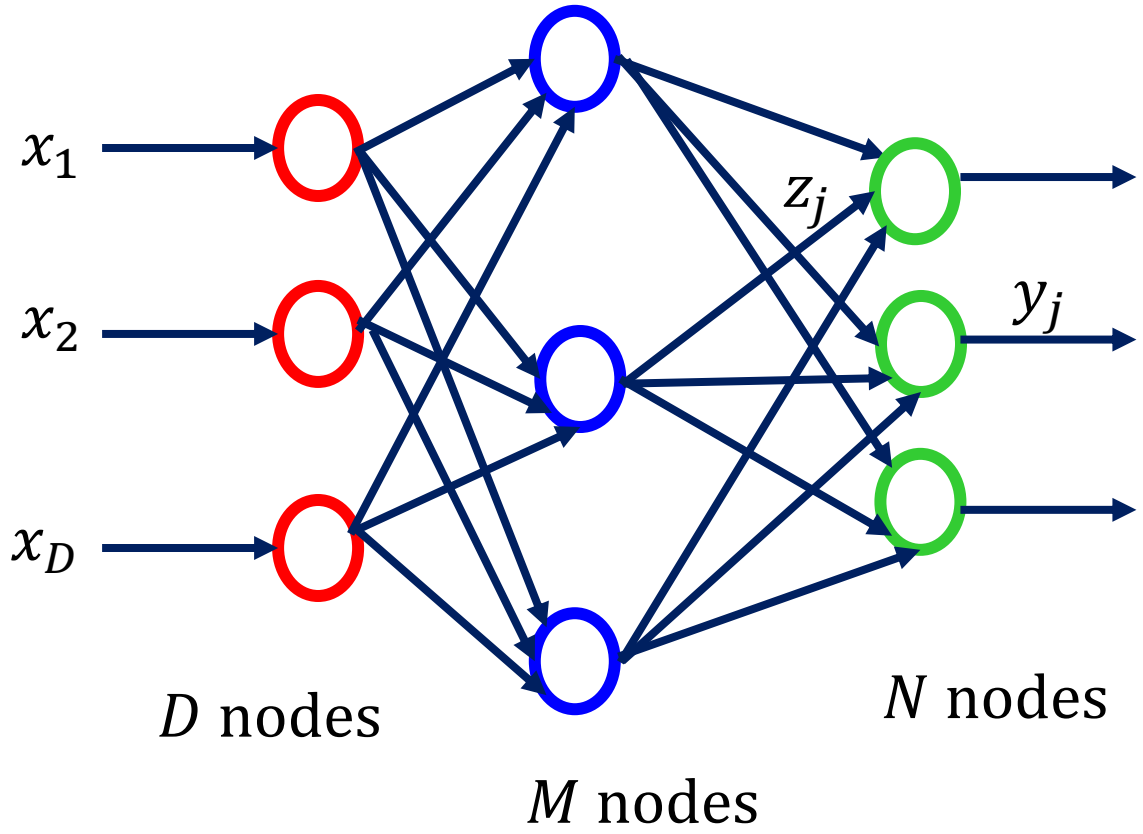


Linear classifier in feature space gives nonlinear classifier in original space

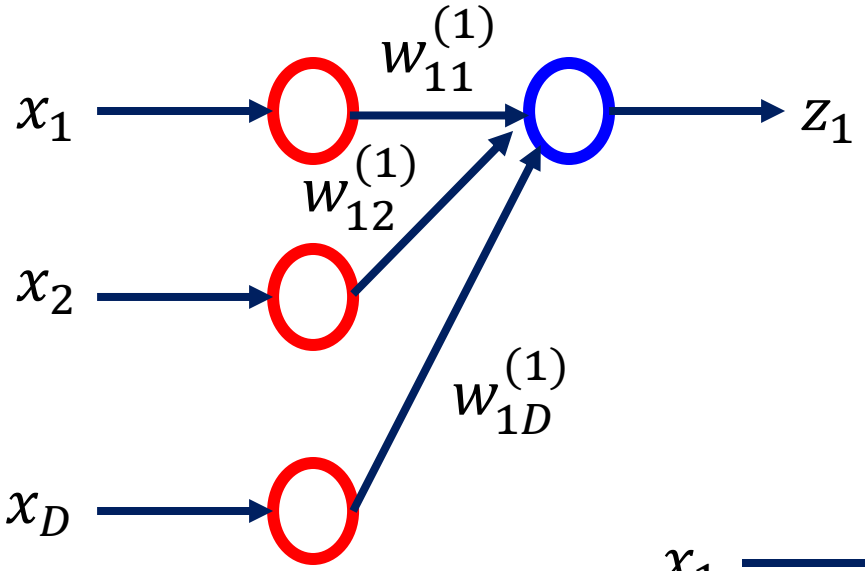
Points are linearly separable in feature space!

Source: J. Johnson

Feed-forward Network



Feed-forward Network



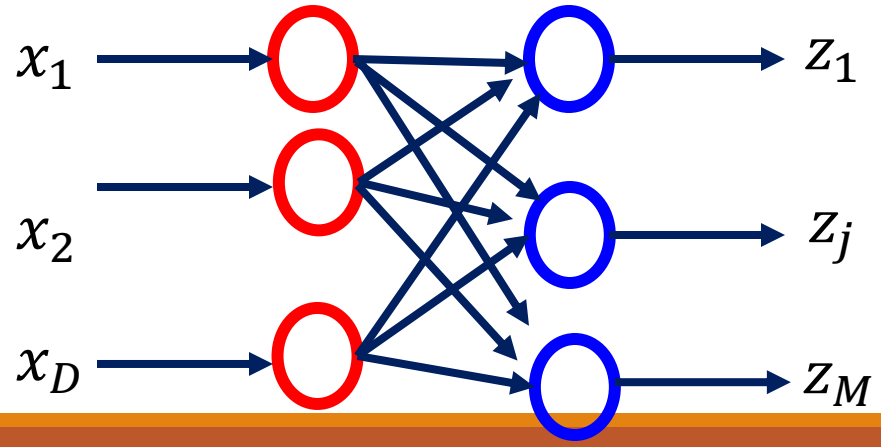
Activations $a_j = \left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i \right) + w_{j0}^{(1)}$

bias

Weights

$z_j = f(a_j)$

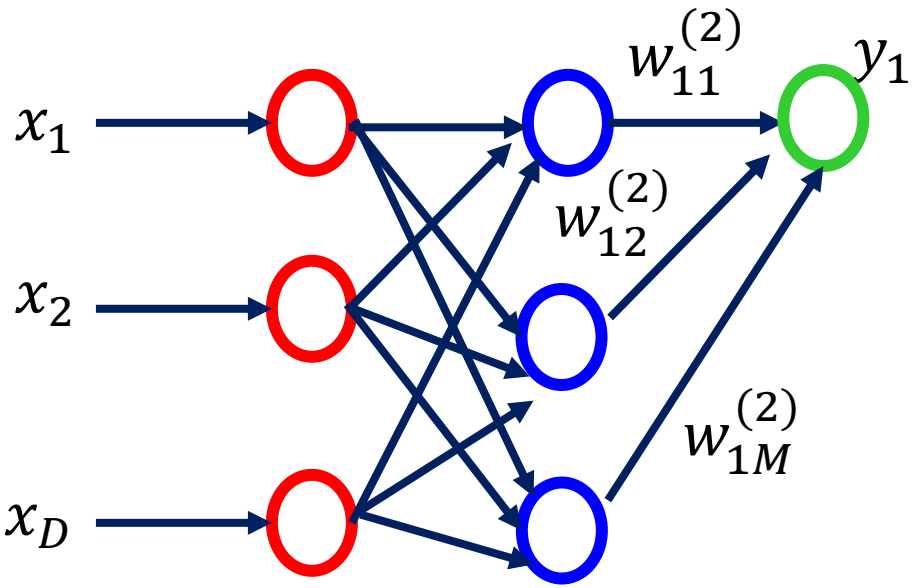
For sigmoid activation function $z_j = \frac{1}{1+e^{-a_j}}$



$z_j = f(a_j)$

j =1,2,…M

Feed-forward Network



Activations $a_j^{(2)} = \left(\sum_{i=1}^{M} w_{ji}^{(2)} z_i\right) + w_{j0}^{(2)}$

bias

Weights

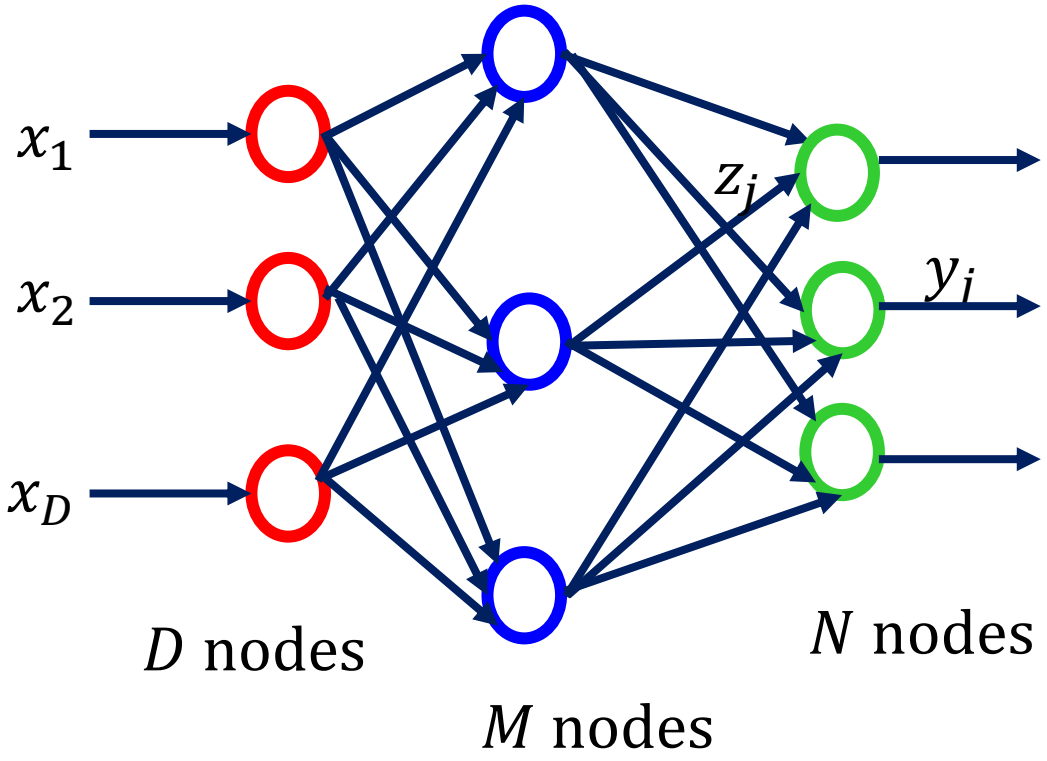
$$y_j = f\left(a_j^{(2)}\right)$$

j =1,2,…N

$$y_j = f\left(\left(\sum_{i=1}^{M} w_{ji}^{(2)} z_i\right) + w_{j0}^{(2)}\right) \quad \& \quad z_i = f\left(\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right) + w_{j0}^{(1)}\right)$$

$$y_j = f\left(\left(\sum_{i=1}^{M} w_{ji}^{(2)} \left(f\left(\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right) + w_{j0}^{(1)}\right)\right)\right) + w_{j0}^{(2)}\right)$$

Feed-forward Network



output activation

inner activation

$$y_j = f\left(\left(\sum_{i=1}^{M} w_{ji}^{(2)}\left(f\left(\left(\sum_{i=1}^{D} w_{ji}^{(1)} x_i\right) + w_{j0}^{(1)}\right)\right)\right) + w_{j0}^{(2)}\right)$$

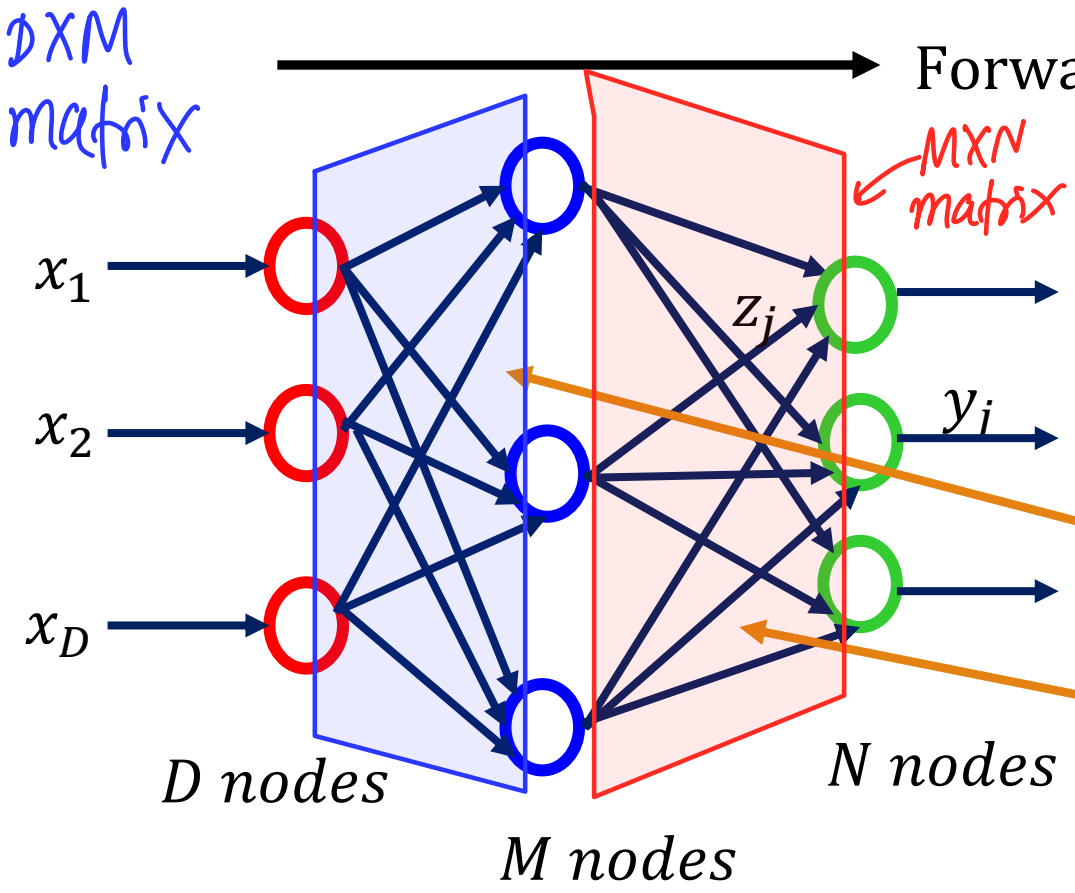
Generally：Sigmoid or tanh function

Binary classification: Sigmoid function

Multi-class: Softmax function

Regression: Identity function

Feed-forward Network



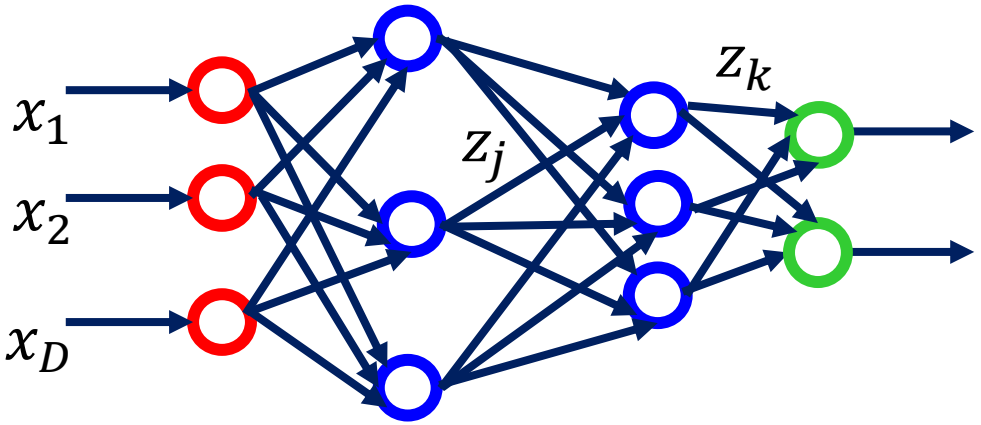
$$y_j = f\left(\left(\sum_{i=1}^{M} w_{ji}^{(2)}\left(f\left(\left(\sum_{i=1}^{D} w_{ji}^{(1)}\, x_i\right) + w_{j0}^{(1)}\right)\right)\right) + w_{j0}^{(2)}\right)$$

D x M weights and M biases

M x N weights and N biases

Feed-forward Network

Forward propagation



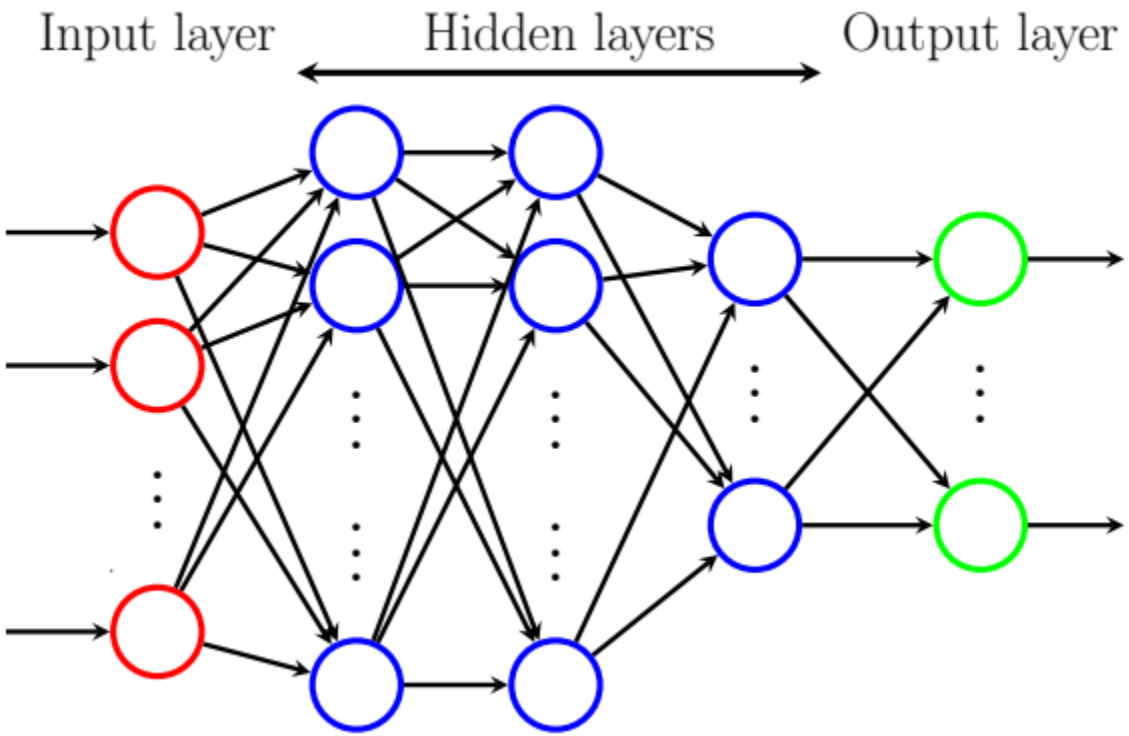
$$Z_k = f\left(\left(\sum_{i=1}^{M} w_{ki}^{(k)} Z_j\right) + w_{k0}^{(k)}\right)$$

Bias parameter is included into $\widetilde{w}$

$$Z_k = f\left(\sum_{i} \widetilde{w_{ki}^{(k)}}\, \widetilde{Z}_J\right)$$

Neural networks are universal approximators (capability to approximate and represent a wide range of complex functions). E.g., A two-layer network with linear outputs can approximate any continuous function on a compact input domain with enough hidden units.

Feed-forward Network



NNs resemble the perceptron model

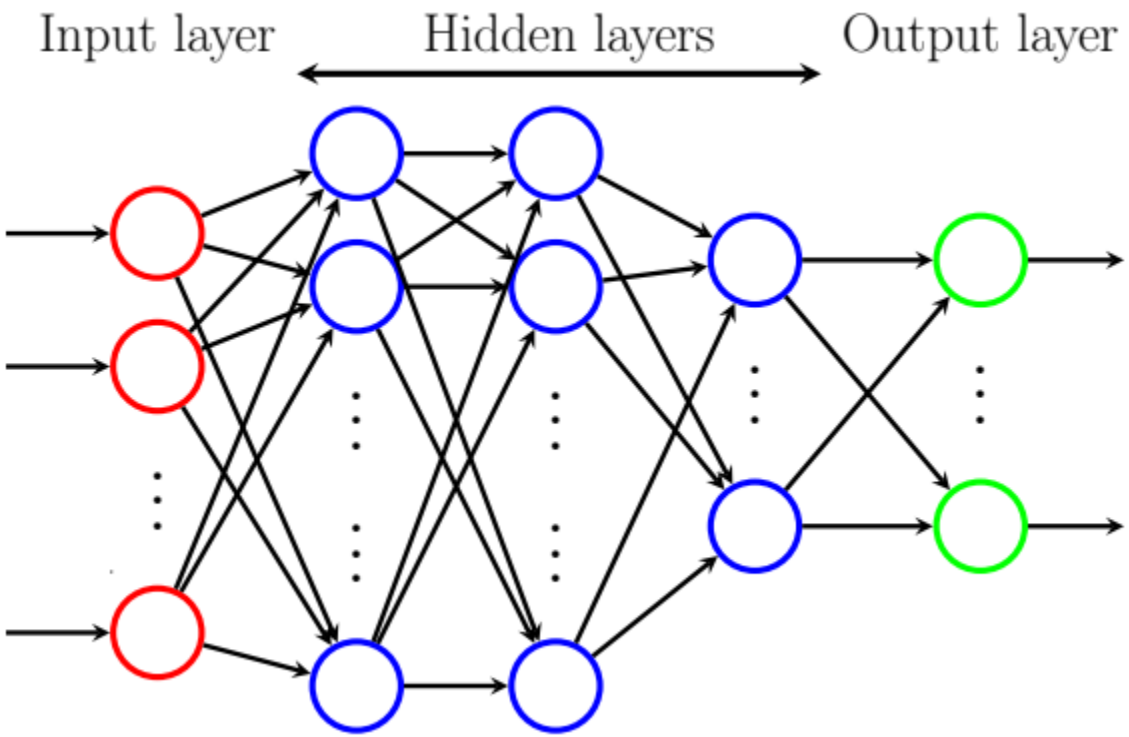
Feed-forward Network also called Multilayer Perceptron (MLP).

NN typically use continuous sigmoidal nonlinearities in their hidden units

NNs are differentiable with respect to their parameters: Plays crucial role in training

Perceptron step-function function

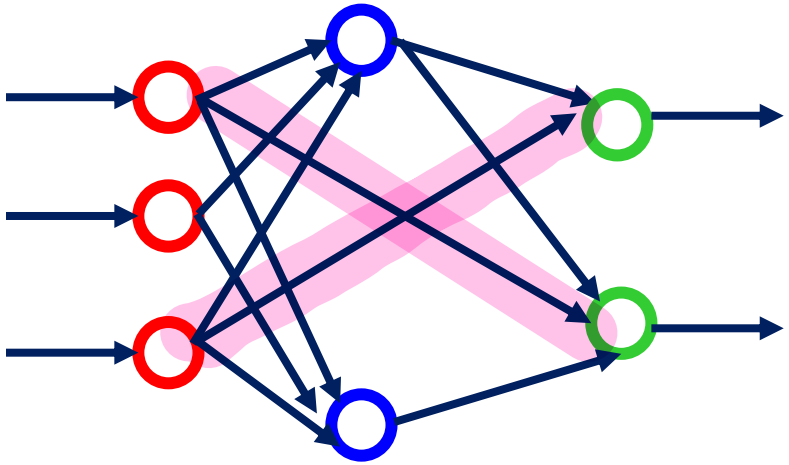
Feed-forward Network



- If all the activation functions in the hidden layers are linear
  - An equivalent network without hidden units can always be constructed (composition of linear transformations remains linear).
  - Little interest on this kind of networks

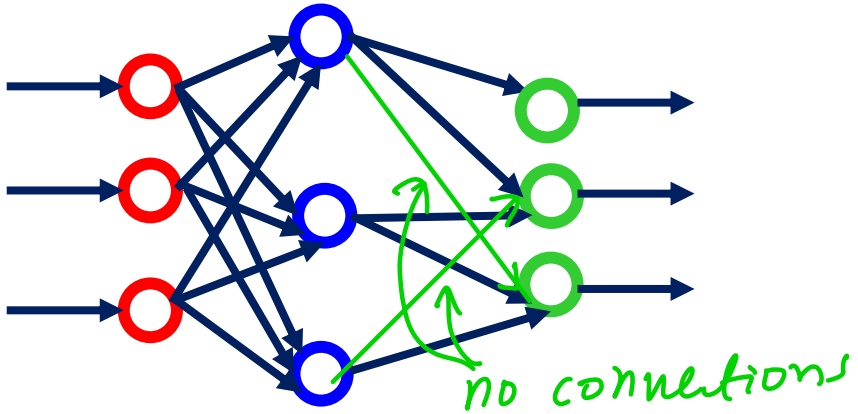
- Terminology for counting the number of layers
  - 5-layer neural network (input+ 3 hidden layers+ output)
  - 3-hidden layer network
  - 4-layer neural network (by counting weights)

Feed-forward Network



Skip-layer connection

Direct connection from input to output

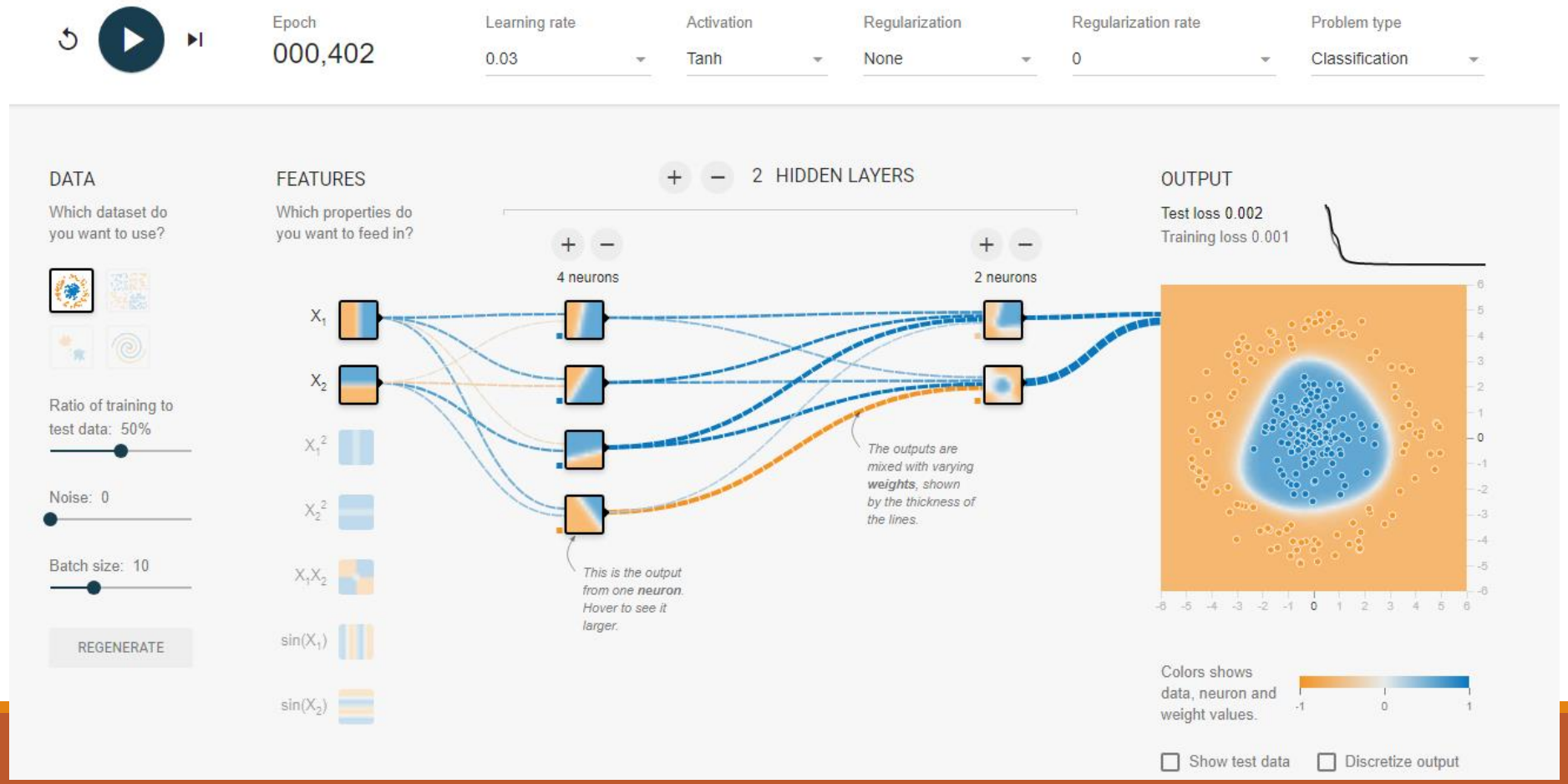


Sparse connection

All nodes (neurons) are not connected

Feed-forward Network

http://playground.tensorflow.org



Training of Multi-layer Network

What is training?

Adjusting its internal parameters (weights and biases) so that it can map input data to the desired output or make accurate predictions for a given task.

Goal: Minimize the difference between the network's predictions and the true values*.



Mean Squared Error (MSE)

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{y}_i - \hat{\boldsymbol{y}}_i\|_2^2 + R(\boldsymbol{w})$$

Regularization loss

dataloss

$$R(\boldsymbol{w}) - \text{regularization term. e. g } R(\boldsymbol{w}) = \lambda\|\mathbf{w}\|_2^2$$

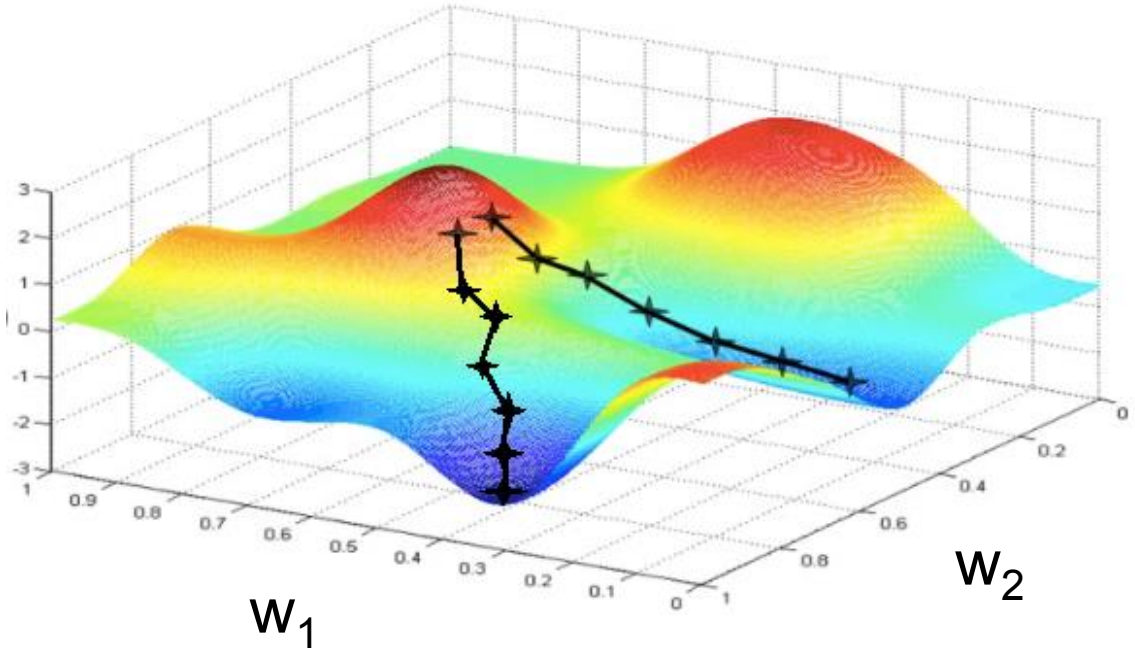
* Based on the scenario different loss function can be used

Training of Multi-layer Network

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{y}_i - f_{\boldsymbol{w}}(\boldsymbol{x}_i)\|_2^2 + R(\boldsymbol{w})$$

Gradient descent

$$\boldsymbol{w}_{\text{new}} \leftarrow \boldsymbol{w} - \alpha \frac{\partial L(\mathbf{w})}{\partial \boldsymbol{w}}$$

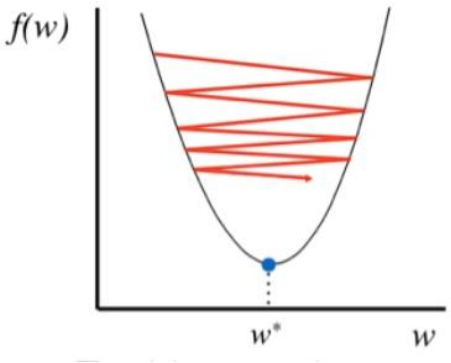


Slide credit: Prof. Svetlana Lazebnik

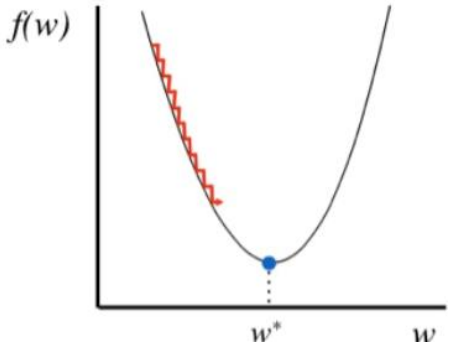
Training of Multi-layer Network

Gradient descent $\boldsymbol{w}_{\text{new}} \leftarrow \boldsymbol{w} - \alpha \frac{\partial L(\mathbf{w})}{\partial \boldsymbol{w}}$

Learning rate too high



Learning rate too low



good decay schedule

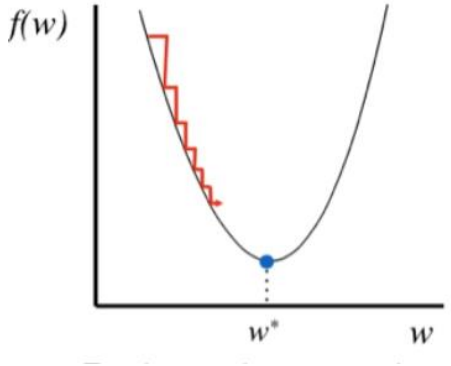


Image from slideshare.net/ssuserf88631/scalable-machine-learning-73621818

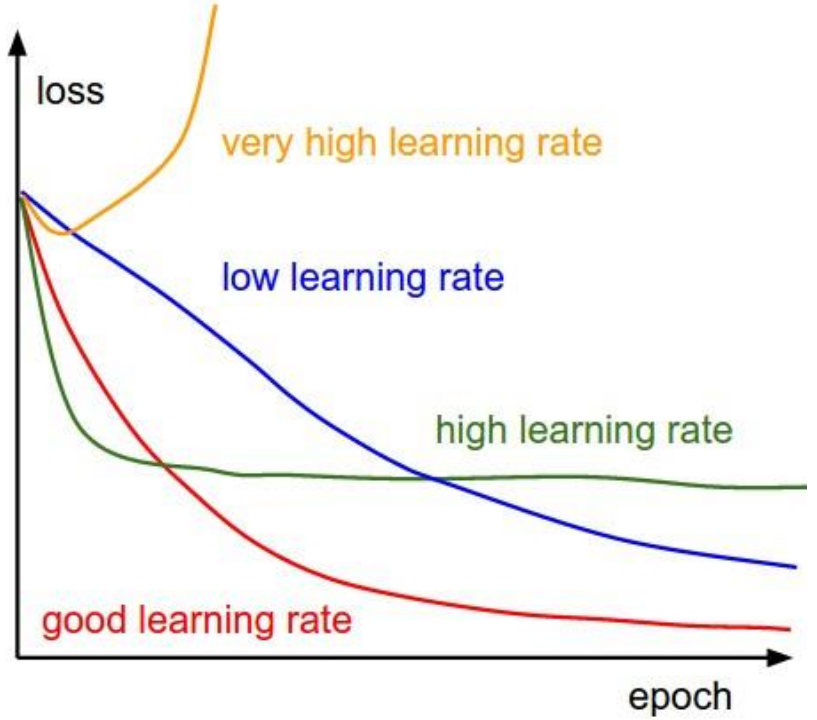


Image from CS231n Convolutional Neural Networks for Visual Recognition

Slide credit: Prof. Svetlana Lazebnik

Training of Multi-layer Network

How to use adjustable learning rate.

torch.optim — PyTorch 2.0 documentation

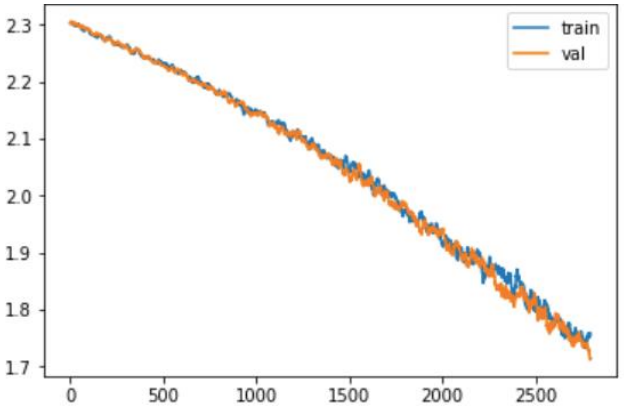
Learning rate schedules API (keras.io)

learning rate decay methods

- Decay formulas*
  - Exponential: $\eta_t = \eta_0 e^{-kt}$, where η_0 and k are hyperparameters, t is the iteration or epoch number
  - Inverse: $\eta_t = \eta_0/(1+kt)$
  - Inverse sqrt: $\eta_t = \eta_0/\sqrt{t}$
  - Linear: $\eta_t = \eta_0(1 - t/T)$, where T is the total number of epochs
  - Cosine: $\eta_t = \frac{1}{2}\eta_0(1 + \cos(t\pi/T))$

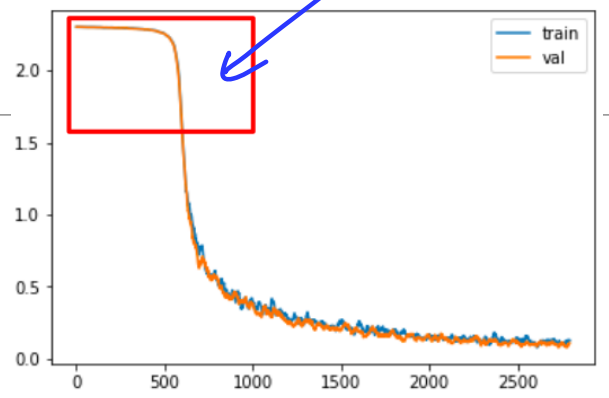
*Source: Prof. Svetlana Lazebnik

Diagnosing learning curves



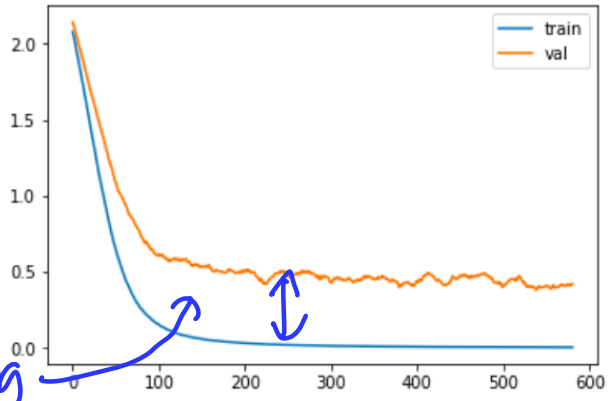
haven't converged

Not converged yet
Keep training, possibly increase learning rate



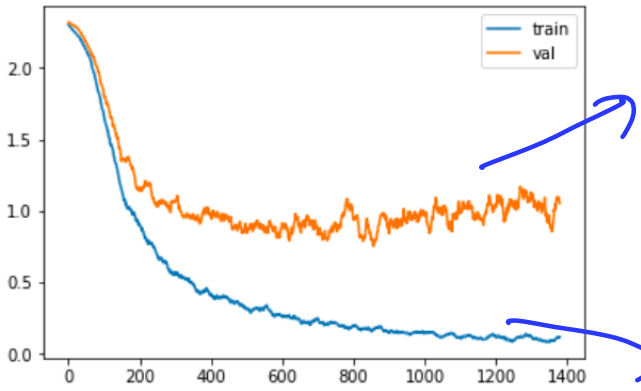
bad initialization

Slow start
Bad initialization?



overfitting

Possible overfitting



severe overfitting

Definite overfitting

Source: Stanford CS231n

Training of Multi-layer Network: Backpropagation

The backpropagation algorithm was originally discovered by Frank Rosenblatt , and independently Henry J. Kelley.

The backpropagation algorithm is a technique used to calculate the gradient of a loss function applied to the output of a neural network with respect to the parameters in each layer.

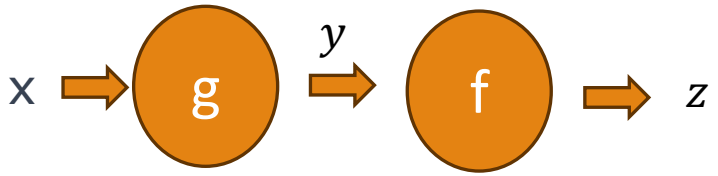
Chain rule

h(x)=f(g(x))

$h'(x)=f'(g(x))\,g'(x)$

Also, it can be expressed in Leibniz's notation

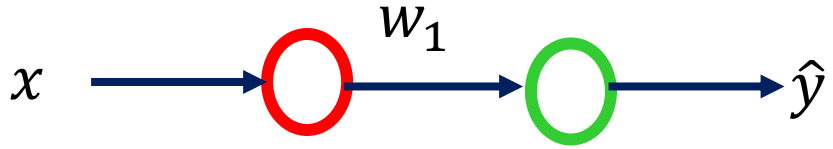
$$\frac{dz}{dx}=\frac{dz}{dy}\frac{dy}{dx}$$



Backpropagation - Wikipedia

Chain rule - Wikipedia

Training of Multi-layer Network: Backpropagation: One data sample



$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{y}_i - f_{\boldsymbol{w}}(\boldsymbol{x}_i)\|_2^2$$

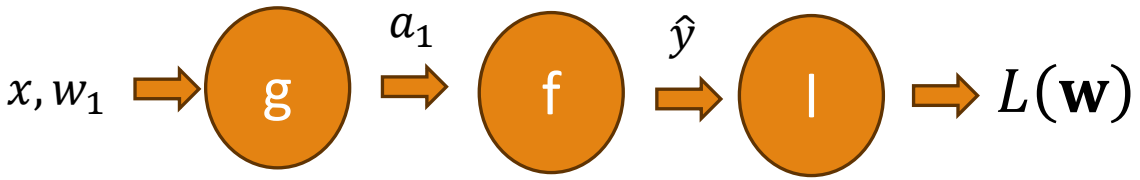
$$a_1 = w_1 x + b_1$$

$$\hat{y} = f(a_1) = \sigma(a_1) = \frac{1}{1+e^{-a_1}}$$

For one data sample

$$L(\mathbf{w}) = (\hat{y} - y)^2$$

Gradient descent $\boldsymbol{w}_{\text{new}} \leftarrow \boldsymbol{w} - \alpha \frac{\partial L(\mathbf{w})}{\partial \boldsymbol{w}}$

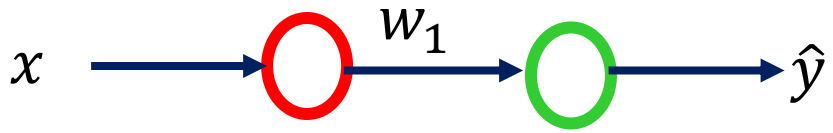


$$\frac{\partial L(\mathbf{w})}{\partial w_1} = \frac{\partial L(\mathbf{w})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_1} \frac{\partial a_1}{\partial w_1}$$

$$\frac{\partial L(\mathbf{w})}{\partial w_1} = 2(\hat{y} - y)\,\sigma(a_1)\,(1 - \sigma(a_1))\,x$$

Training of Multi-layer Network: Backpropagation: One data sample

Homework:

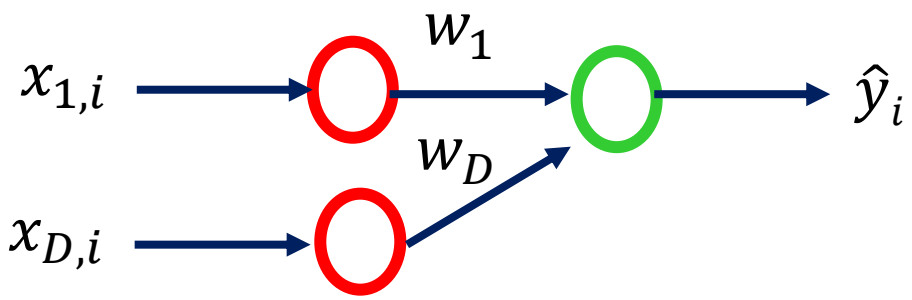


For one data sample, assume that $y = 1$

$$L(\mathbf{w}) = -\log(\hat{y})$$

$$\frac{\partial L(\mathbf{w})}{\partial w_1} = ?$$

Training of Multi-layer Network: Backpropagation



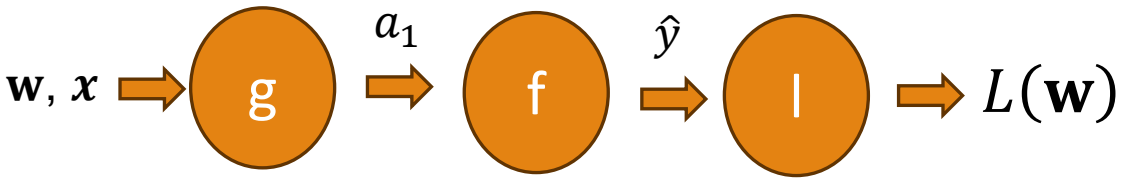
$$a_1 = \mathbf{w}^T \boldsymbol{x_i} + b$$

$$\hat{y}_i = f(a_1) = \sigma\ (a_1) = \frac{1}{1+e^{-a_1}}$$

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \|\boldsymbol{y}_i - f_{\boldsymbol{w}}(\boldsymbol{x}_i)\|_2^2$$

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

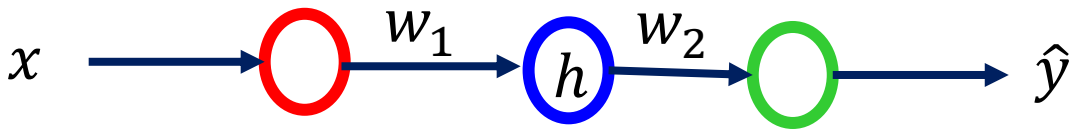
Gradient descent $\boldsymbol{w}_{\text{new}} \leftarrow \boldsymbol{w} - \alpha \frac{\partial L(\mathbf{w})}{\partial \boldsymbol{w}}$



$$\frac{\partial L(\mathbf{w})}{\partial w_1} = \frac{\partial L(\mathbf{w})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial a_1} \frac{\partial a_1}{\partial w_1}$$

$$\frac{\partial L(\mathbf{w})}{\partial \mathbf{w}} = \frac{1}{N}\sum_{i=1}^{N} 2(\hat{y}_i - y)\ \sigma\ (a_1)\ (1 - \sigma\ (a_1))\ \boldsymbol{x}_i$$

Training of Multi-layer Network: Backpropagation: One data sample



For one data sample

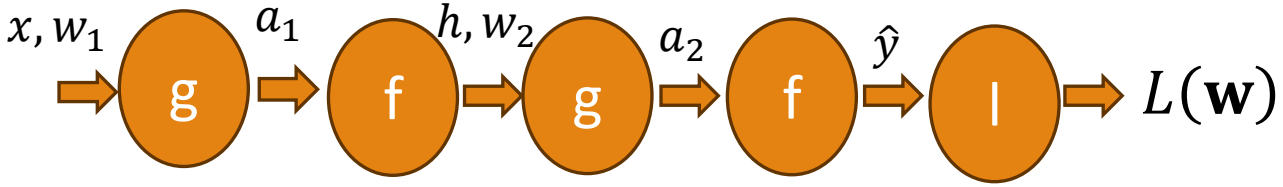
$L(\mathbf{w}) = (\hat{y} - y)^2$

$a_1 = w_1 x + b_1$

$h = f(a_1) = \sigma(a_1) = \frac{1}{1+e^{-a_1}}$

$a_2 = w_2 h + b_2$

$\hat{y} = f(a_2) = \sigma(a_2) = \frac{1}{1+e^{-a_2}}$



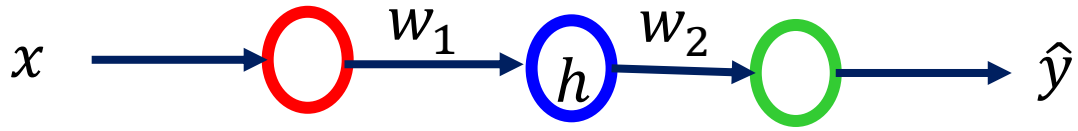
$\frac{\partial L(\mathbf{w})}{\partial w_1} = ?$

$\frac{\partial L(\mathbf{w})}{\partial w_1} = \frac{\partial L(\mathbf{w})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h} \frac{\partial h}{\partial w_1}$

$\frac{\partial h}{\partial w_1} = \frac{\partial h}{\partial a_1} \frac{\partial a_1}{\partial w_1} = \sigma(a_1)(1 - \sigma(a_1))x$

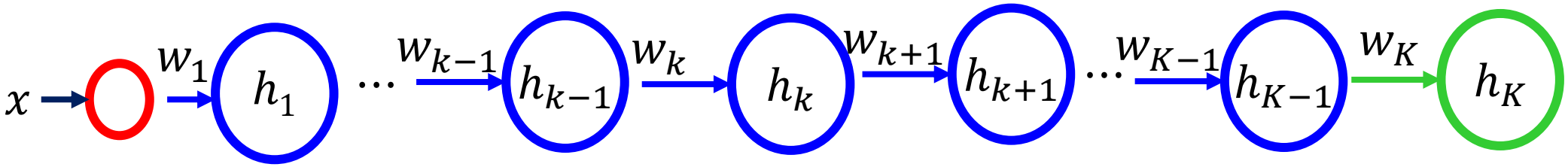
$\frac{\partial L(\mathbf{w})}{\partial \hat{y}} = 2(\hat{y} - y)$, $\frac{\partial \hat{y}}{\partial h} = \frac{\partial \hat{y}}{\partial a_2} \frac{\partial a_2}{\partial h} = \sigma(a_2)(1 - \sigma(a_2))w_2$

Training of Multi-layer Network: Backpropagation:



$$\frac{\partial L(\mathbf{w})}{\partial w_1} = \boxed{\frac{\partial L(\mathbf{w})}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial h}} \frac{\partial h}{\partial w_1}$$

Upstream gradient $\frac{\partial L(\mathbf{w})}{\partial h}$ Local gradient



$h_K = \hat{y}$

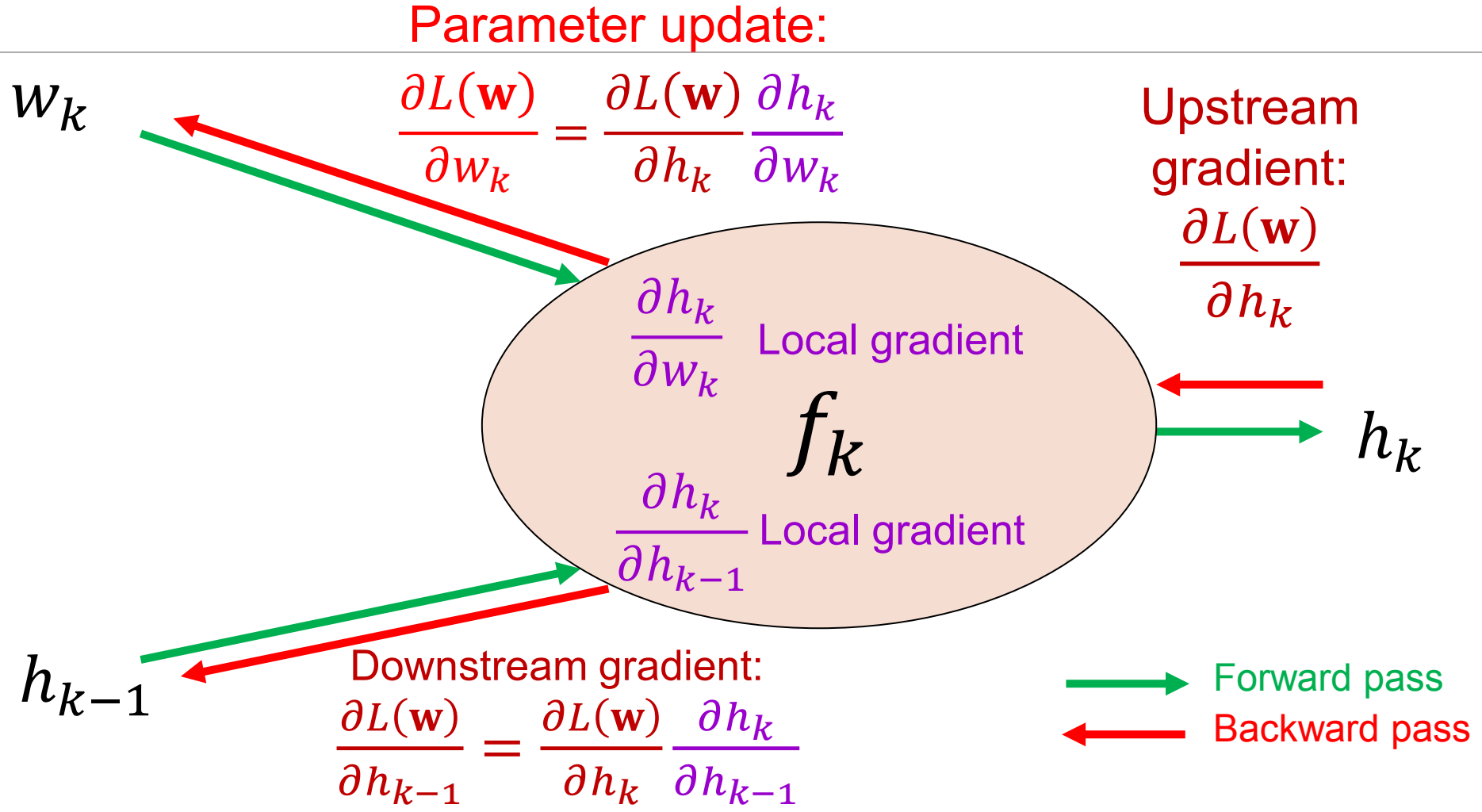
$$\frac{\partial L(\mathbf{w})}{\partial w_k} = \frac{\partial h_k}{\partial w_k} \frac{\partial L(\mathbf{w})}{\partial h_k}$$

Local gradient

Upstream gradient

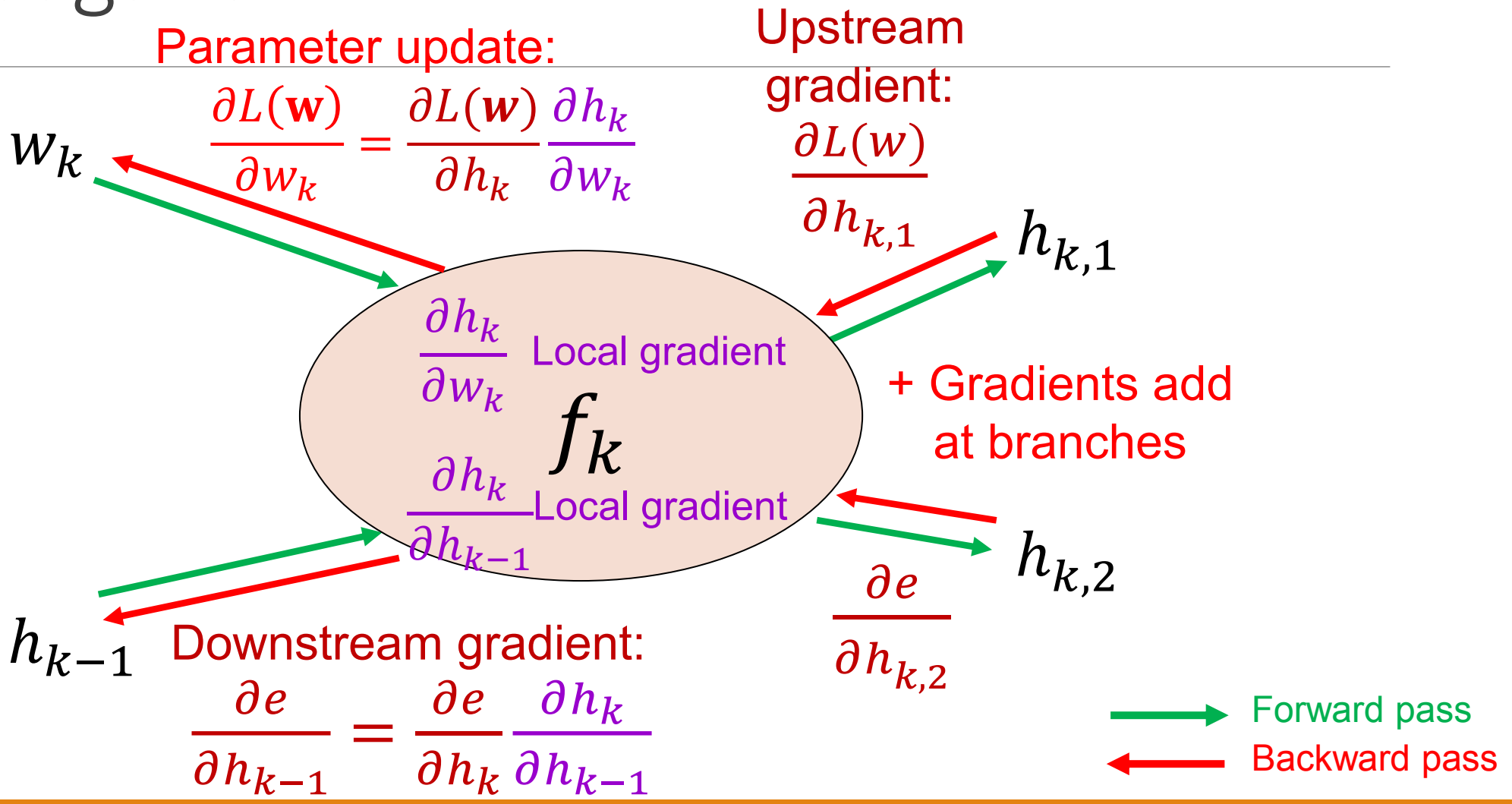
$$\frac{\partial L(\mathbf{w})}{\partial h_k} = \frac{\partial L(\mathbf{w})}{\partial h_K} \frac{\partial h_K}{\partial h_{K-1}} \frac{\partial h_{K-1}}{\partial h_{K-2}} \cdots \frac{\partial h_{k+1}}{\partial h_k}$$

Training of Multi-layer Network: Backpropagation:



Slide credit: Prof. Svetlana Lazebnik

Training of Multi-layer Network: Backpropagation:

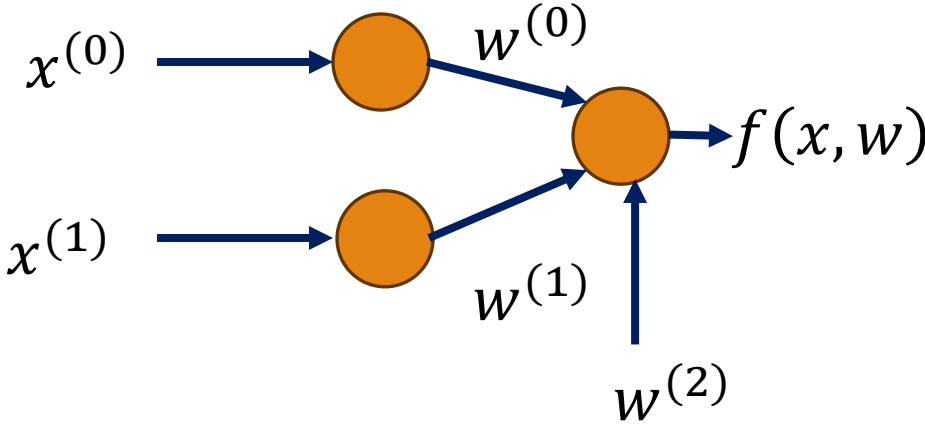


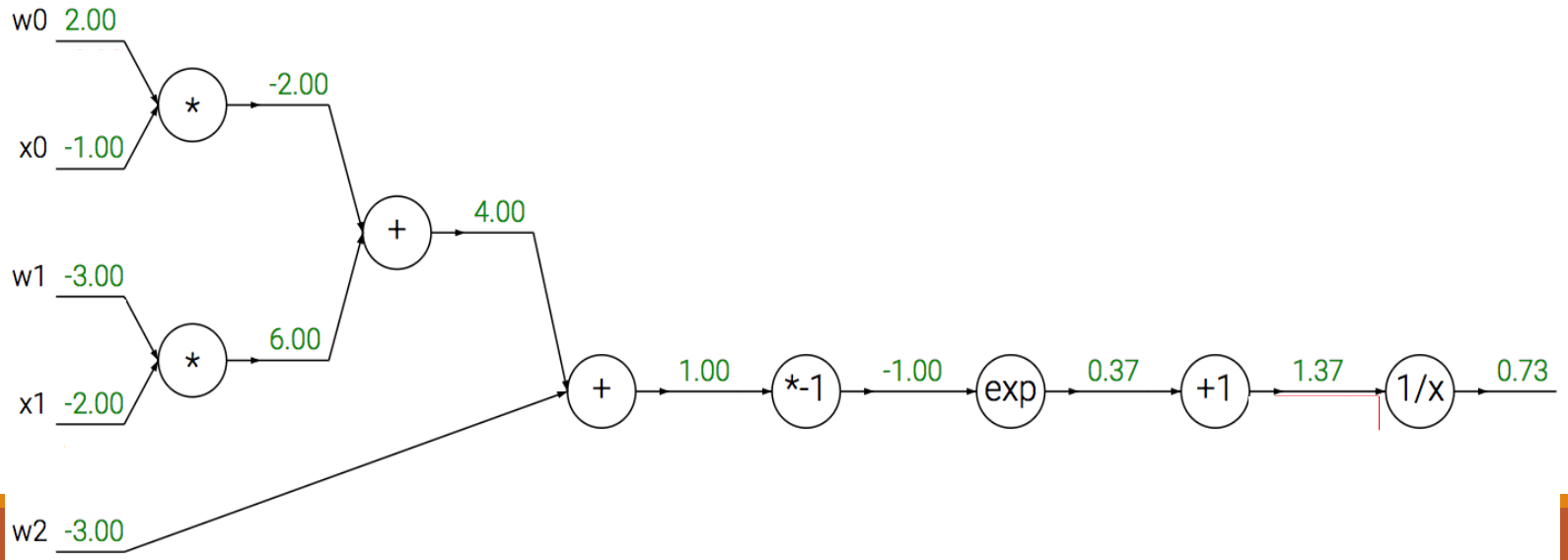
Backpropagation example

Example from CS231n: Deep Learning for Computer Vision-Stanford

Stanford University CS231n: Deep Learning for Computer Vision

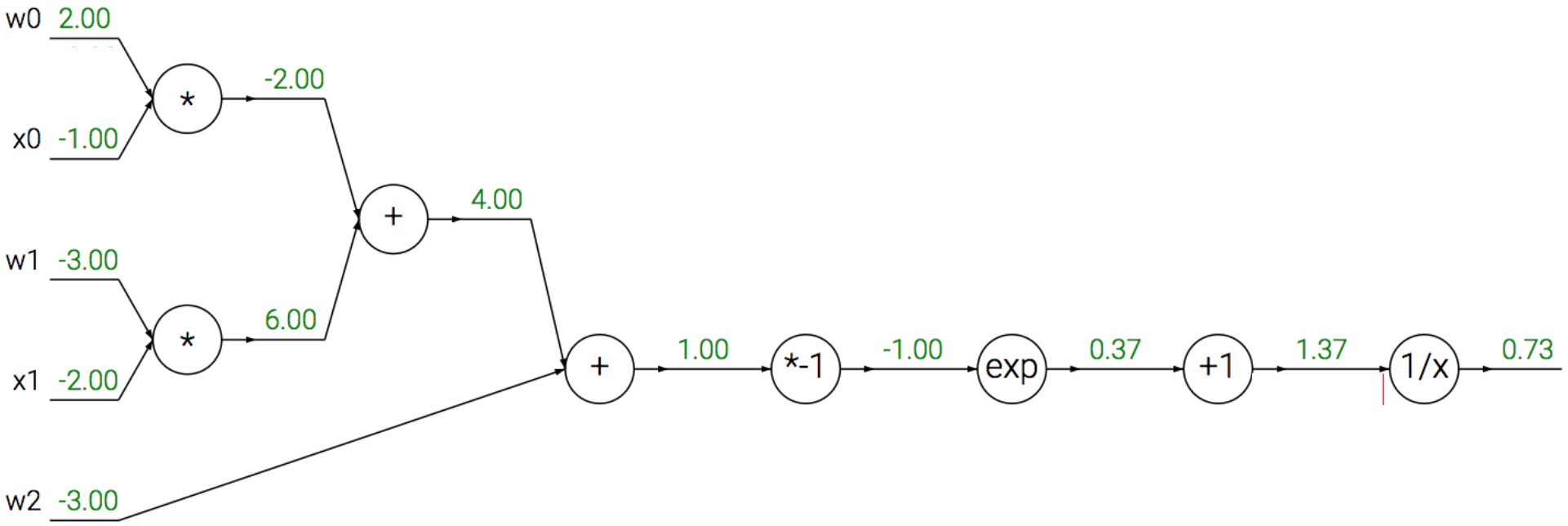
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$





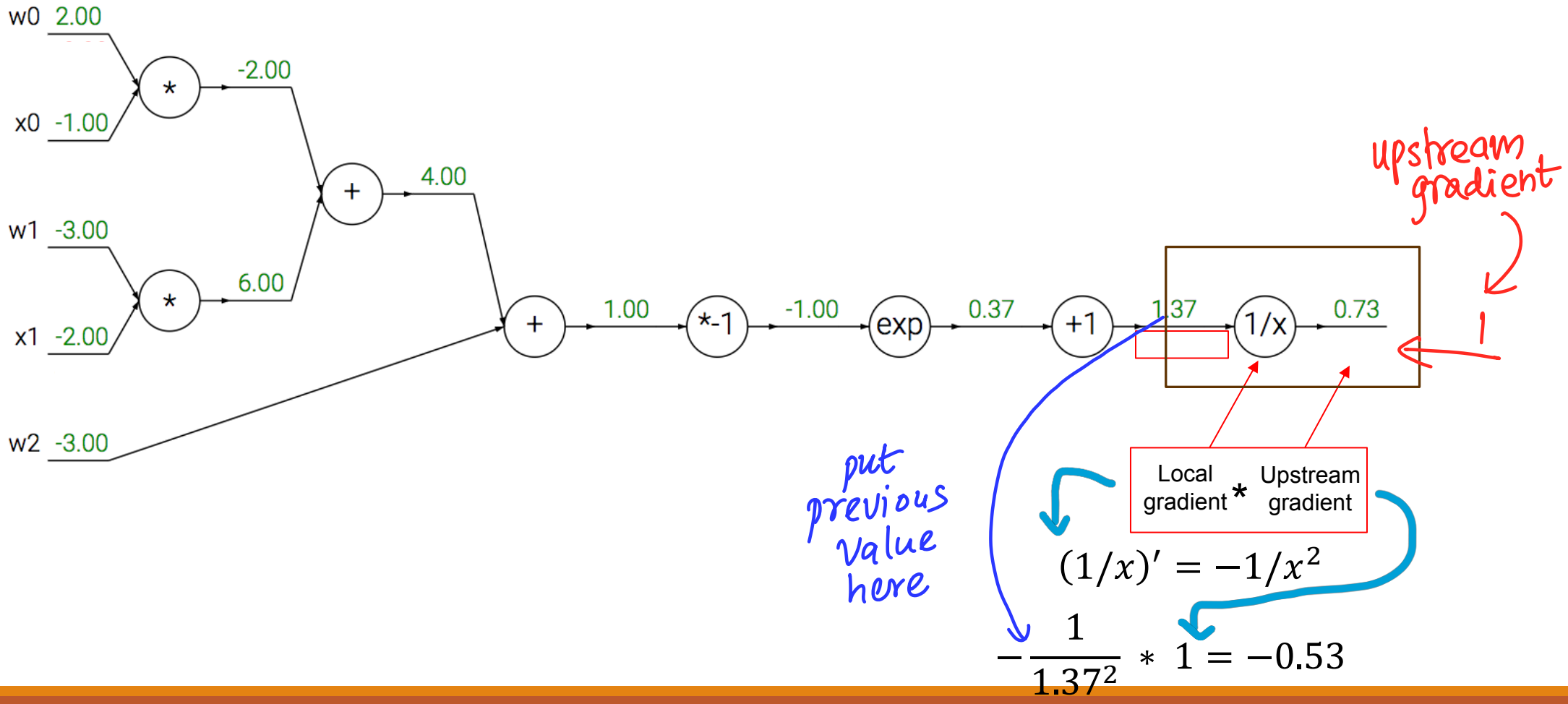
computation graph

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Source: Stanford 231n

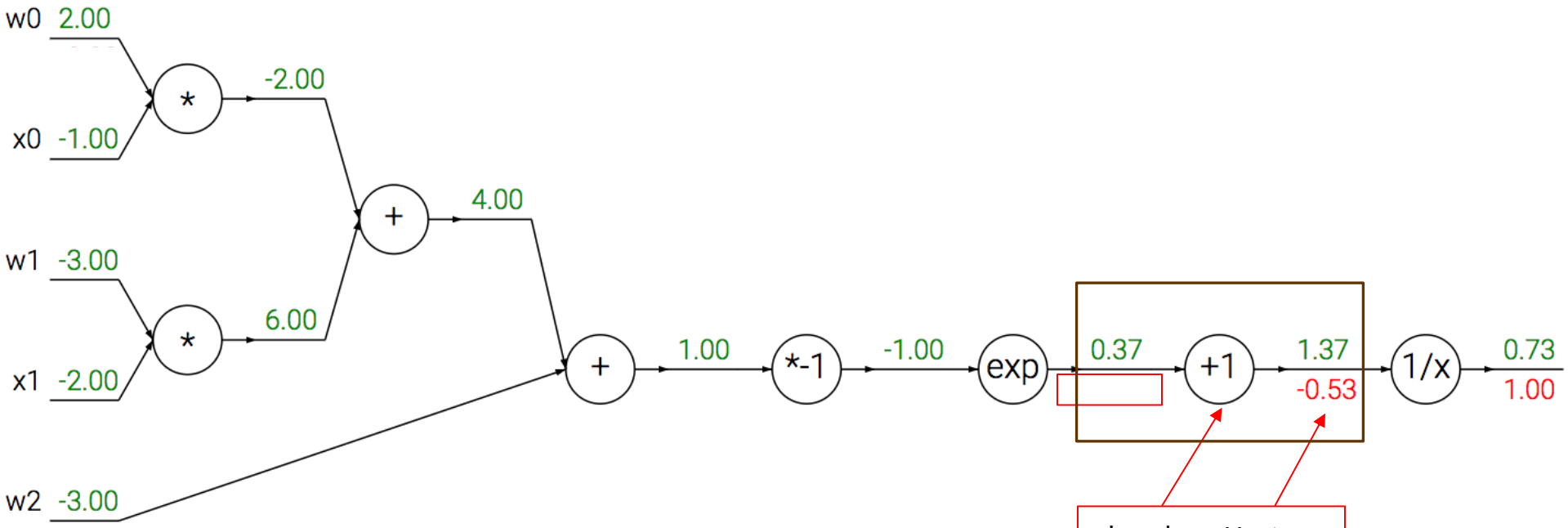
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Slide credit: Prof. Svetlana Lazebnik

Source: Stanford 231n

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$

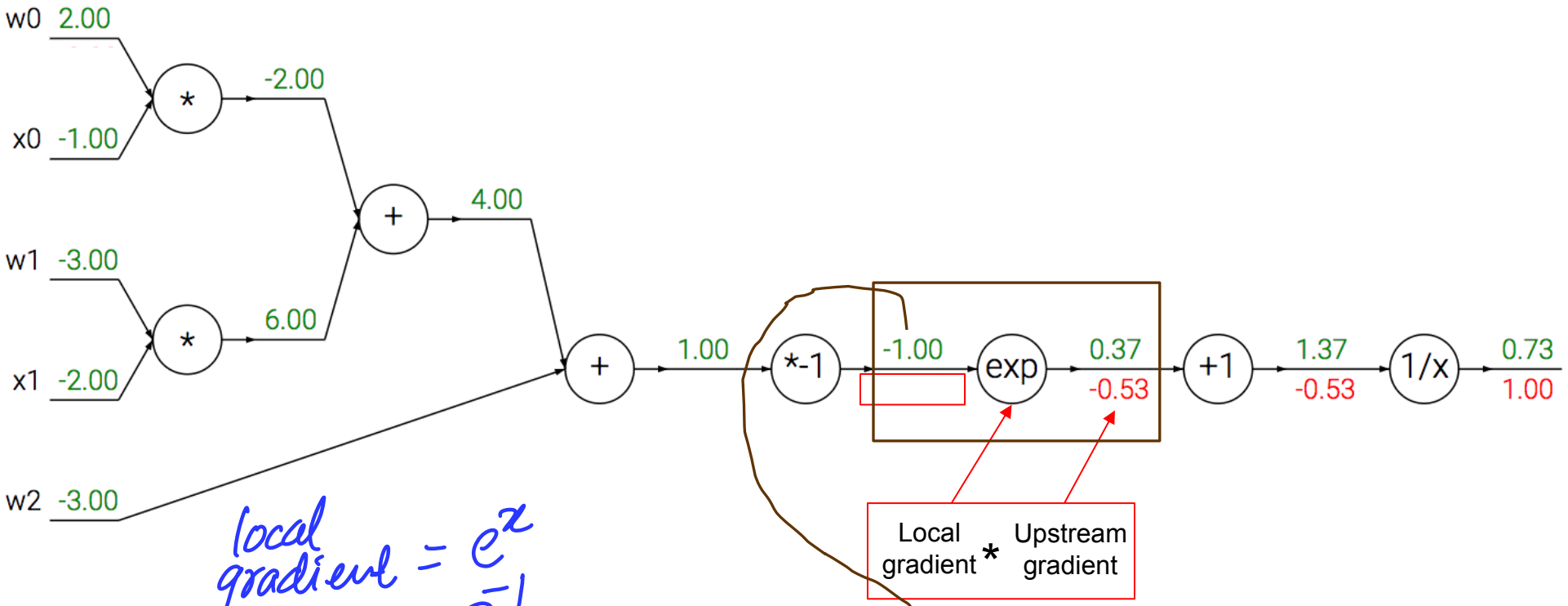


$$f(x) = a + x, \frac{df(x)}{dx} = 1$$

Slide credit: Prof. Svetlana Lazebnik

Source: Stanford 231n

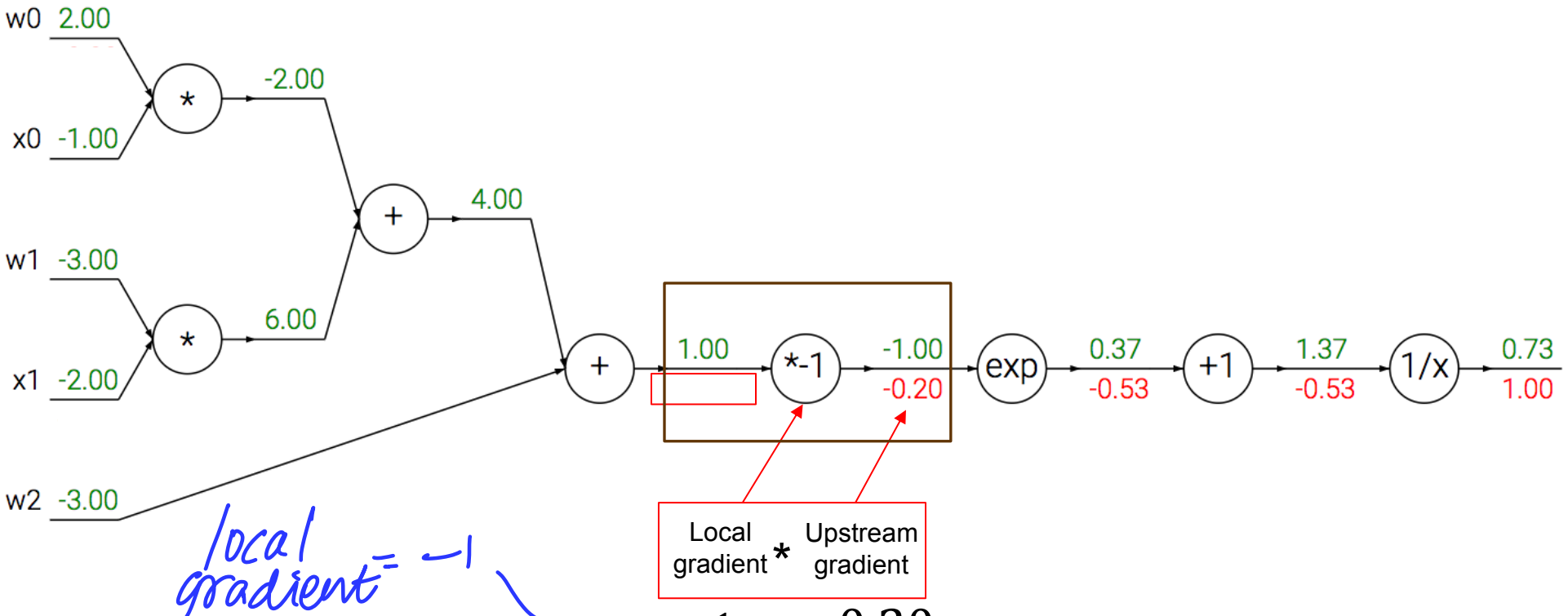
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Slide credit: Prof. Svetlana Lazebnik

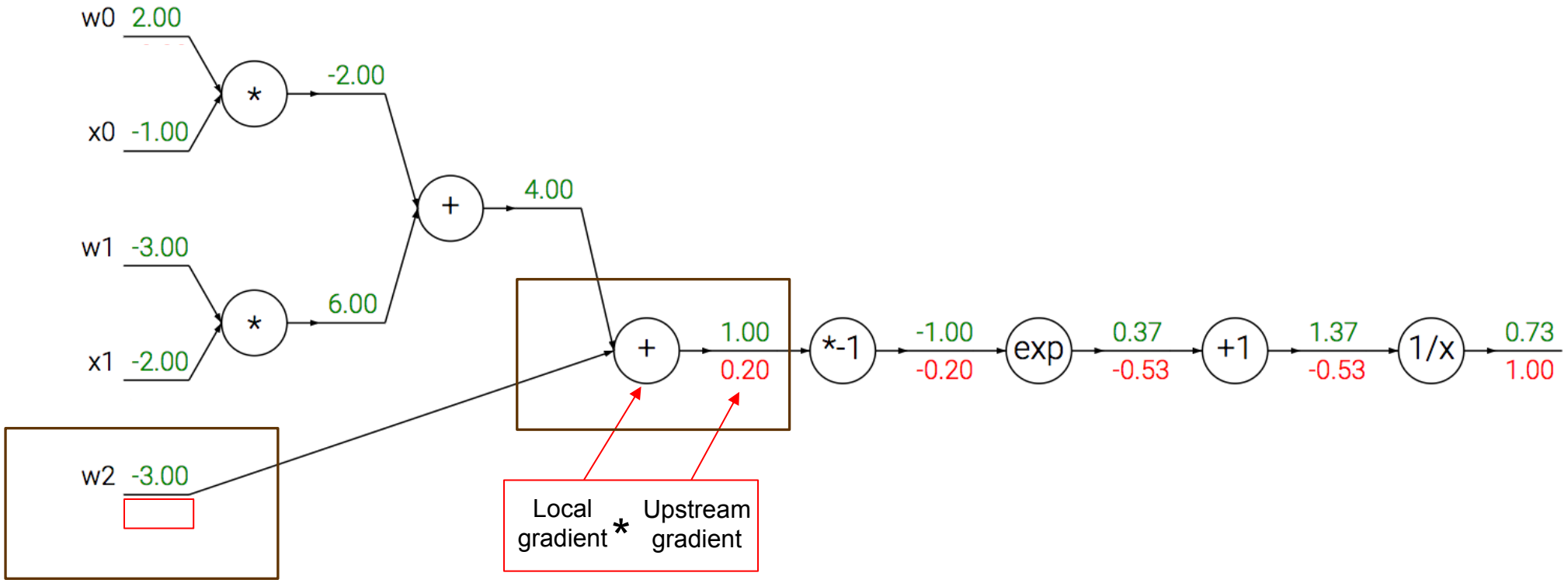
Source: Stanford 231n

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



$$f(x) = ax, \frac{df(x)}{dx} = a$$

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$

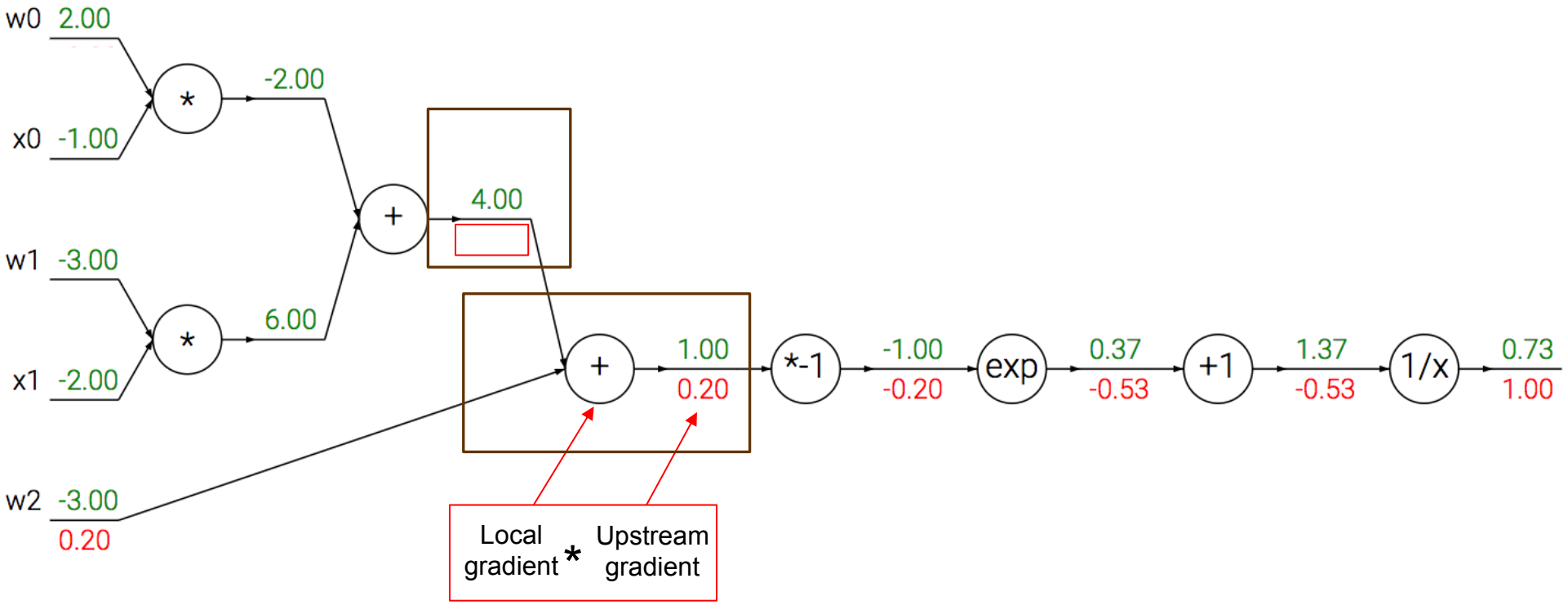


$$f(x) = a + x, \frac{df(x)}{dx} = 1$$

Slide credit: Prof. Svetlana Lazebnik

Source: Stanford 231n

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



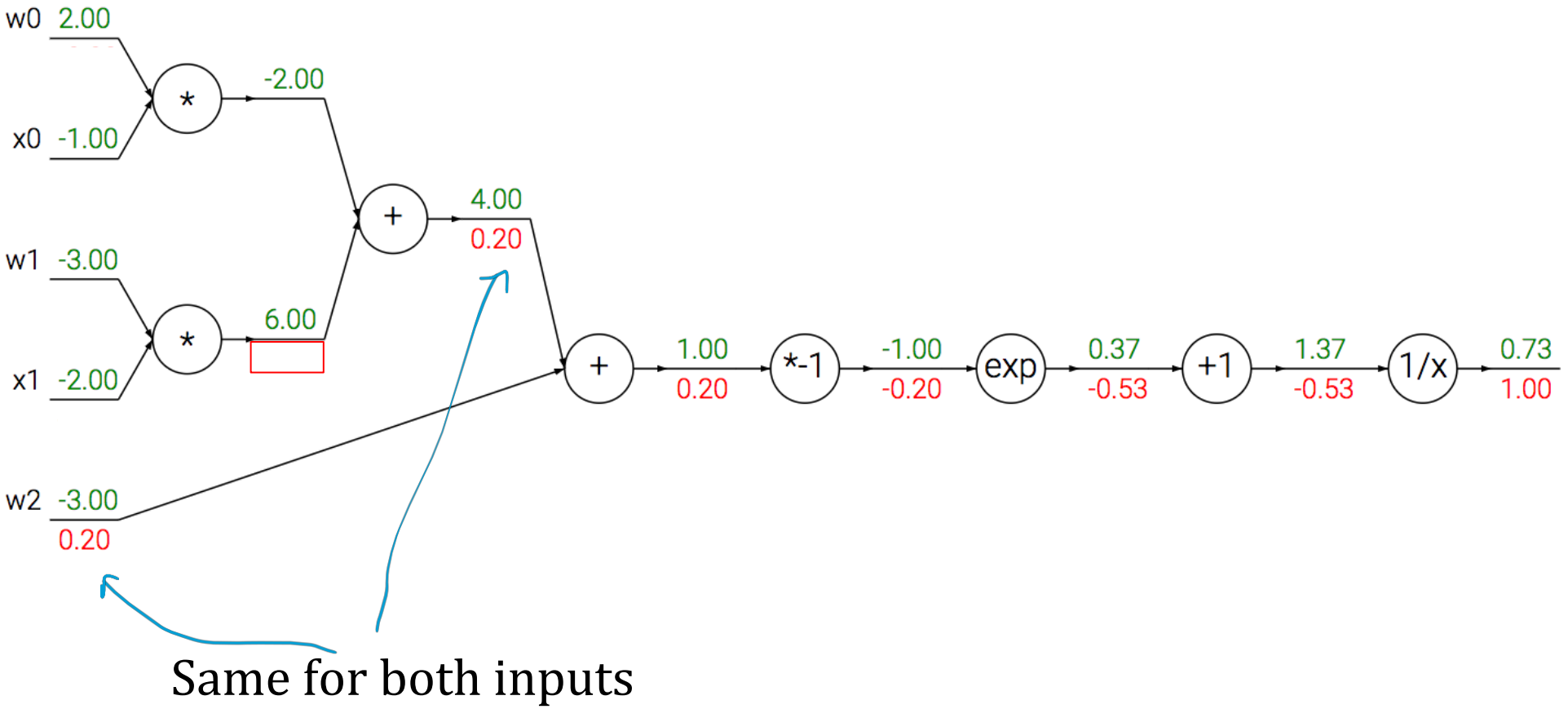
1 0.20

$$f(x) = a + x, \frac{df(x)}{dx} = 1$$

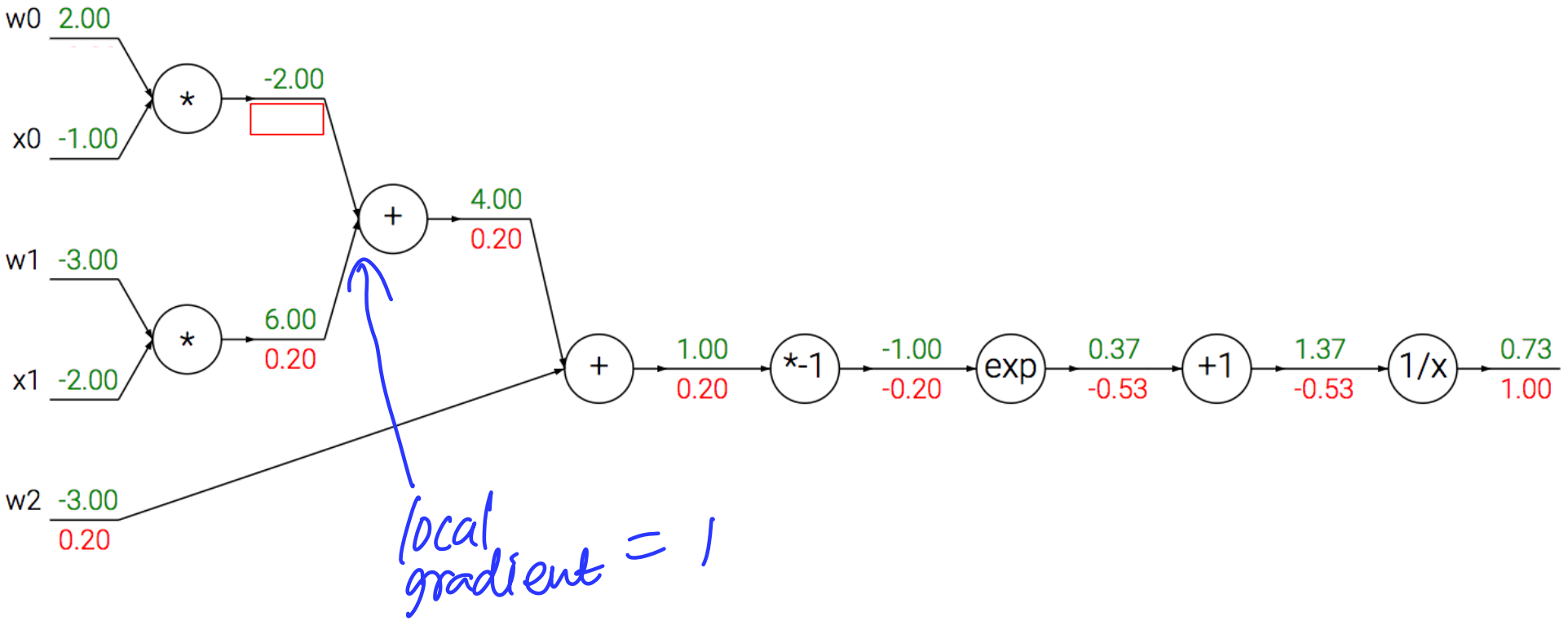
Slide credit: Prof. Svetlana Lazebnik

Source: Stanford 231n

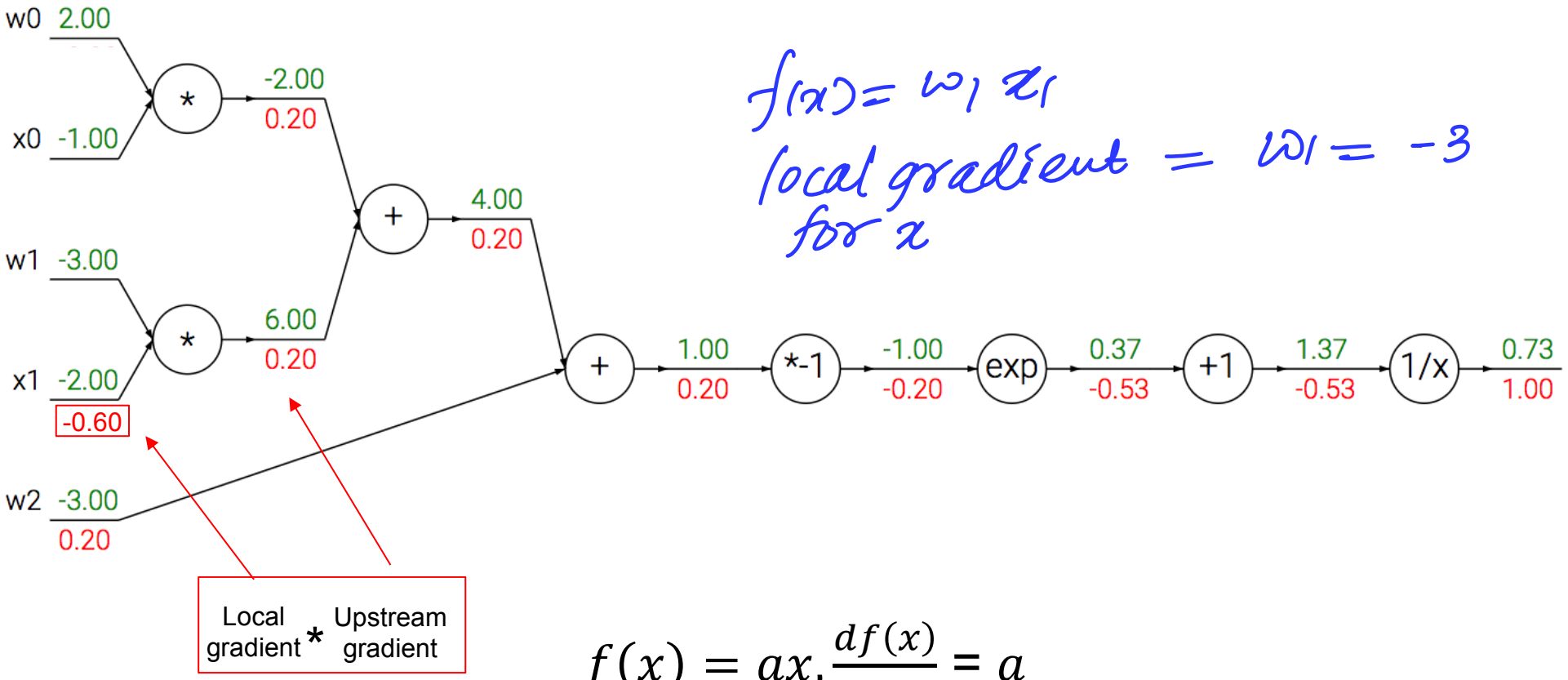
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$

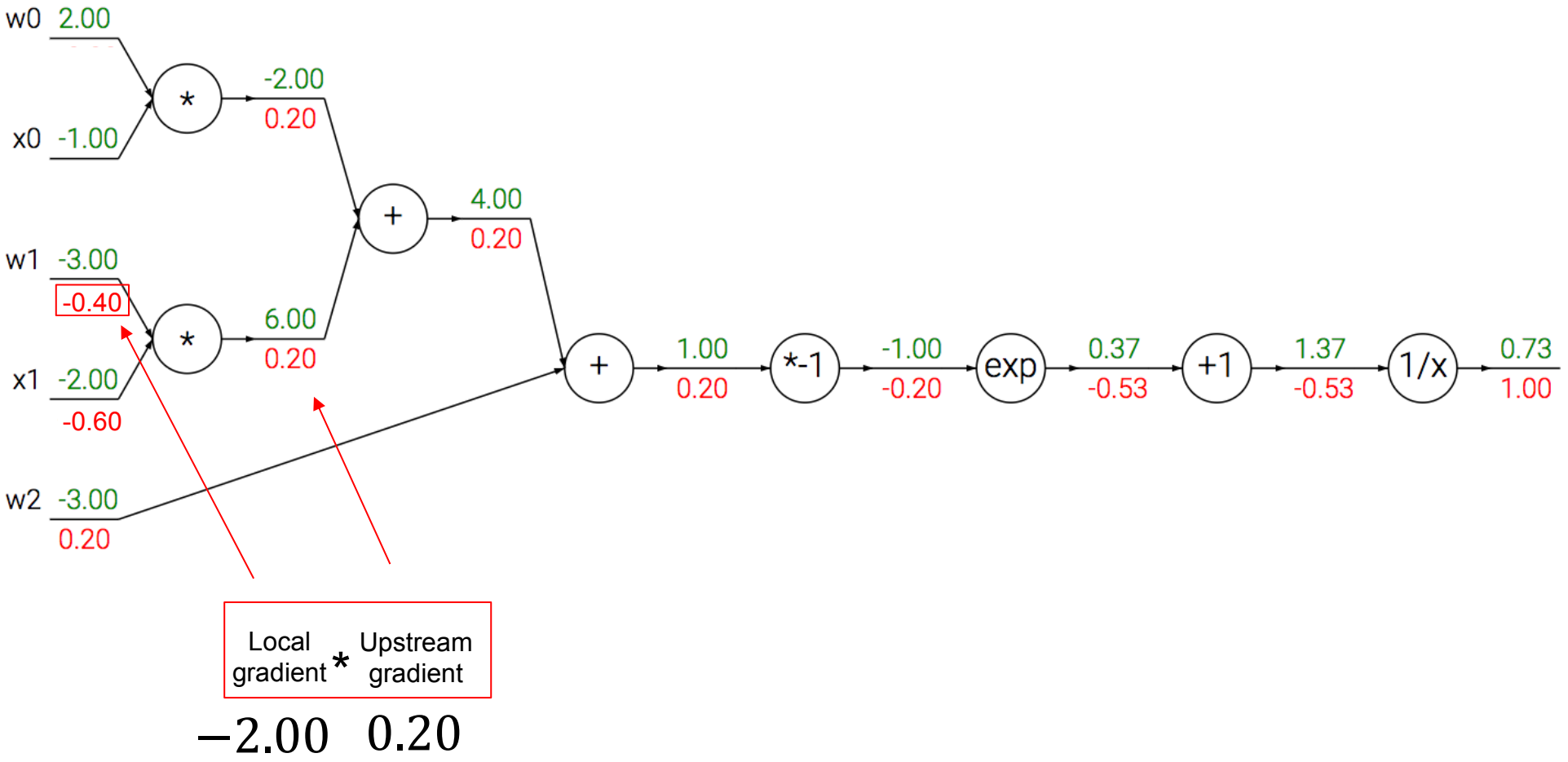


$f(x) = w_1 x_1$

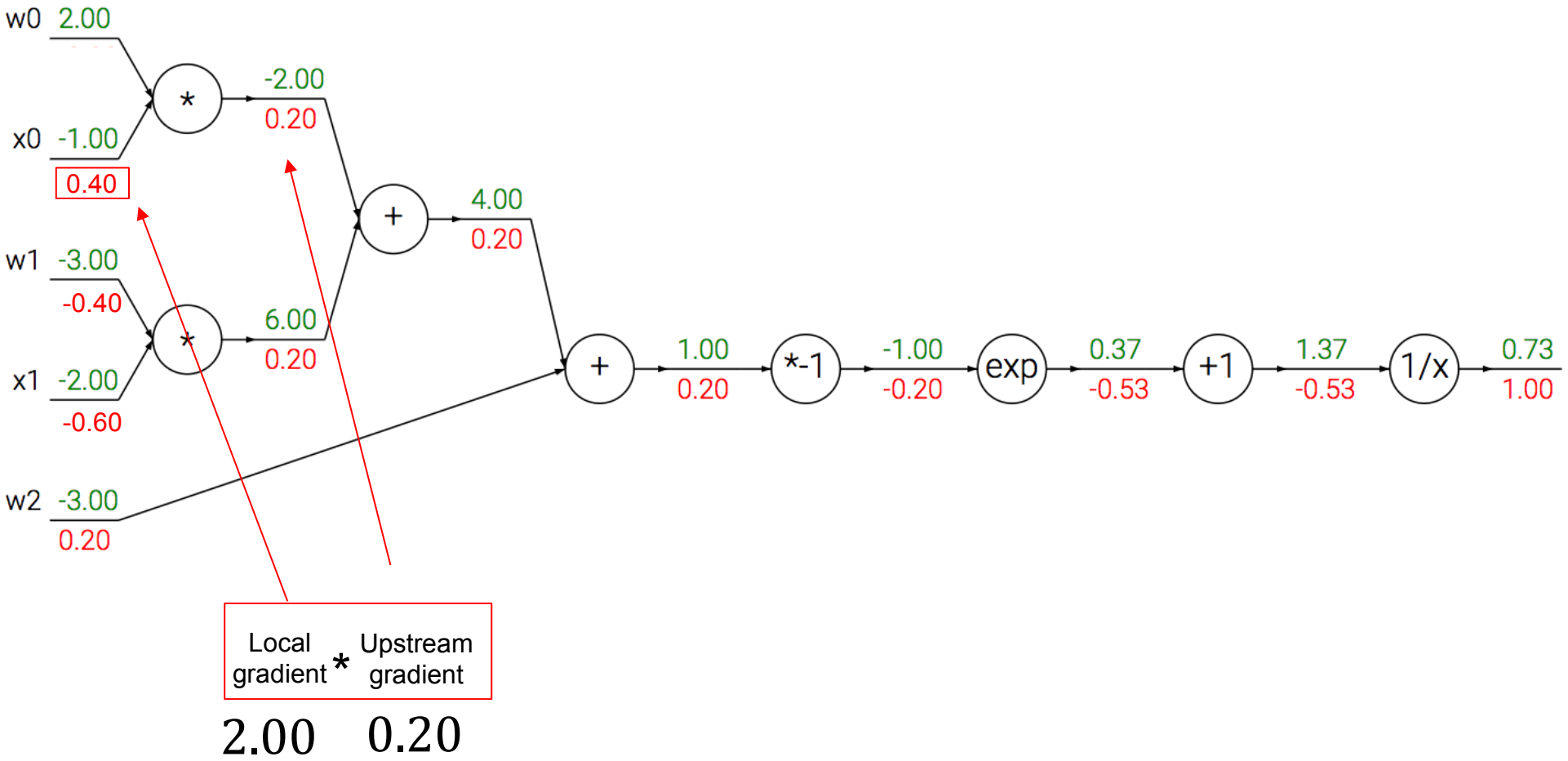
local gradient = $w_1 = -3$ for x

$$f(x) = ax, \frac{df(x)}{dx} = a$$

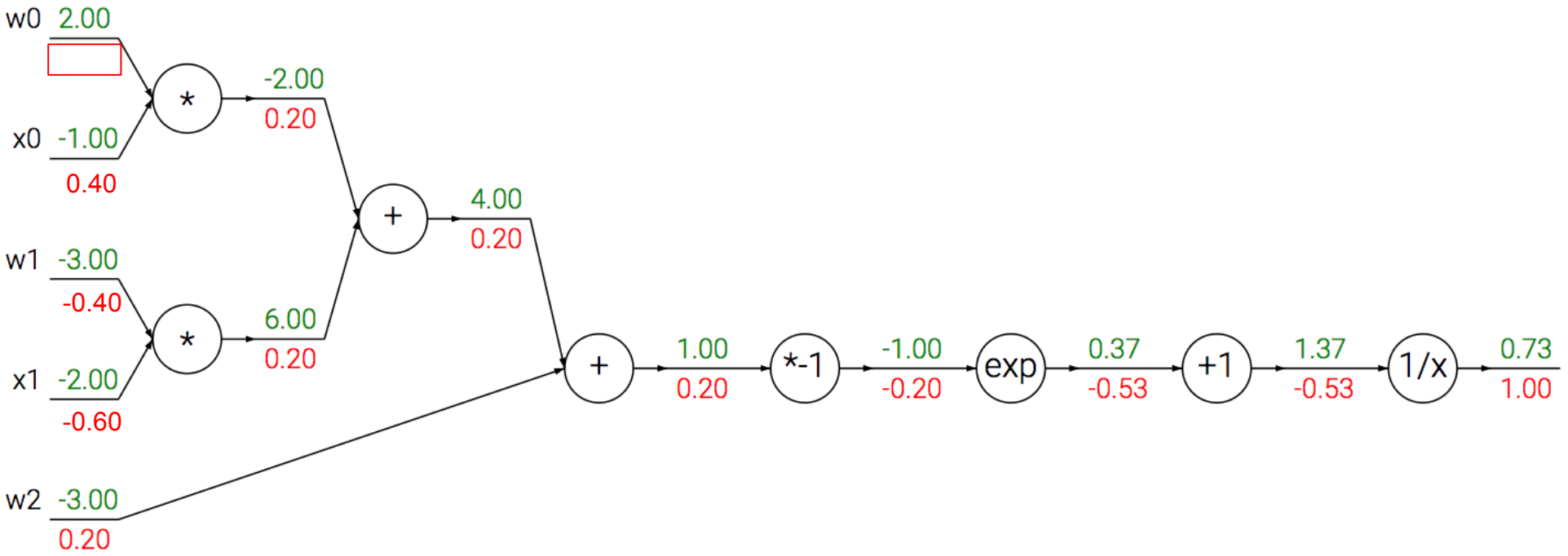
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



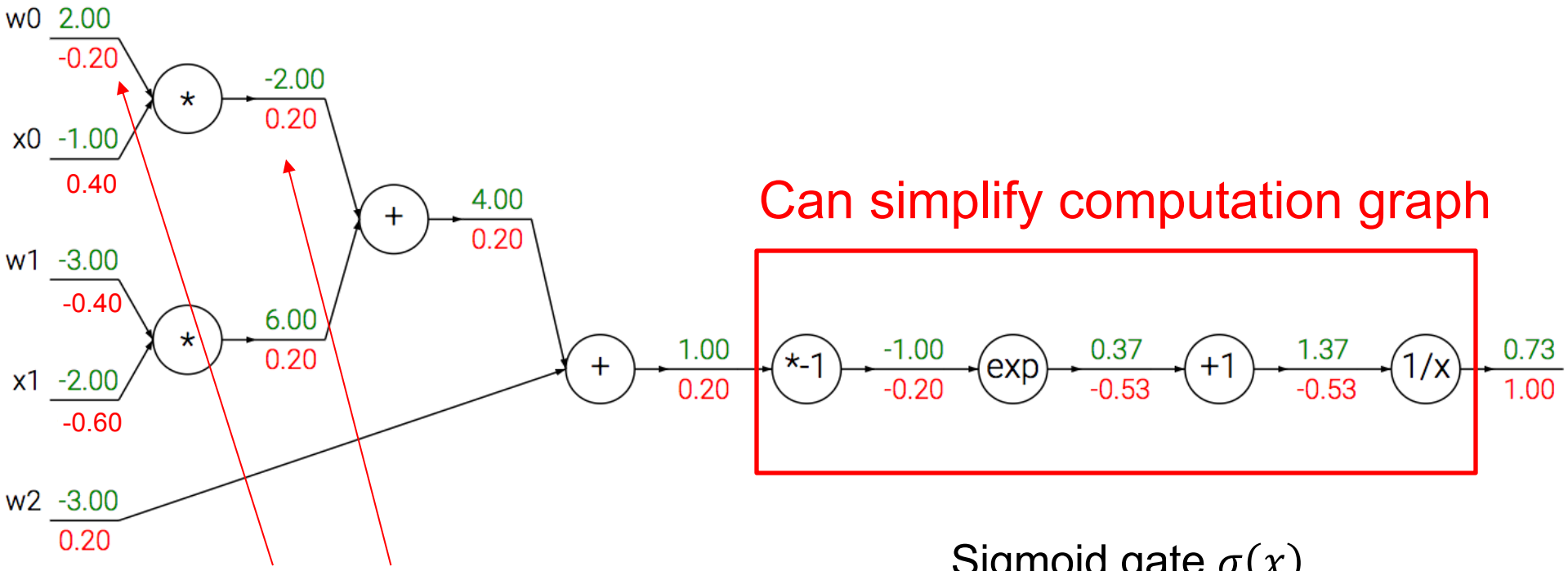
$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Slide credit: Prof. Svetlana Lazebnik

Source: Stanford 231n

$$f(x,w) = \frac{1}{1+\exp[-(w^{(0)}x^{(0)} + w^{(1)}x^{(1)} + w^{(2)})]}$$



Can simplify computation graph

Local gradient * Upstream gradient

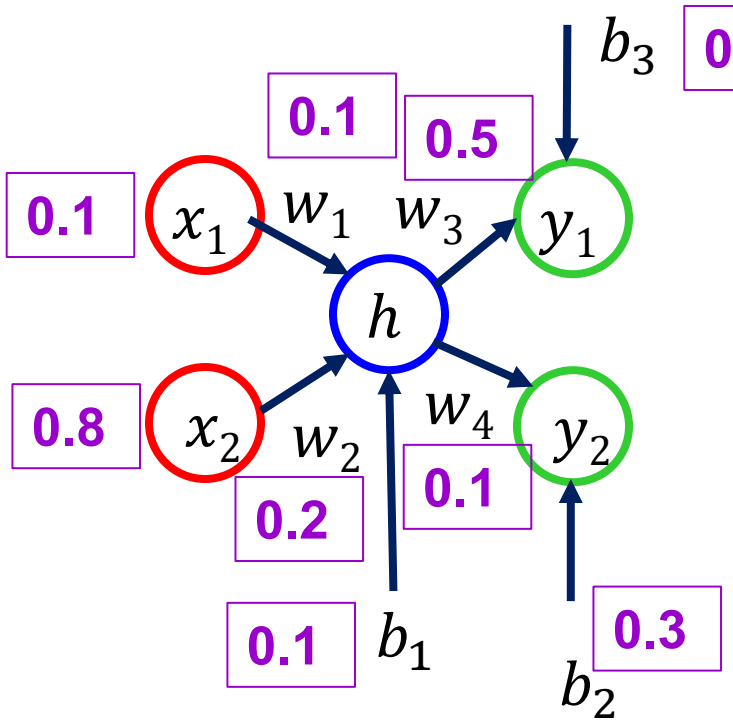
$-1.00 \quad 0.20$

Sigmoid gate $\sigma(x)$

$$\sigma'(x) = \sigma(x)\big(1-\sigma(x)\big)$$

$$\sigma(1)\big(1-\sigma(1)\big) = 0.73 * (1-0.73) = 0.20$$

Weight update example



Forward path

$a_1 = w_1 x_1 + w_2 x_2 + b_1$ **0.27**

$h = f(a_1) = \sigma(a_1) = \frac{1}{1+e^{-a_1}}$ **0.567**

$a_2 = w_4 h + b_2$ **0.3567**

$y_2 = f(a_2) = \sigma(a_2) = \frac{1}{1+e^{-a_2}}$ **0.5882**

$a_3 = w_3 h + b_3$ **0.2283**

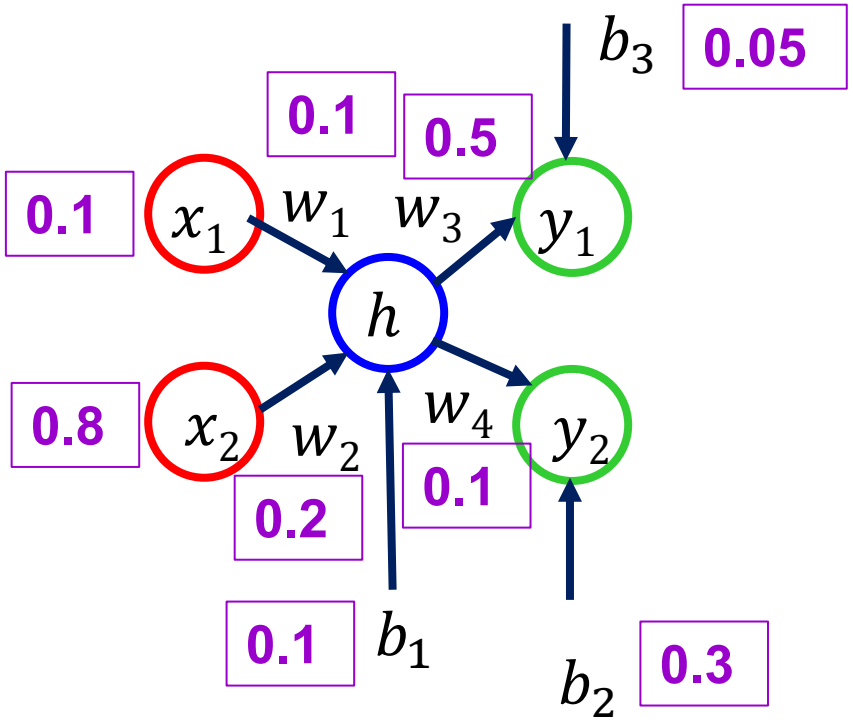
$y_1 = f(a_3) = \sigma(a_3) = \frac{1}{1+e^{-a_3}}$ **0.5568**

$x_1 = 0.1, x_2 = 0.8, w_1 = 0.1, w_2 = 0.2, w_3 = 0.5, w_4 = 0.1, b_1 = 0.1, b_2 = 0.3, b_3 = 0.05,\ N = 1$ (one sample)

True $\tilde{y}$: $\tilde{y_1} = 1, \tilde{y_2} = 0$ $\quad L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \|\tilde{y}_i - y_i\|_2^2 = (1 - \mathbf{0.5568})^2 + (0 - \mathbf{0.5882})^2$ **0.5424**

True NN output

Weight update example



$a_1 = w_1x_1 + w_2x_2 + b_1$

$h = f(a_1) = \sigma(a_1) = \frac{1}{1+e^{-a_1}}$

$a_2 = w_4h + b_2$

$y_2 = f(a_2) = \sigma(a_2) = \frac{1}{1+e^{-a_2}}$

$a_3 = w_3h + b_3$

$y_1 = f(a_3) = \sigma(a_3) = \frac{1}{1+e^{-a_3}}$

$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\|\tilde{\boldsymbol{y}}_i - \boldsymbol{y}_i\|_2^2$

$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\{(\tilde{y}_{1i} - y_{1i})^2 + (\tilde{y}_{2i} - y_{2i})^2\}$

$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\{(\tilde{y}_{1i} - y_{1i})^2\} + \frac{1}{N}\sum_{i=1}^{N}\{(\tilde{y}_{2i} - y_{2i})^2\}$

$L(\mathbf{w}) = L_1(\mathbf{w}) + L_2(\mathbf{w})$

$\frac{\partial L(\mathbf{w})}{\partial w_k} = \frac{\partial L(\mathbf{w})}{\partial h_k}\frac{\partial h_k}{\partial w_k}$

$\frac{\partial L(\mathbf{w})}{\partial w_3} = \frac{\partial L(\mathbf{w})}{\partial y_1}\frac{\partial y_1}{\partial w_3}$ $= 2(\tilde{y_1} - y_1)(-1)\sigma(a_3)(1 - \sigma(a_3))h$ $= -0.1240$

$w_{3\,\text{new}} \leftarrow w_3 - \alpha\frac{\partial L(\mathbf{w})}{\partial w_3}$

$w_{3\,\text{new}} = 0.5124$

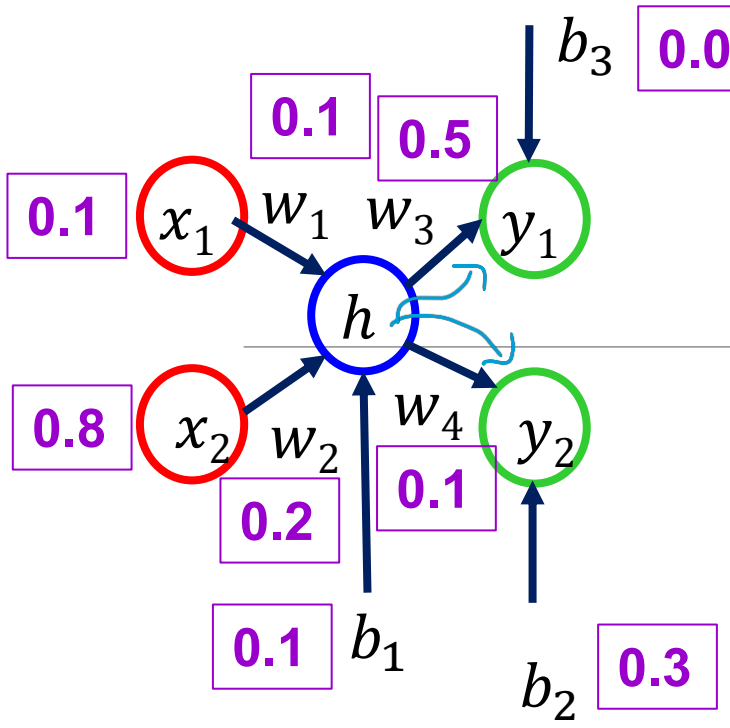
With updated w_3

$y_1 = 0.5843$ $L(\mathbf{w}) = 0.5187$

α =0.1

True $\tilde{y}$: $\tilde{y_1} = 1, \tilde{y_2} = 0$ $L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\|\tilde{\boldsymbol{y}}_i - \boldsymbol{y}_i\|_2^2 = (1 - \mathbf{0.5843})^2 + (0 - \mathbf{0.5882})^2$ **=0.5187**

Weight update example



$$a_1 = w_1x_1 + w_2x_2 + b_1$$

$$h = f(a_1) = \sigma(a_1) = \frac{1}{1+e^{-a_1}}$$

$$a_3 = w_3h + b_3$$

$$y_1 = f(a_3) = \sigma(a_3) = \frac{1}{1+e^{-a_3}}$$

$$a_2 = w_4h + b_2$$

$$y_2 = f(a_2) = \sigma(a_2) = \frac{1}{1+e^{-a_2}}$$

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\|\tilde{\boldsymbol{y}}_i - \boldsymbol{y}_i\|_2^2$$

$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\{(\tilde{y}_{1i} - y_{1i})^2\} + \frac{1}{N}\sum_{i=1}^{N}\{(\tilde{y}_{2i} - y_{2i})^2\}$$

$$\frac{\partial L(\mathbf{w})}{\partial w_k} = \frac{\partial L(\boldsymbol{w})}{\partial h_k}\frac{\partial h_k}{\partial w_k}$$

$$\frac{\partial L(\mathbf{w})}{\partial w_1} = \frac{\partial L(\boldsymbol{w})}{\partial h}\frac{\partial h}{\partial w_1}$$

$$\frac{\partial h}{\partial w_1} = \frac{\partial h}{\partial a_1}\frac{\partial a_1}{\partial w_1} = \sigma(a_1)(1-\sigma(a_1))x_1$$ **0.0245**

$$\frac{\partial L(\boldsymbol{w})}{\partial h} = \frac{\partial L(\boldsymbol{w})}{\partial y_1}\frac{\partial y_1}{\partial h} + \frac{\partial L(\boldsymbol{w})}{\partial y_2}\frac{\partial y_2}{\partial h} = \begin{Bmatrix} 2(\tilde{y}_1 - y_1)(-1)(\sigma(a_3)(1-\sigma(a_3))w_3) + \\ 2(\tilde{y}_2 - y_2)(-1)\sigma(a_2)(1-\sigma(a_2))w_4 \end{Bmatrix}$$ **-0.080**

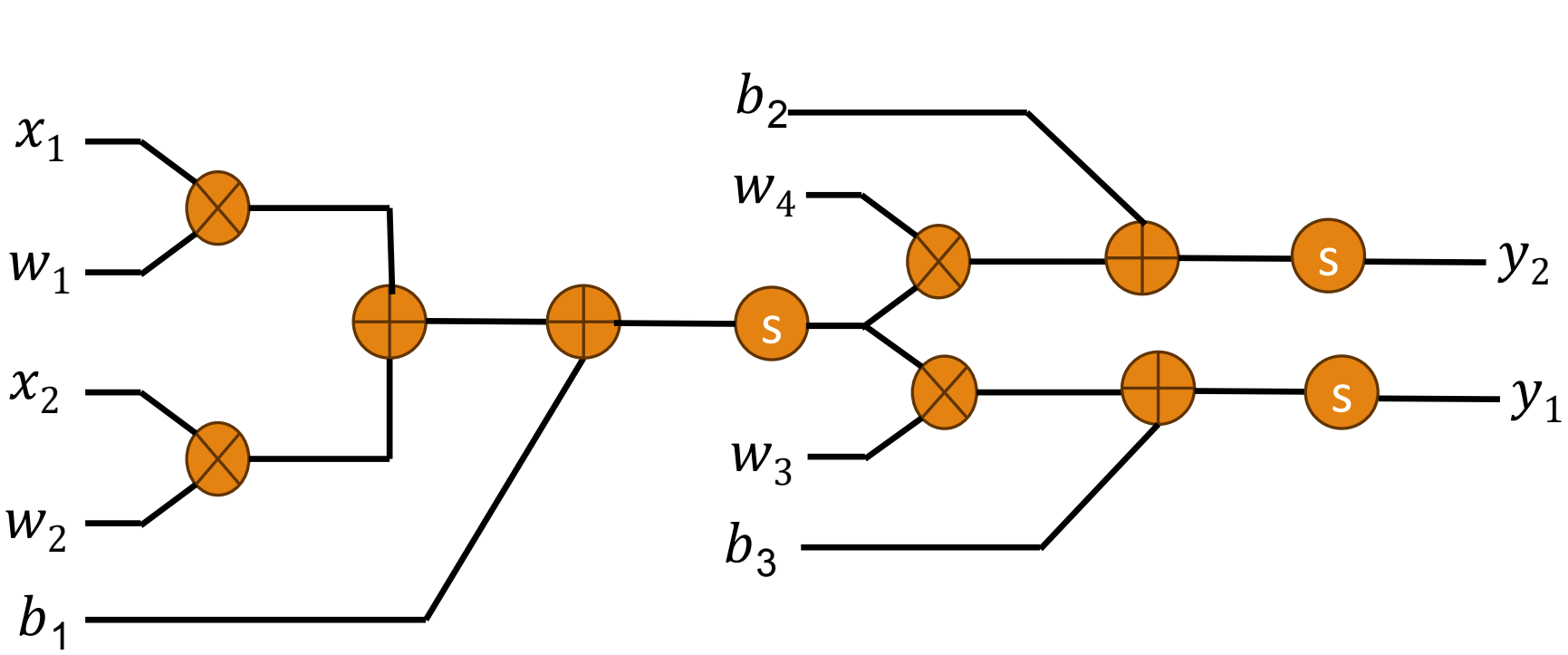
$$\frac{\partial L(\mathbf{w})}{\partial w_1} = \frac{\partial L(\boldsymbol{w})}{\partial h}\frac{\partial h}{\partial w_1}$$ **-0.0020**

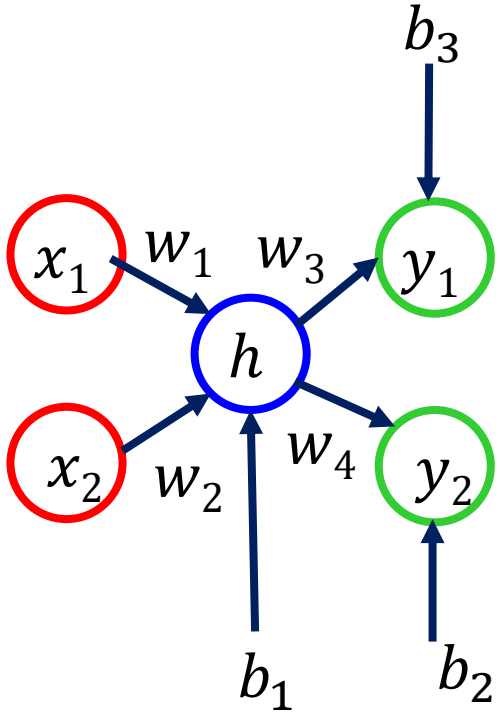
$$w_{1\text{new}} \leftarrow w_1 - \alpha\frac{\partial L(\mathbf{w})}{\partial w_1}$$

$w_{1\text{new}}$ **0.1002**

α =0.1

Weight update example

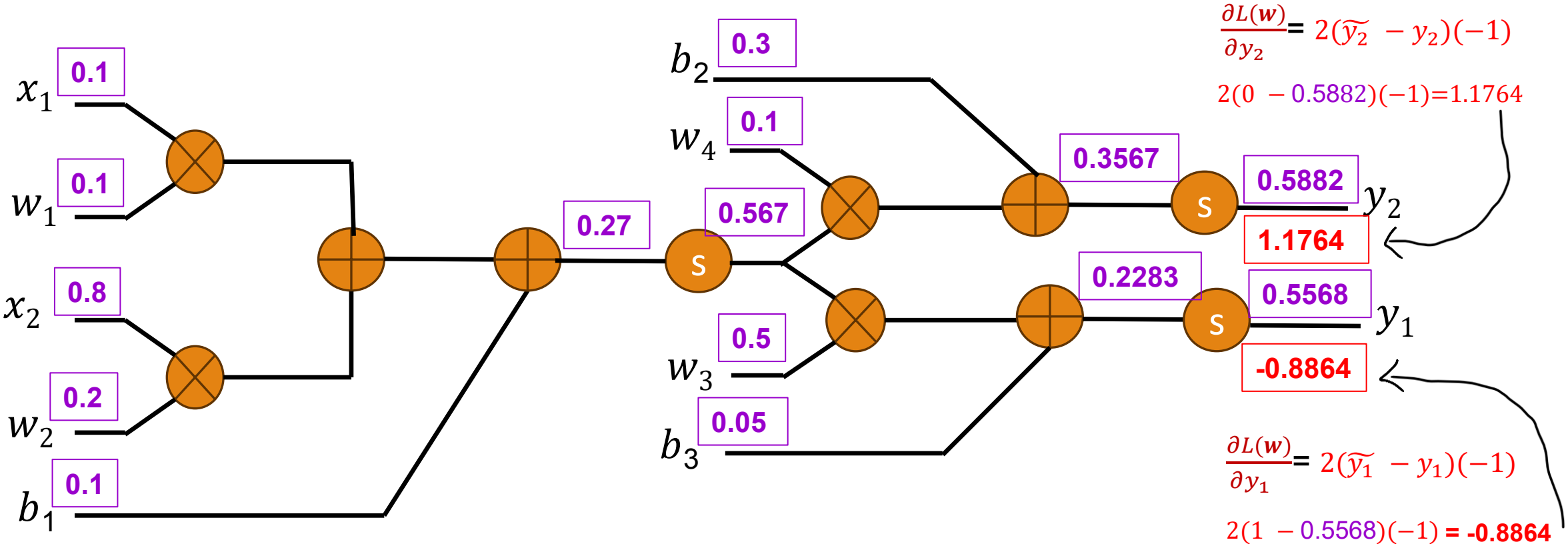




Weight update example

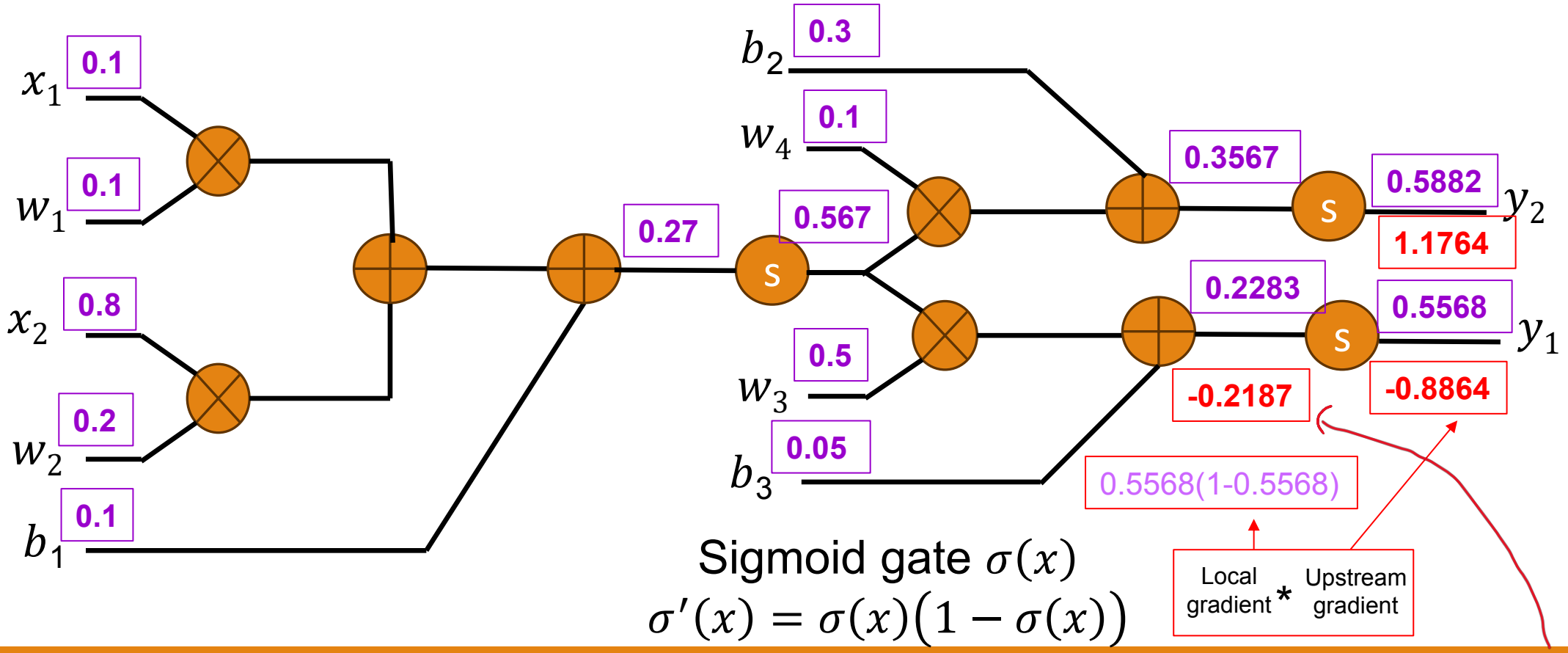
$$L(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\{(\widetilde{y_{1i}} - y_{1i})^2 + (\widetilde{y_{2i}} - y_{2i})^2\}$$

$N = 1$ (one sample)

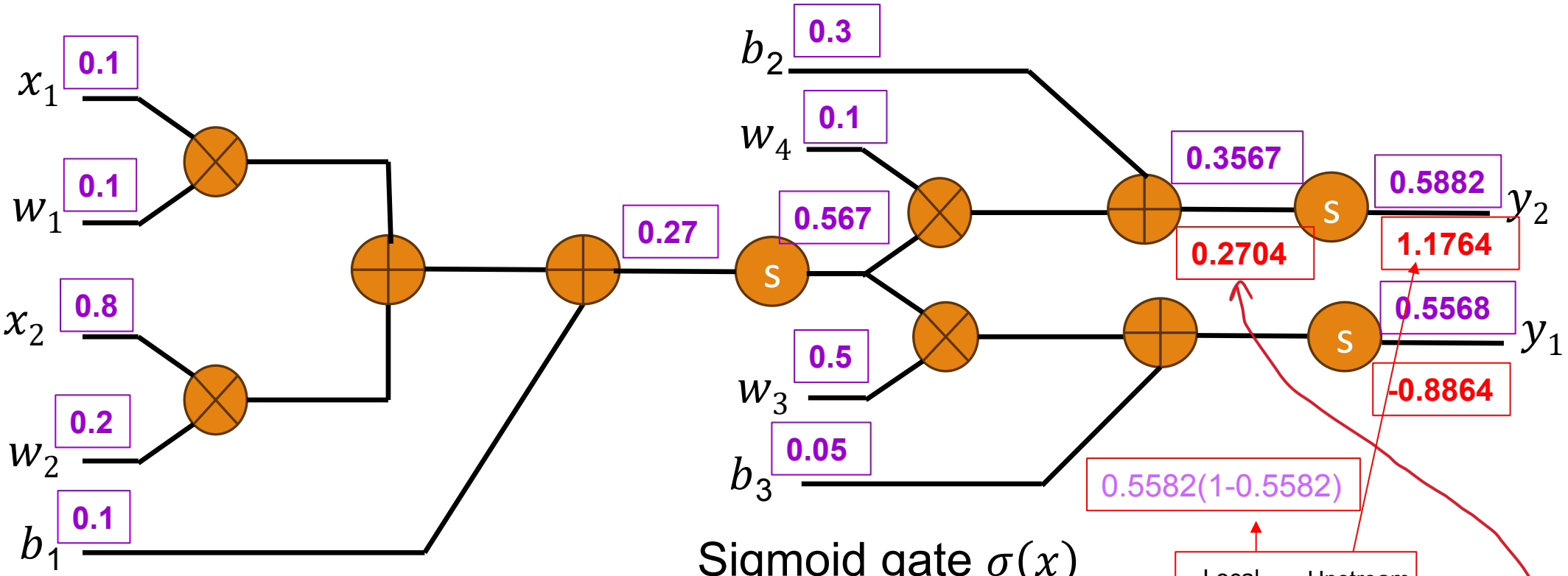


True $\widetilde{y}$: $\widetilde{y_1}$= 1, $\widetilde{y_2}$= 0

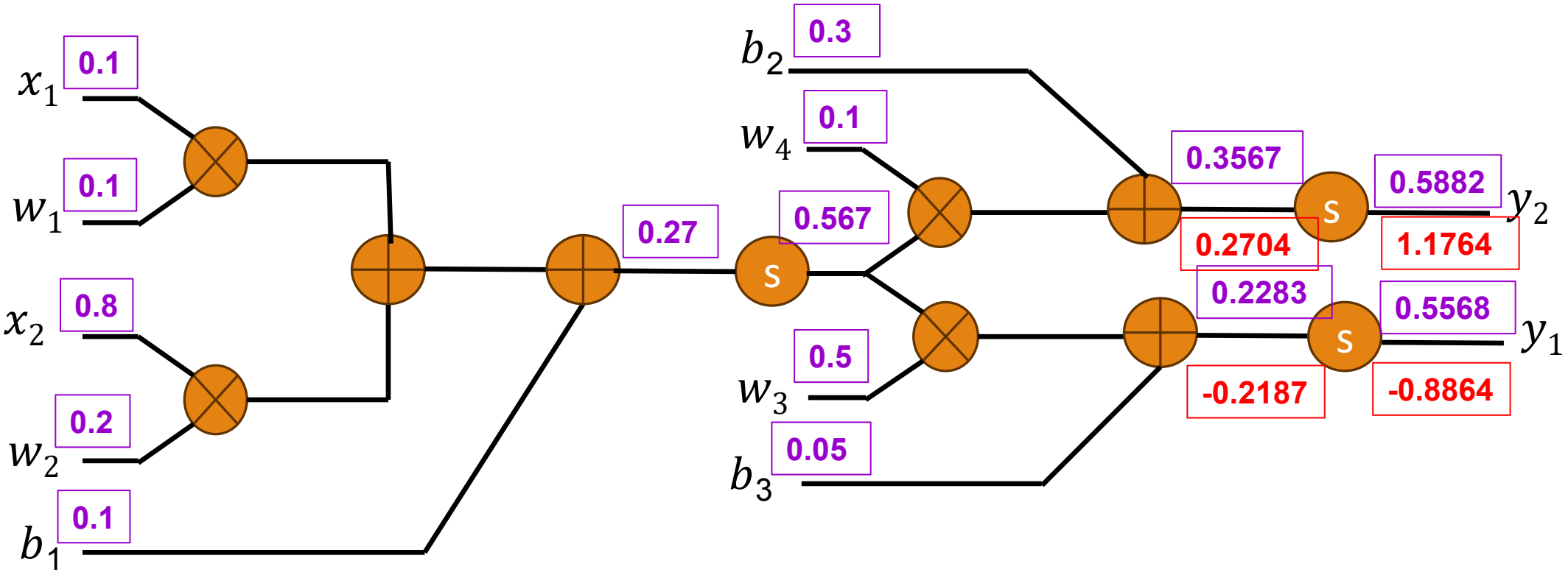
Weight update example



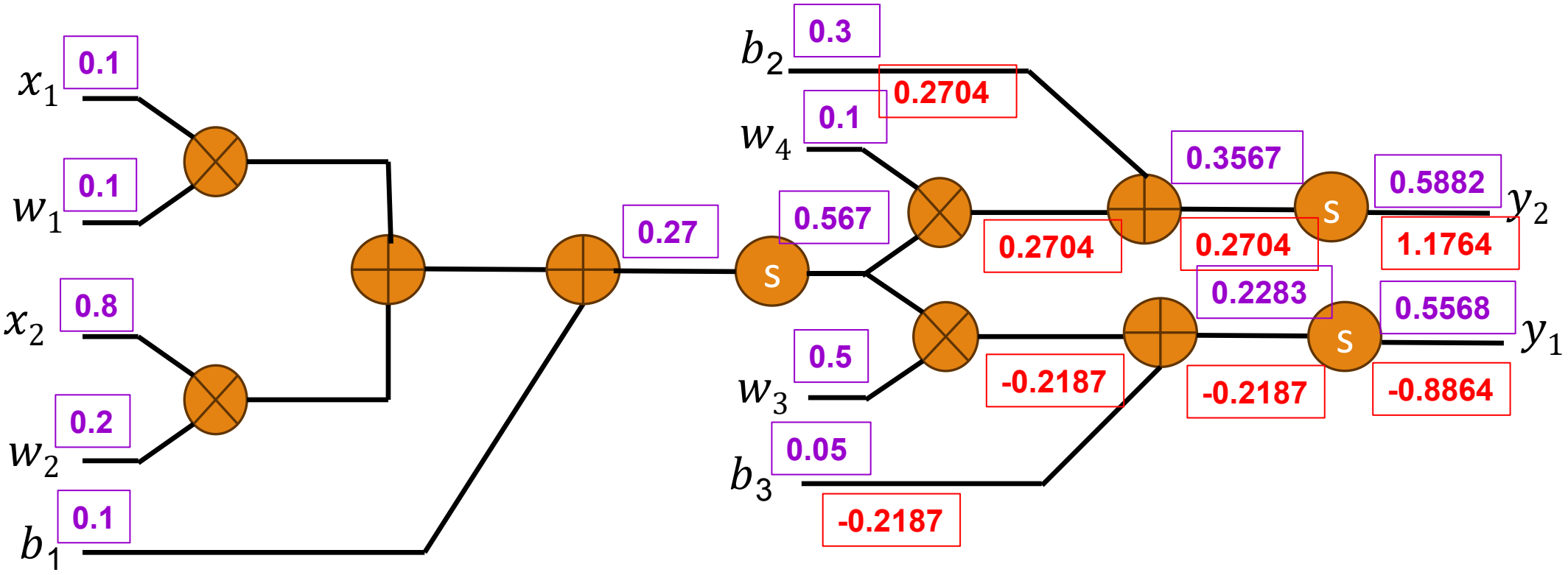
Weight update example



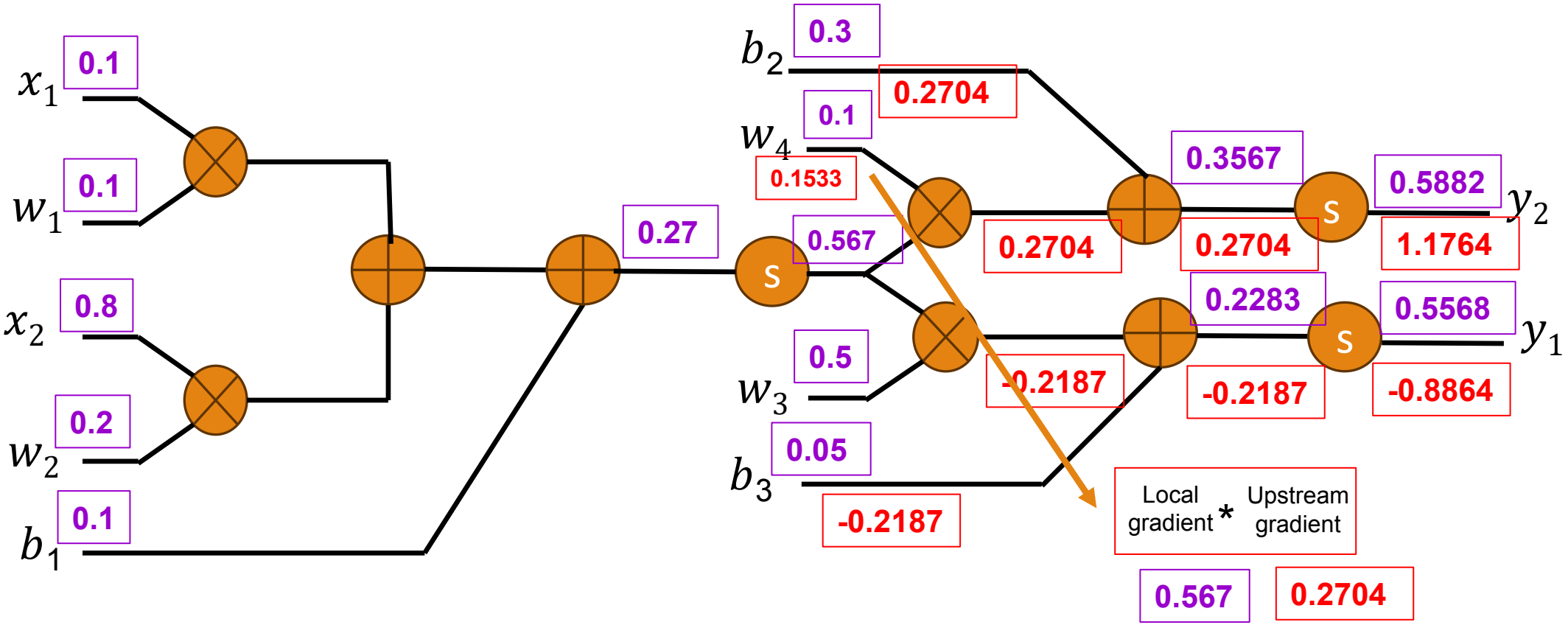
Weight update example



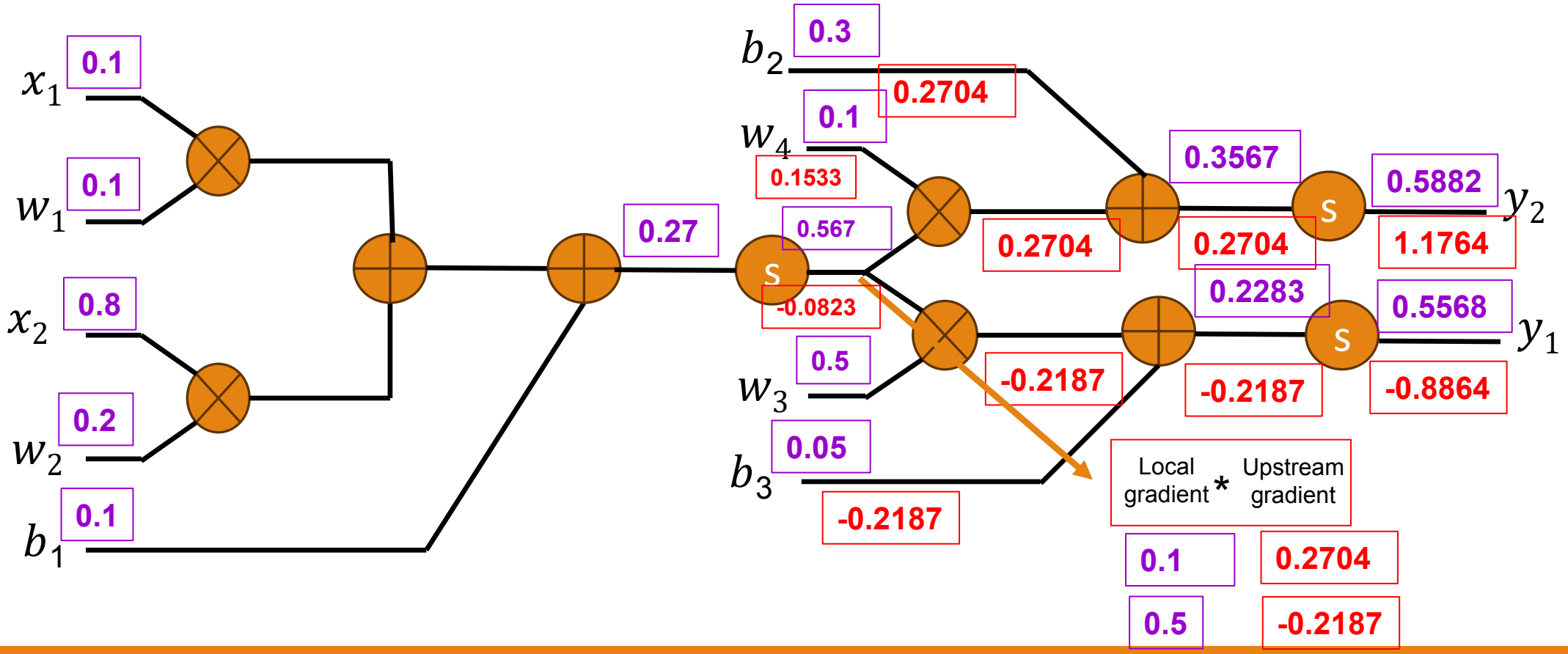
Weight update example



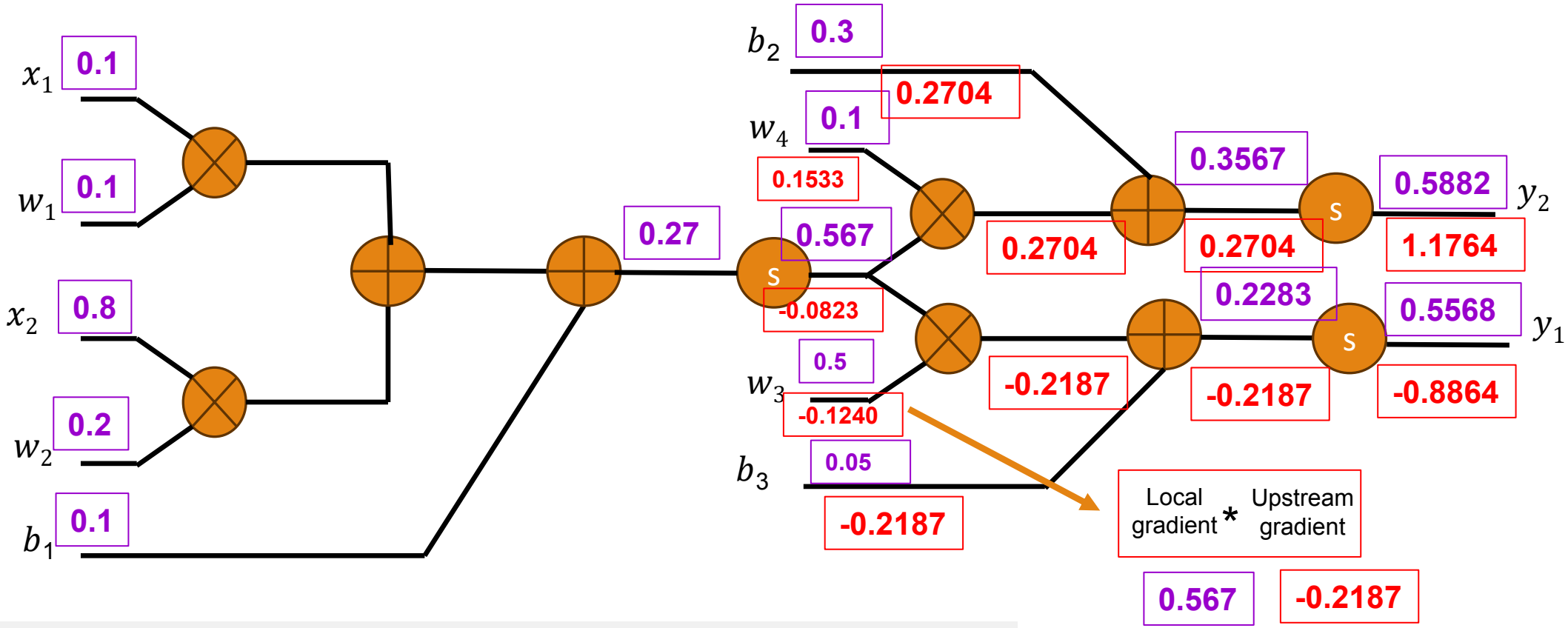
Weight update example



Weight update example

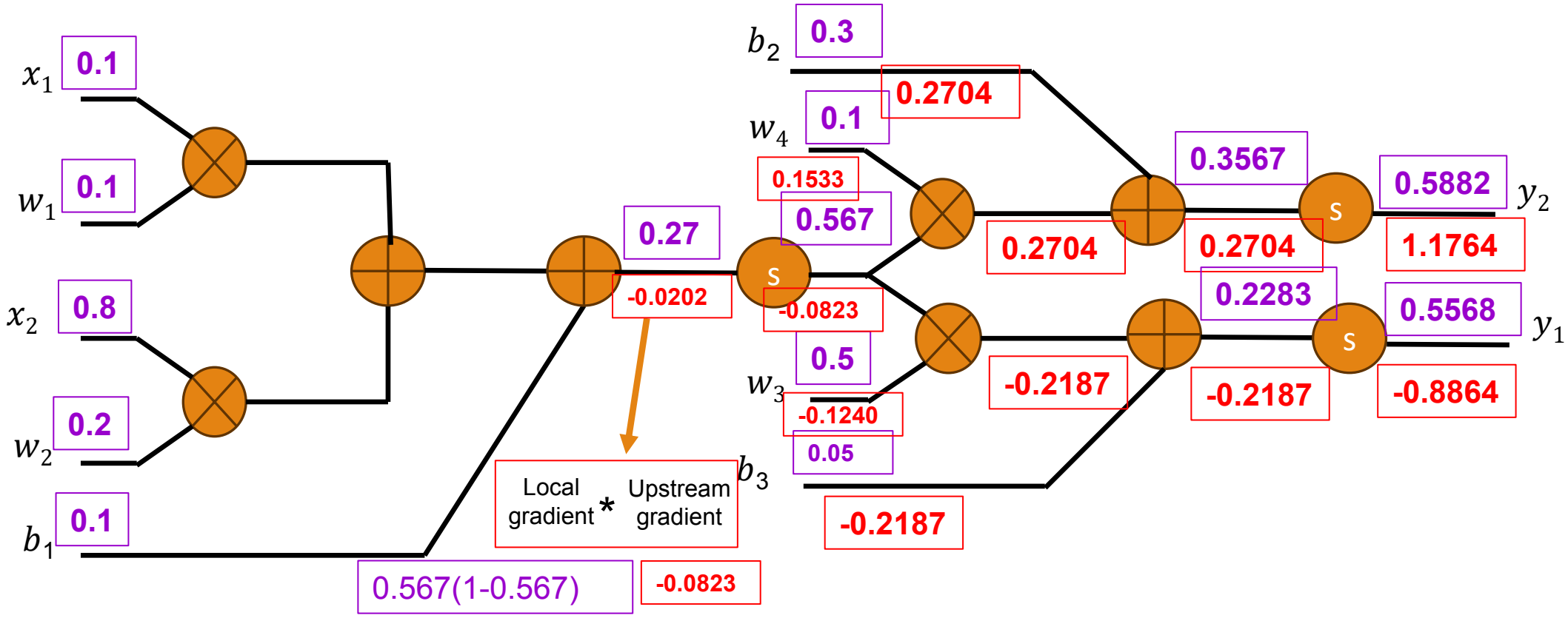


Weight update example

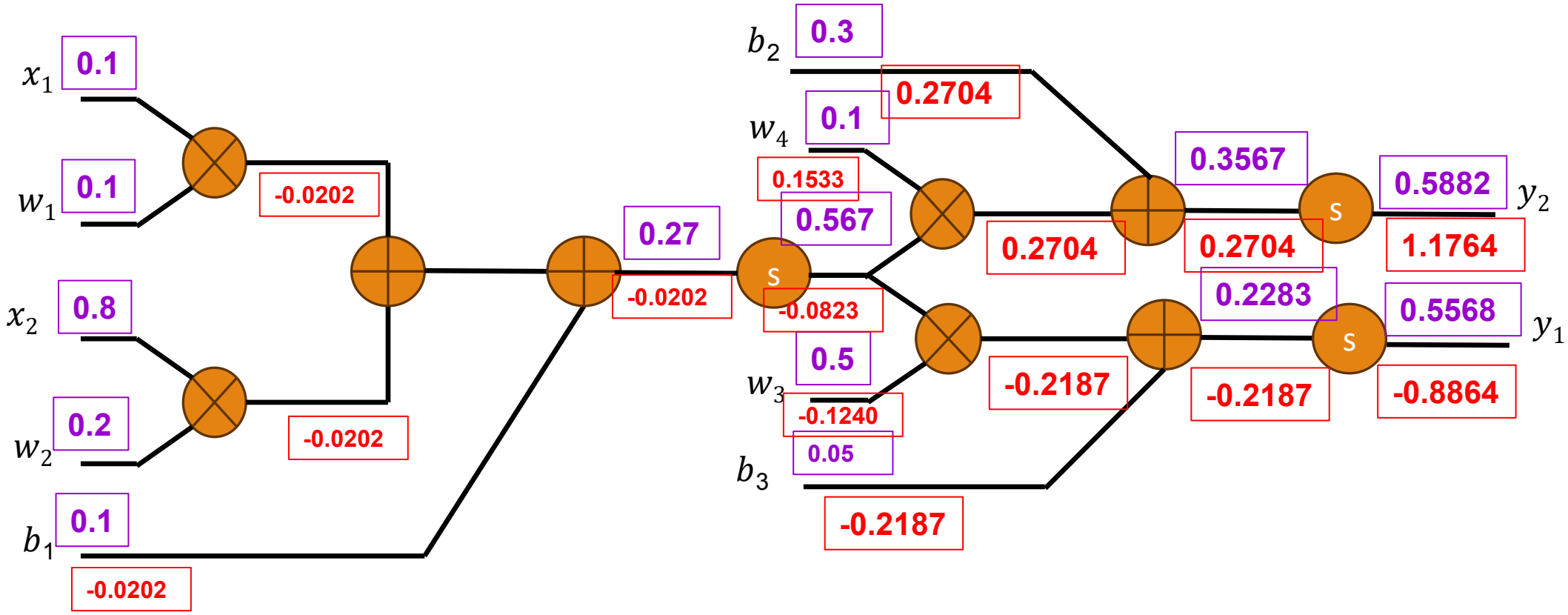


$$\frac{\partial L(\mathbf{w})}{\partial w_3} = \frac{\partial L(\boldsymbol{w})}{\partial y_1}\frac{\partial y_1}{\partial w_3} = 2(\widetilde{y_1} - y_1)\,(-1)\sigma\,(a_3)\,(1-\sigma\,(a_3))h = -0.1240$$

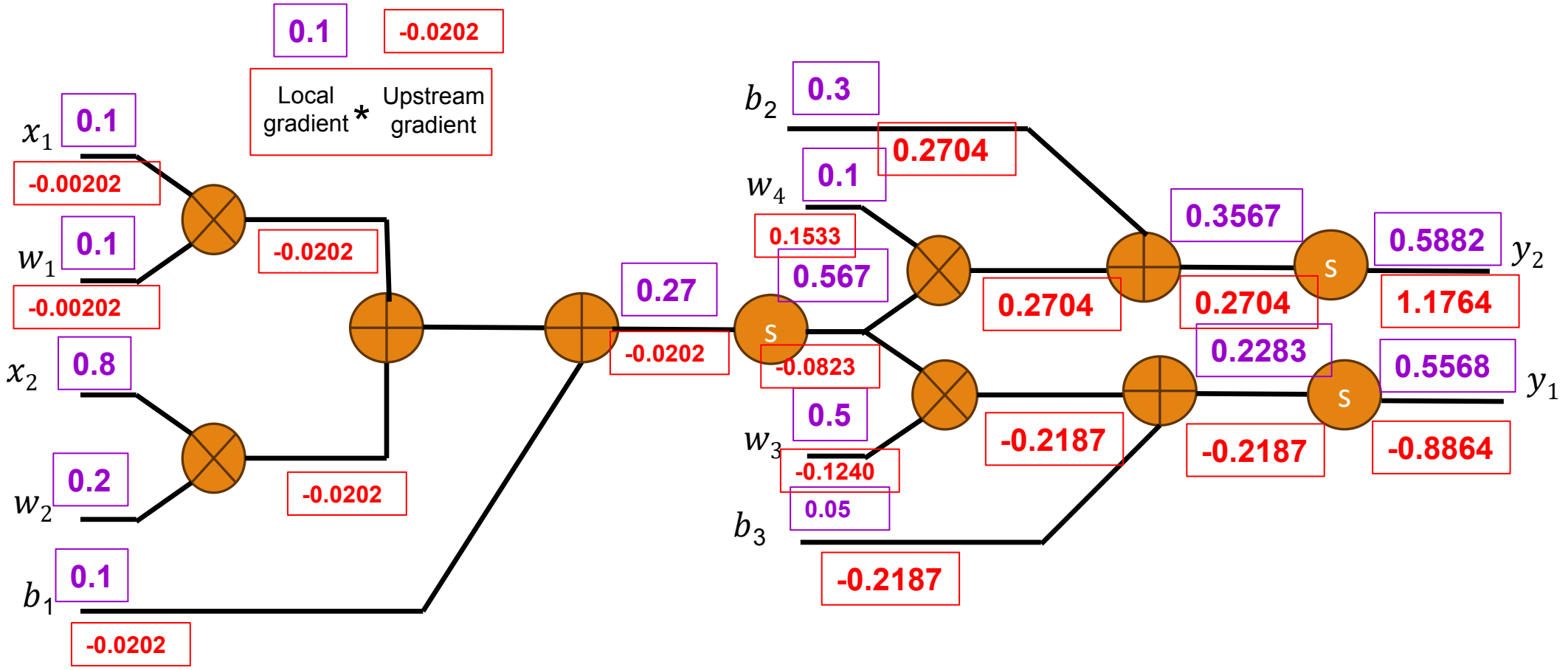
Weight update example



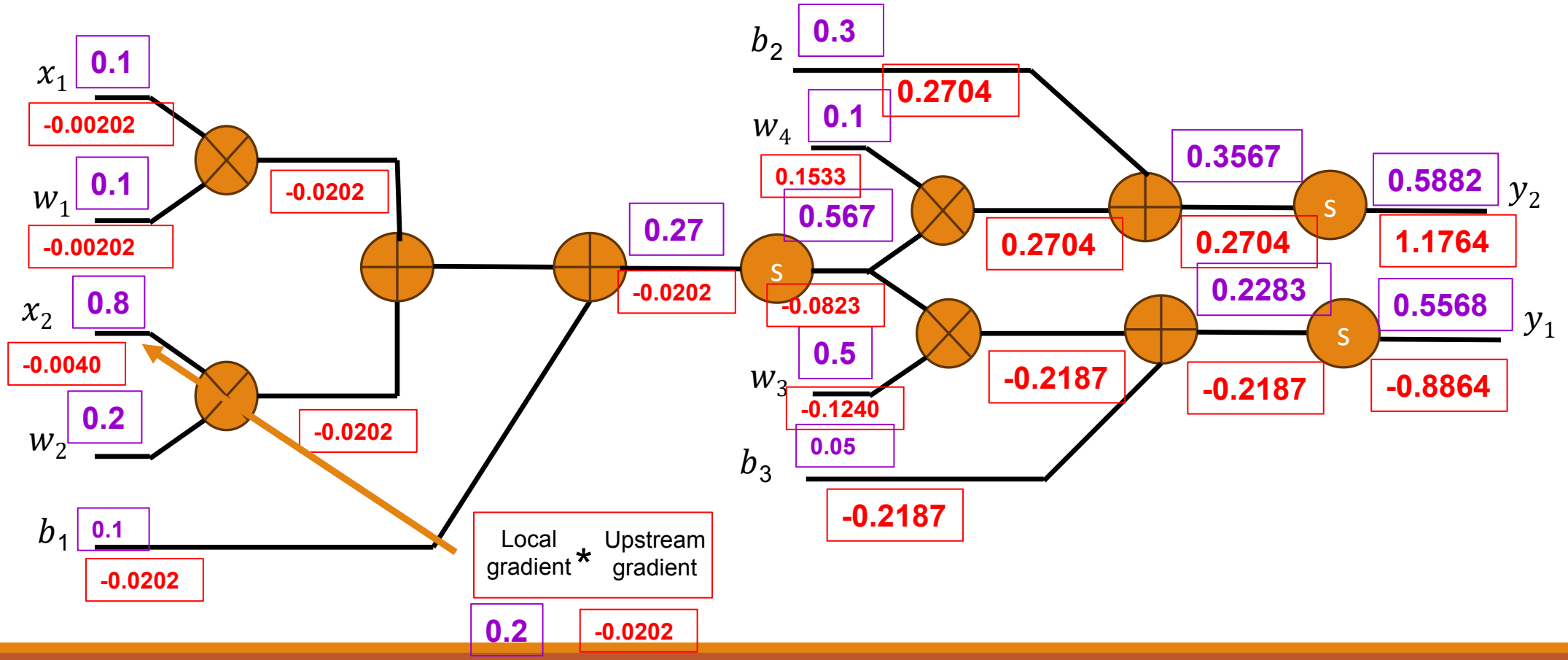
Weight update example



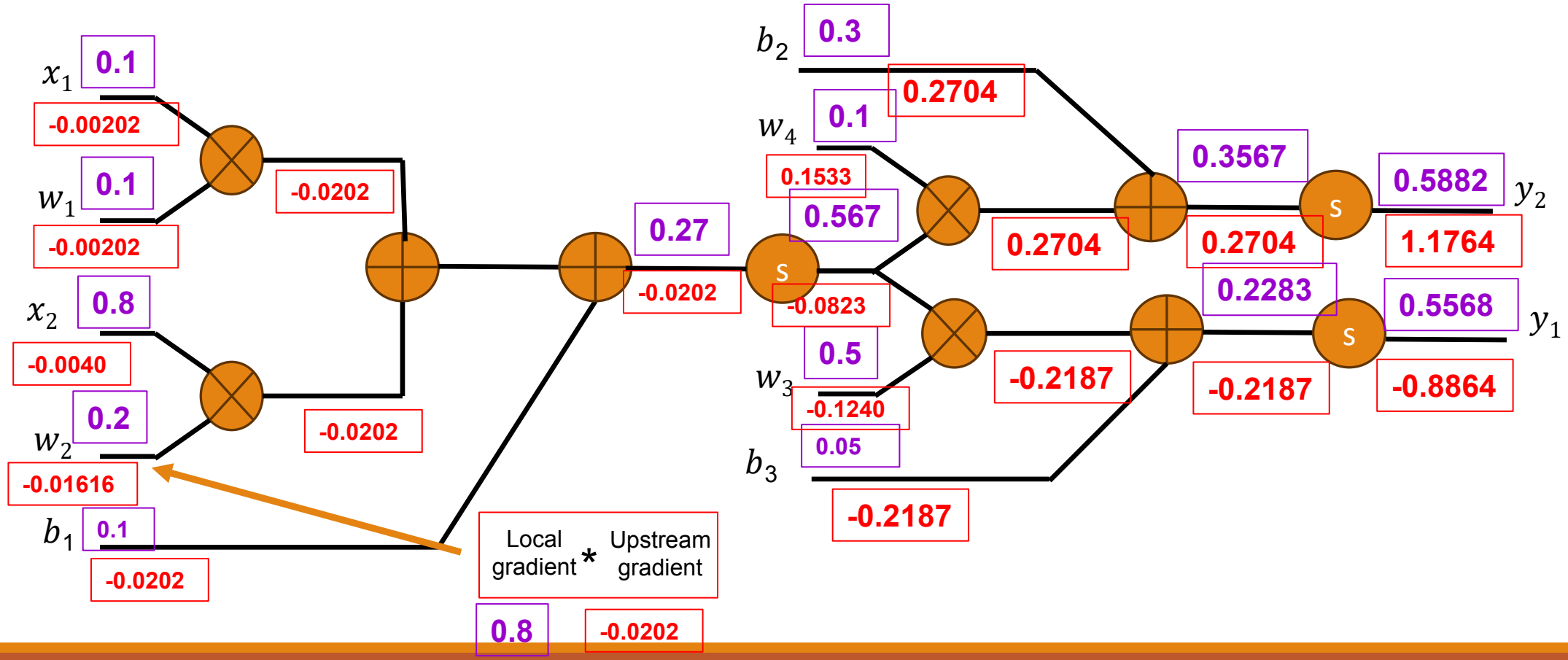
Weight update example



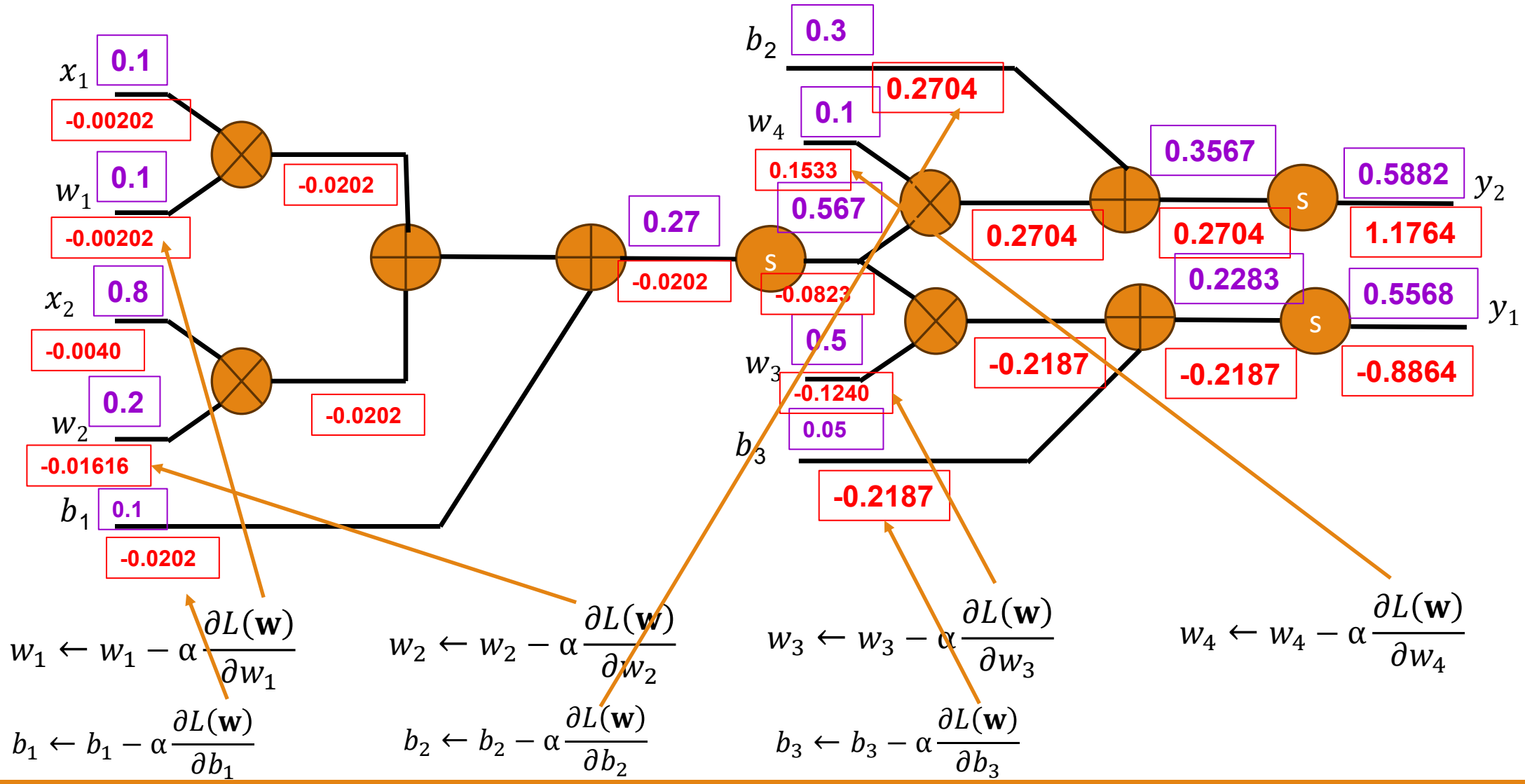
Weight update example



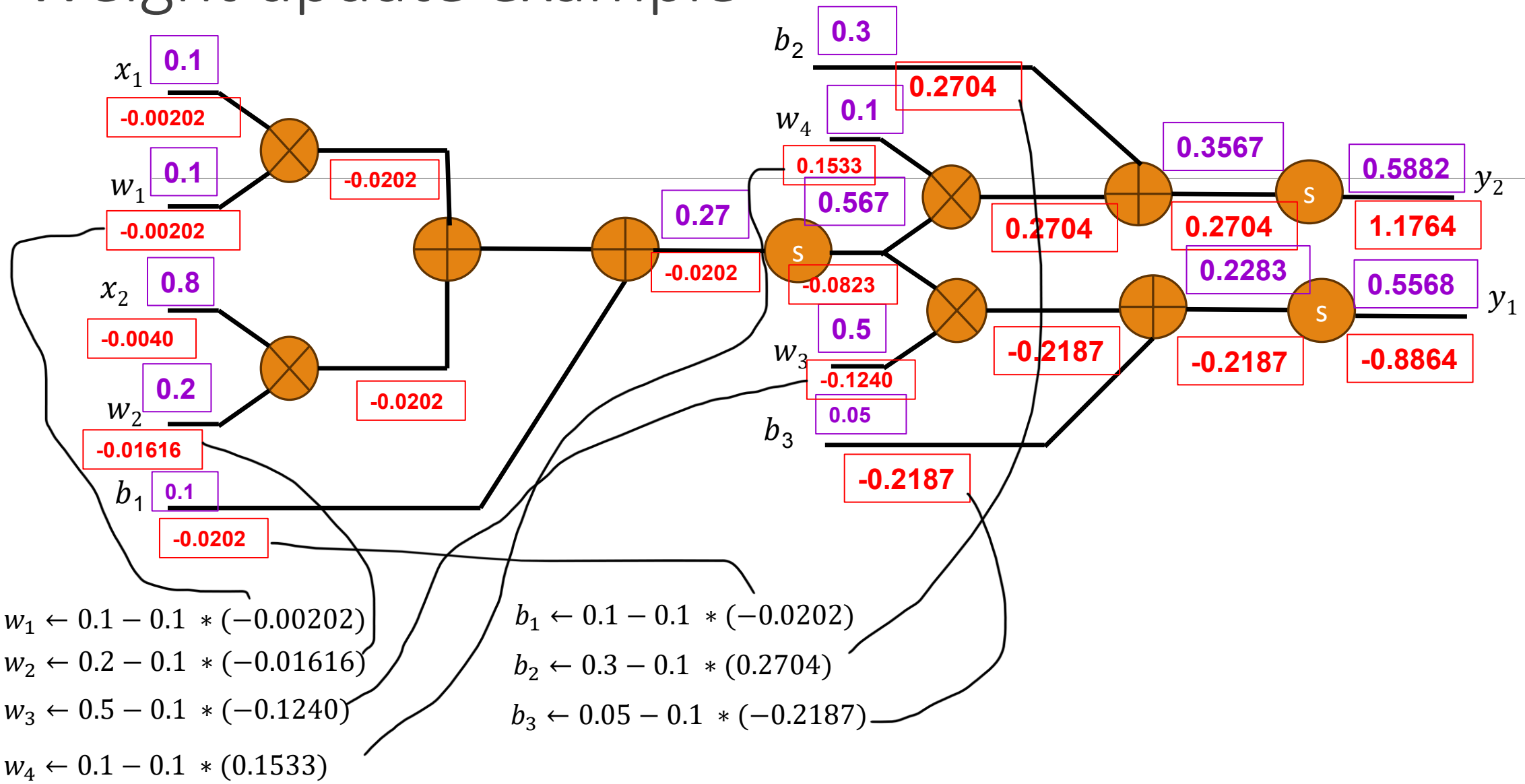
Weight update example



Weight update example



Weight update example



$w_1 \leftarrow 0.1 - 0.1 * (-0.00202)$

$w_2 \leftarrow 0.2 - 0.1 * (-0.01616)$

$w_3 \leftarrow 0.5 - 0.1 * (-0.1240)$

$w_4 \leftarrow 0.1 - 0.1 * (0.1533)$

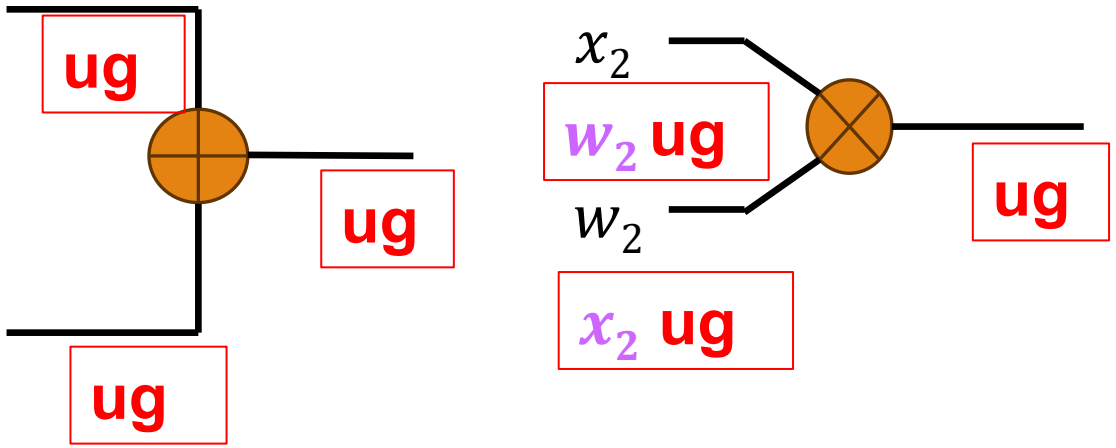
$b_1 \leftarrow 0.1 - 0.1 * (-0.0202)$

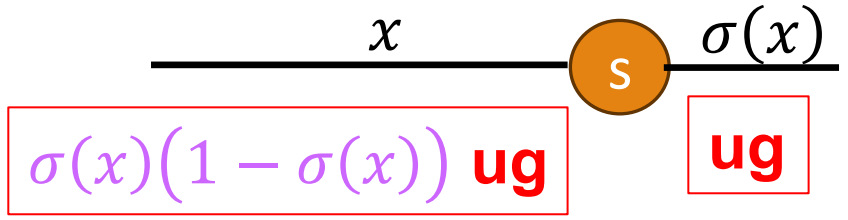
$b_2 \leftarrow 0.3 - 0.1 * (0.2704)$

$b_3 \leftarrow 0.05 - 0.1 * (-0.2187)$

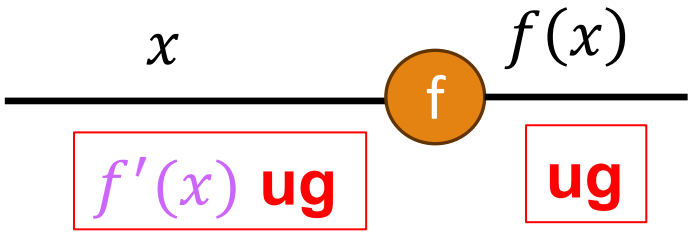
learning rate α =0.1

Backpropagation





Local gradient * Upstream gradient

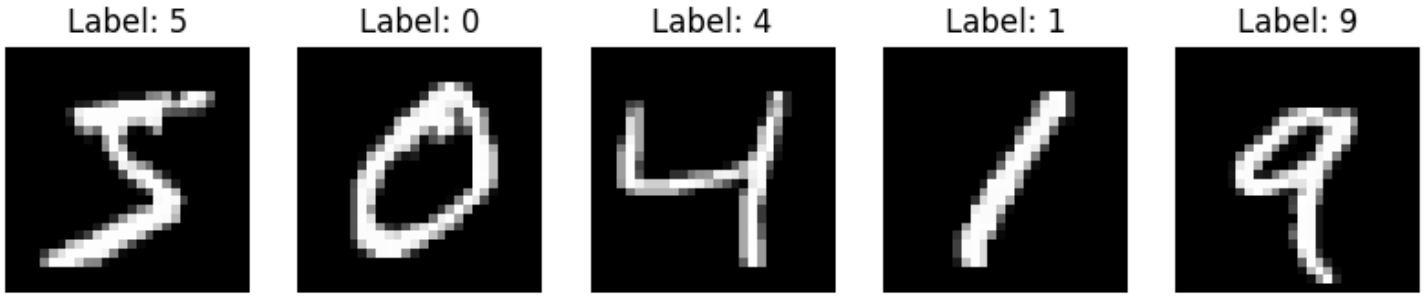


e. g., $f(x)$ relu, tanh

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

Code example

FFNN for image classification



28x28 input images

```
# Define the Sequential model
model = keras.Sequential([
    # Flatten the 28x28 input images to a 1D array (784 elements)
    layers.Flatten(input_shape=(28, 28)),

    # Add a dense (fully connected) layer with 128 units and ReLU activation
    layers.Dense(128, activation='relu’),

     # Add a dense (fully connected) layer with 64 units and ReLU activation
    layers.Dense(64, activation='relu'),

    # Add another dense layer with 10 units (one for each digit) and softmax activation
    layers.Dense(10, activation='softmax')
])

learning_rate=0.0001
optimizer = keras.optimizers.Adam(learning_rate=learning_rate)
model.compile(optimizer=optimizer, loss='sparse_categorical_crossentropy’, metrics=['accuracy'])
```

https://colab.research.google.com/drive/17uTRV73lRhO_xusg1QphatnUpmbBx7qQ?usp=sharing

Code example

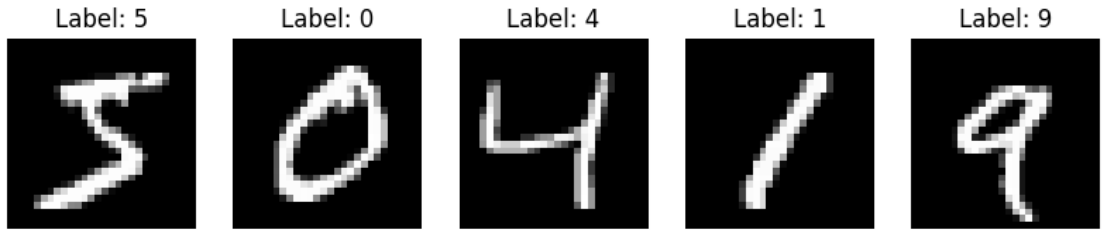
```
_________________________________________________________________
Layer (type) Output Shape Param #
=================================================================
flatten_8 (Flatten) (None, 784) 0
_________________________________________________________________
dense_18 (Dense) (None, 128) 100480
_________________________________________________________________
dense_19 (Dense) (None, 64) 8256
_________________________________________________________________
dense_20 (Dense) (None, 10) 650
=================================================================
Total params: 109,386
Trainable params: 109,386
Non-trainable params: 0
```

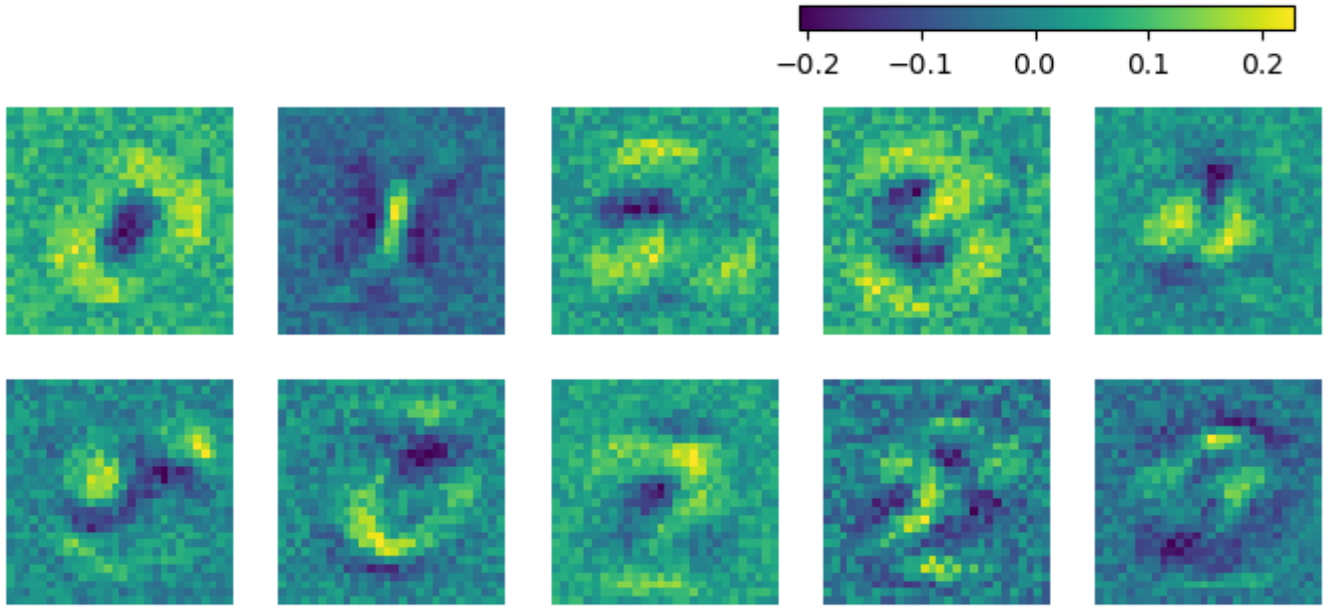
784 x 128 + 128

64 x 128 + 64

64 x 10 + 10

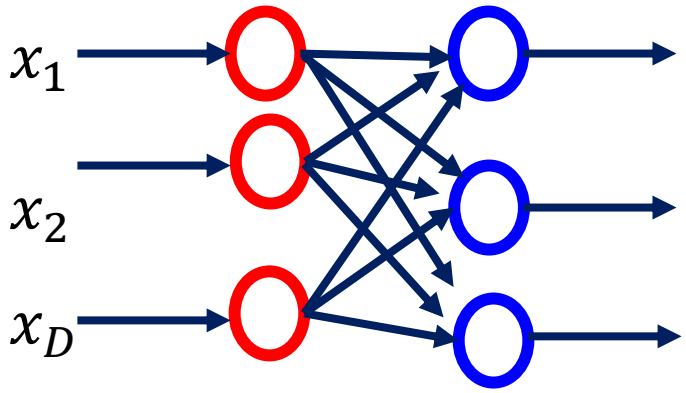
Code example





Learned weights (Only input and output layer)

Layer (type)	Output Shape	Param #
flatten_24 (Flatten)	(None, 784)	0
dense_58 (Dense)	(None, 10)	7850



Useful example for beginners

1. Learn the Basics — PyTorch Tutorials 2.1.0+cu121 documentation
2. Building Models with PyTorch
3. What is *torch.nn really*?
4. Build the Neural Network
5. PyTorch Examples

1. Getting started (keras.io)
2. Simple MNIST convnet
3. Tutorials | TensorFlow Core

Backpropagation

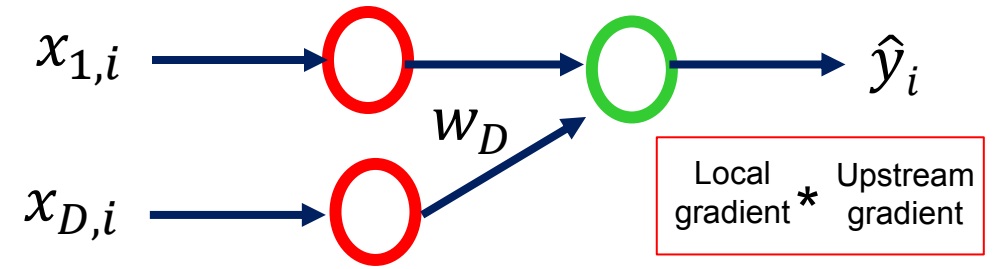
Additional reading

1. “**Yes you should understand backprop”** by Andrej Karpathy
2. CS231n Lecture 04 (Stanford)

Vanishing gradients

Sigmoid $\sigma(x)$

$\sigma'(x) = \sigma(x)\big(1 - \sigma(x)\big)$



$a_1 = \mathbf{w}^T \boldsymbol{x_i} + b$

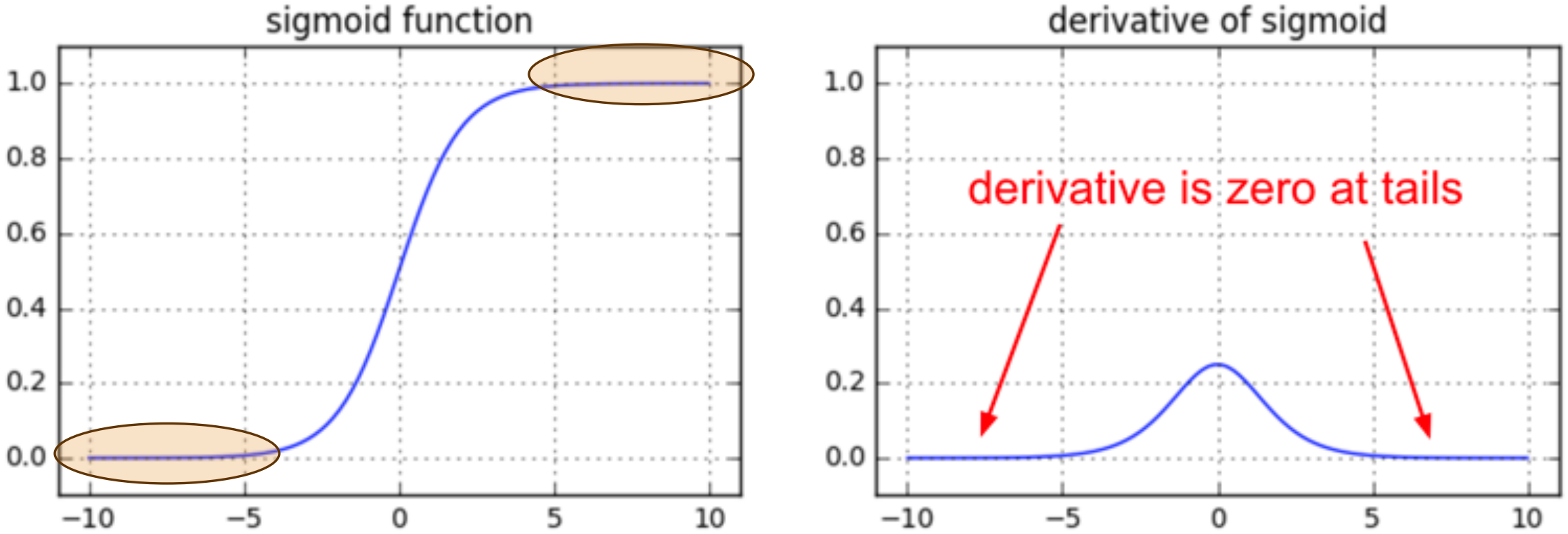
$\hat{y}_i = f(a_1) = \sigma\ (a_1) = \frac{1}{1+e^{-a_1}}$

$\sigma(x)\big(1 - \sigma(x)\big) \geq 0$

- If we initialize **w** to be large
- $\sigma(x) \approx 1\ or\ \sigma(x) \approx 0$
- $\big(1 - \sigma(x)\big) \approx 0\ or\ \sigma(x) \approx 0$
- $\sigma'(x) \approx 0$

The vanishing gradient problem occurs when gradients become extremely small as they flow backward through the layers of a neural network, causing early layers to learn very slowly or not at all because their weights are hardly updated.

Vanishing gradients



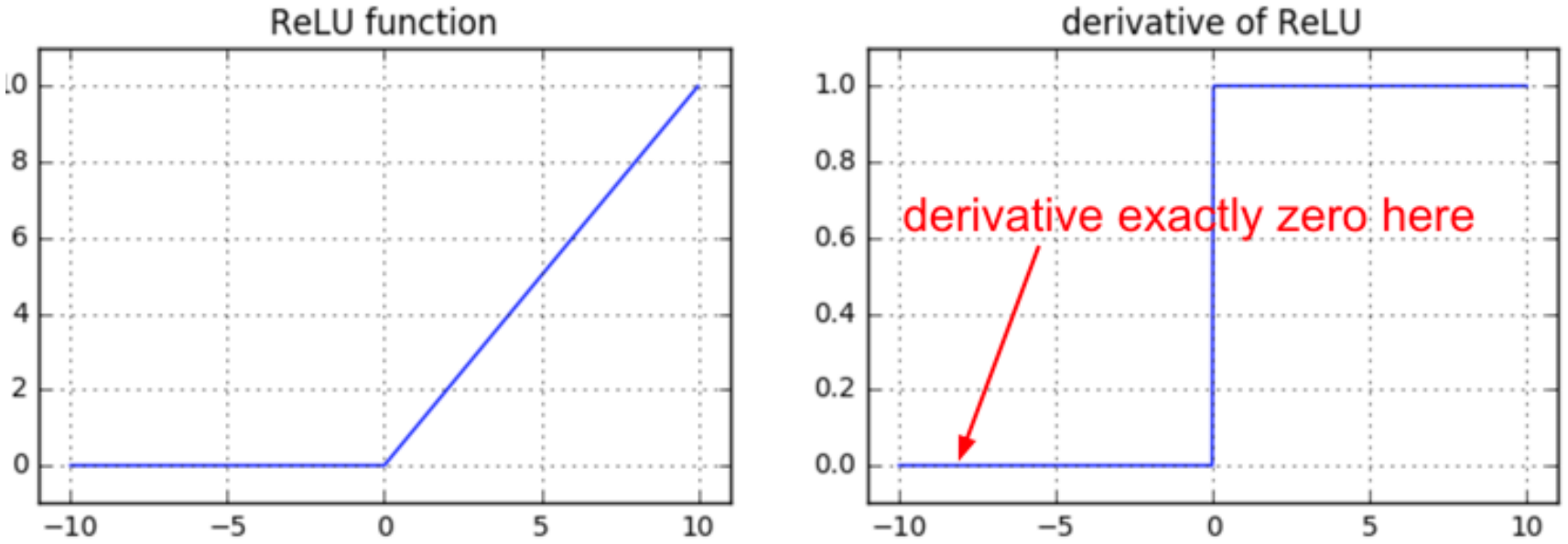
If you're using sigmoid or tanh activation functions in your neural network and, you should always be cautious about the initialization of weights to prevent these activations from becoming fully saturated.

Source: "Yes you should understand backprop" by Andrej Karpathy

CS231n lecture video

Vanishing gradients

- ReLu solve this to some extend. It does not saturate for positive values.
- Computationally efficient (thresholding).

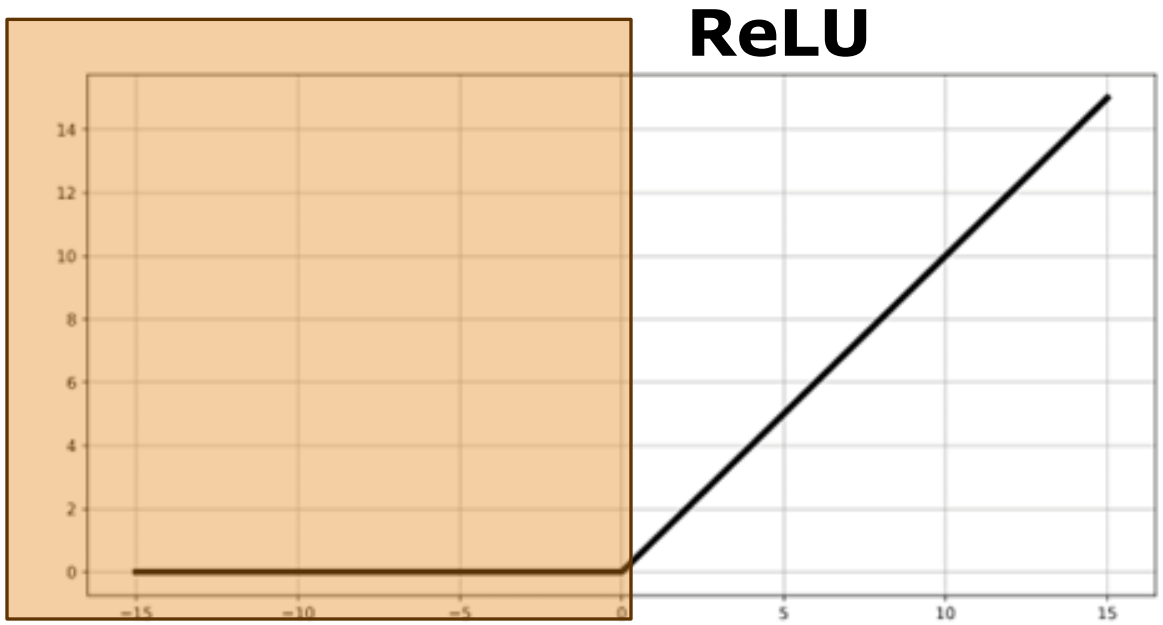


Source: "Yes you should understand backprop" by Andrej Karpathy

CS231n lecture video

Vanishing gradients

- Dying ReLUs: NN get stuck during training and always output zero, effectively becoming inactive neurons. No gradient update.
- Not zero centered



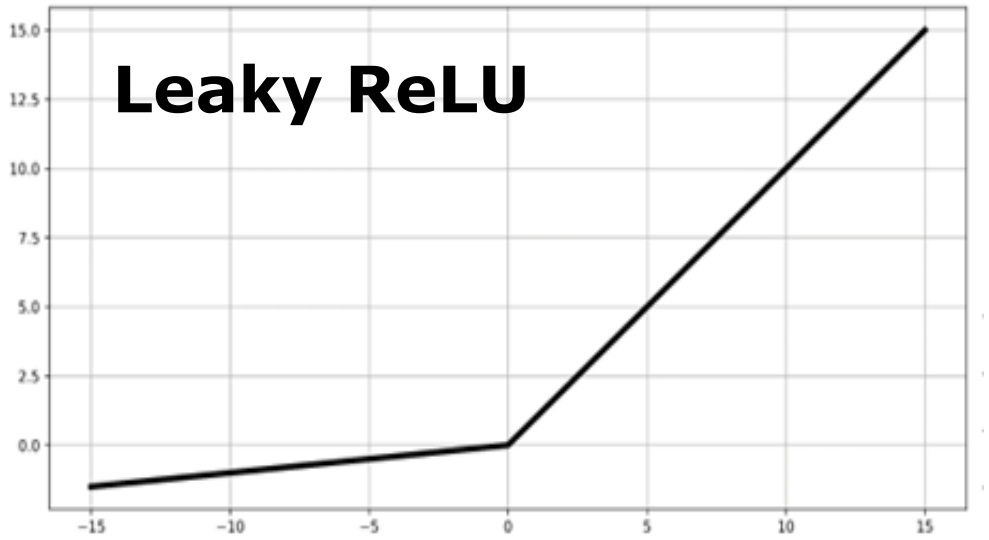
- Recommended for most cases

Source: "Yes you should understand backprop" by Andrej Karpathy

CS231n lecture video

Vanishing gradients

- Not saturate
- Neurons not die
- Computational efficient



$$\mathrm{LReLU}(x, \alpha) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & \text{otherwise} \end{cases}$$

Parametric Rectified Linear Unit (PReLU)

α is a learnable parameter that can be adjusted during training through backpropagation.

Source: "Yes you should understand backprop" by Andrej Karpathy

CS231n lecture video

Optimization algorithms

- Stochastic Gradient Descent (SGD)
- Stochastic Gradient Descent (SGD) with momentum
- Adam (Adaptive Moment Estimation)
- Root Mean Square Propagation (RMSprop)
- Adagrad
- Adadelta

torch.optim — PyTorch 2.0 documentation

Optimizers (keras.io)

Optimization algorithms

- Stochastic Gradient Descent (SGD)

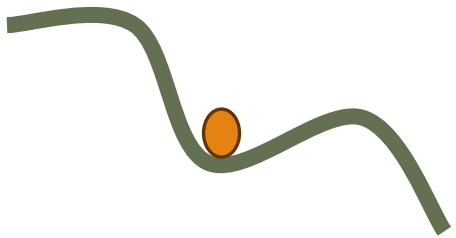
Usually slow to converge

$$w \leftarrow w - \alpha \frac{\partial L(\mathbf{w})}{\partial w}$$

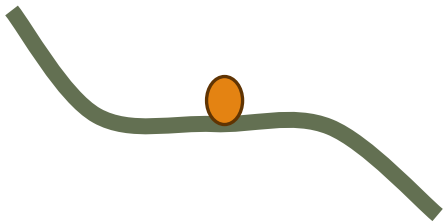
```
opt = tf.keras.optimizers.SGD(learning_rate=0.1)
```

- SGD can get stuck in local minima, making it slow to converge to a global minimum.

Local minima



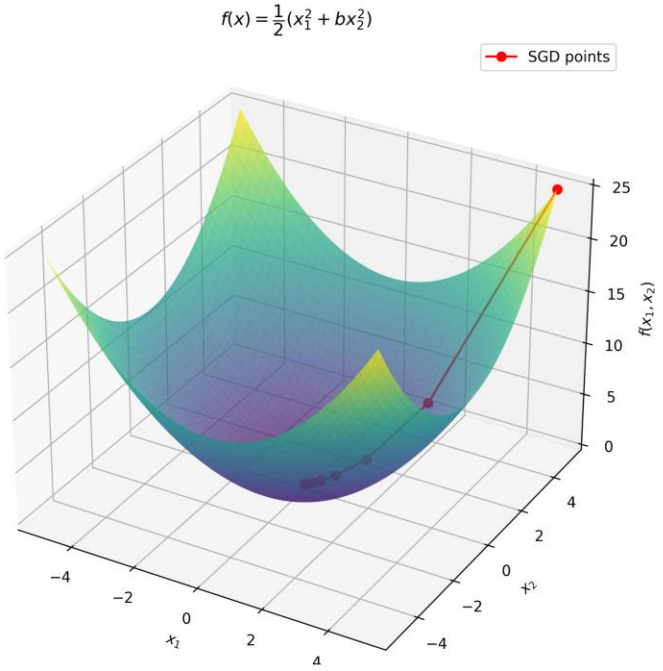
Saddle points



Slide credit: Prof. Svetlana Lazebnik

Optimization algorithms

Gradient descent : challenges

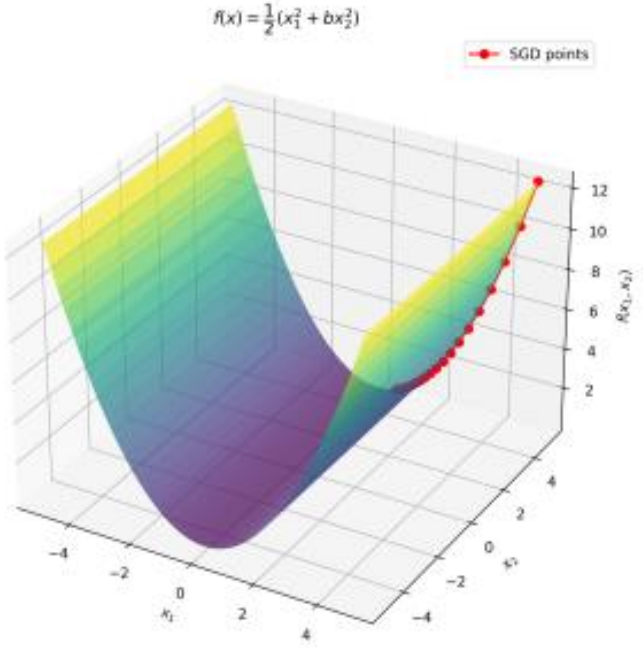


$[x_1$ update, x_2 update]

```
[5.         , 5.         ]
[4.5        , 4.5        ]
[4.05       , 4.05       ]
[3.645      , 3.645      ]
[3.2805     , 3.2805     ]
[2.95245    , 2.95245    ]
[2.657205   , 2.657205   ]
[2.3914845  , 2.3914845  ]
[2.15233605, 2.15233605]
[1.93710244, 1.93710244]
[1.7433922 , 1.7433922 ]
[1.56905298, 1.56905298]
[1.41214768, 1.41214768]
[1.27093291, 1.27093291]
[1.14383962, 1.14383962]
[1.02945566, 1.02945566]
[0.92651009, 0.92651009]
[0.83385908, 0.83385908]
[0.75047318, 0.75047318]
[0.67542586, 0.67542586]
[0.60788327, 0.60788327]
```

$f(\boldsymbol{x}) = \frac{1}{2}(x_1^2 + bx_2^2)$ with $b = 1$

$\boldsymbol{x}_0 = [x_{1,0}, x_{2,0}] = [5, 5]$



$f(\boldsymbol{x}) = \frac{1}{2}(x_1^2 + bx_2^2)$ with $b = 0.001$

$[x_1$ update, x_2 update]

```
[5.         , 5.         ]
[4.5        , 4.9995     ]
[4.05       , 4.99900005]
[3.645      , 4.99850015]
[3.2805     , 4.9980003 ]
[2.95245    , 4.9975005 ]
[2.657205   , 4.99700075]
[2.3914845  , 4.99650105]
[2.15233605, 4.9960014 ]
[1.93710244, 4.9955018 ]
[1.7433922 , 4.99500225]
[1.56905298, 4.99450275]
[1.41214768, 4.9940033 ]
[1.27093291, 4.9935039 ]
[1.14383962, 4.99300455]
[1.02945566, 4.99250525]
[0.92651009, 4.992006  ]
[0.83385908, 4.9915068 ]
[0.75047318, 4.99100765]
[0.67542586, 4.99050855]
[0.60788327, 4.99000949]
```

"This can be slow, especially when some directions of space point steeply downhill, whereas other have a shallower gradient [1]"

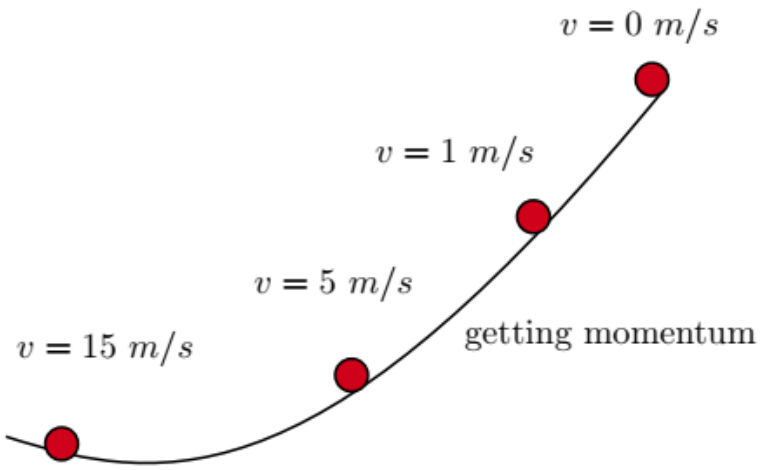
[1] Kevin P. Murphy, *Machine Learning: A Probabilistic Perspective*, 2012.

Optimization algorithms

- Stochastic Gradient Descent (SGD) with momentum

$$w \leftarrow w + \mathrm{v} \text{ and } \mathrm{v} \leftarrow \mathrm{mv} - \alpha \frac{\partial L(\mathbf{w})}{\partial w}$$

- Typically start m=0.5 and increase



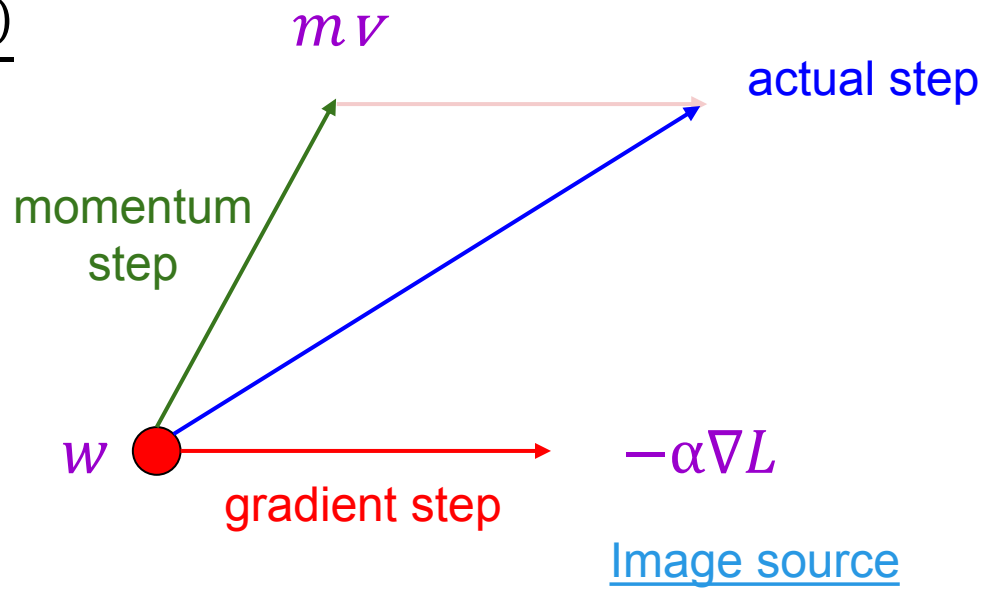


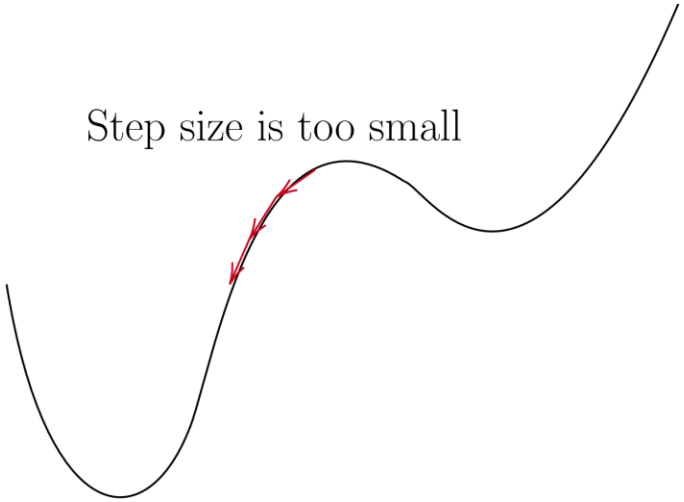
Image source

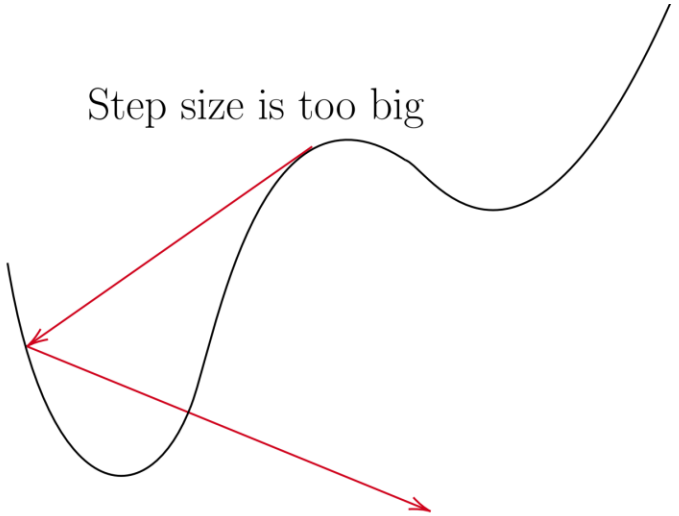
opt = tf.keras.optimizers.SGD(0.1, momentum=0.9)

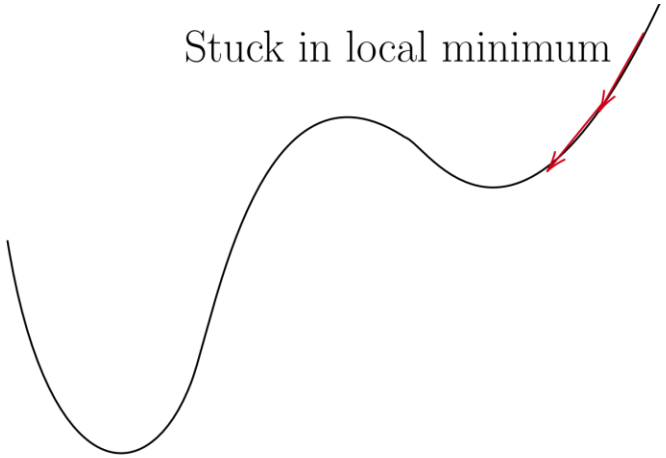
Slide credit: Prof. Svetlana Lazebnik

On the importance of initialization and momentum in deep

Optimization algorithms







Impact of the step size (learning rate) α

Is fixed step size (learning rate) way to go ?

Optimization algorithms

- Adagrad: Adaptive per-parameter learning rates
- For each dimension k of the weight vector:

$$v^{(k)} \leftarrow v^{(k)} + \left(\frac{\partial L}{\partial w^{(k)}}\right)^2$$

history of gradient magnitudes

$$w^{(k)} \leftarrow w^{(k)} - \frac{\alpha}{\sqrt{v^{(k)}+\epsilon}} \frac{\partial L}{\partial w^{(k)}}$$

Scaling the learning rate

Parameters with large gradients tend to get small learning rate (update slowly) and vise versa

Problem: As iterations goes on the learning rates for some parameters become very small, effectively slowing down or halting their updates

ϵ small positive number $10 - 7$

Slide credit: Prof. Svetlana Lazebnik

Duchi, Adaptive subgradient methods for online learning and stochastic optimization, JMLR 2011

Optimization algorithms

Root Mean Square Propagation (RMSprop)

- Address the learning rates become infinitesimally small before the optimization process converges of Adagrad
- Propose a decay factor $\beta \in [0, 1]$ (typically ≥ 0.9)
- For each dimension k of the weight vector:

$$v^{(k)} \leftarrow \beta v^{(k)} + (1 - \beta)\left(\frac{\partial L}{\partial w^{(k)}}\right)^2$$

Focuses on more recent gradients, forgets very old values

$$w^{(k)} \leftarrow w^{(k)} - \frac{\alpha}{\sqrt{v^{(k)} + \epsilon}} \frac{\partial L}{\partial w^{(k)}}$$

Scaling the learning rate

```
opt = tf.keras.optimizers.RMSprop(learning_rate=0.1)
```

Slide credit: Prof. Svetlana Lazebnik

Optimization algorithms

- Adam (Adaptive Moment Estimation) (Adagrad + momentum)

  Momentum update: $m \leftarrow \beta_1 m + (1 - \beta_1)\nabla L$

$$v^{(k)} \leftarrow \beta_2 v^{(k)} + (1 - \beta_2)\left(\frac{\partial L}{\partial w^{(k)}}\right)^2$$

$$w^{(k)} \leftarrow w^{(k)} - \frac{\alpha}{\sqrt{v^{(k)}} + \epsilon} m^{(k)}$$

Hyper parameter β_1 , β_2

Slide credit: Prof. Svetlana Lazebnik

Adam: A Method for Stochastic Optimization (arxiv.org)

Optimization algorithms

- What optimizer to use?

Adaptive optimization methods tend to achieve faster initial reductions in training error compared to SGD and are often considered to be more stable or robust during training.

Andrej Karpathy: *"In the early stages of setting baselines I like to use Adam with a learning rate of 3e-4. In my experience Adam is much more forgiving to hyperparameters, including a bad learning rate. For ConvNets a well-tuned SGD will almost always slightly outperform Adam, but the optimal learning rate region is much more narrow and problem-specific."*

- Adam + SGD

A common approach is to start with Adam for a certain number of initial epochs (e.g., 5-10 epochs) to achieve rapid convergence and then switch to SGD with a smaller fixed learning rate or a learning rate schedule for the remainder of the training process.

"We observe that evaluating multiple optimizers with default parameters works approximately as well as tuning the hyperparameters of a single, fixed optimizer." See for more detail Schmidt et al., 2021

Slide credit: Prof. Svetlana Lazebnik





Thank You

Q & A